

Full length article

VARTOOLS: A program for analyzing astronomical time-series data[☆]J.D. Hartman^{*}, G.Á. Bakos

Princeton University, Department of Astrophysical Sciences, 4 Ivy Lane, Princeton, NJ, 08544, USA

ARTICLE INFO

Article history:

Received 3 March 2016

Accepted 21 May 2016

Available online 31 May 2016

Keywords:

Methods: data analysis

Methods: statistical

Time

Techniques: photometric

ABSTRACT

This paper describes the VARTOOLS program, which is an open-source command-line utility, written in C, for analyzing astronomical time-series data, especially light curves. The program provides a general-purpose set of tools for processing light curves including signal identification, filtering, light curve manipulation, time conversions, and modeling and simulating light curves. Some of the routines implemented include the Generalized Lomb–Scargle periodogram, the Box-Least Squares transit search routine, the Analysis of Variance periodogram, the Discrete Fourier Transform including the CLEAN algorithm, the Weighted Wavelet Z-Transform, light curve arithmetic, linear and non-linear optimization of analytic functions including support for Markov Chain Monte Carlo analyses with non-trivial covariances, characterizing and/or simulating time-correlated noise, and the TFA and SYSREM filtering algorithms, among others. A mechanism is also provided for incorporating a user's own compiled processing routines into the program. VARTOOLS is designed especially for batch processing of light curves, including built-in support for parallel processing, making it useful for large time-domain surveys such as searches for transiting planets. Several examples are provided to illustrate the use of the program.

© 2016 The Author(s). Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license

(<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

1. Introduction

Measuring the variation over time in the apparent brightness of a distant object is one of the primary techniques that an astronomer may use to study the universe. A few classic examples include using supernovae or pulsating variable stars as standard candles in setting the cosmic distance ladder (e.g. [Hubble, 1929](#); [Riess et al., 1998](#); [Perlmutter et al., 1999](#)), using eclipsing binaries to measure the fundamental physical properties of stars (e.g. [Kopal, 1959](#); [Torres et al., 2010](#)), observing microlensing events to constrain the population of MAssive Compact Halo Objects (MACHOs) in the Galaxy or to identify extrasolar planets (e.g. [Paczynski, 1986](#); [Bond et al., 2004](#)), and finding extrasolar planets by searching for stars that show periodic transit events (e.g. [Henry et al., 1999](#); [Charbonneau et al., 2000](#); [Konacki et al., 2003](#)).

Developments in technology, together with scientific interest in finding rare types of variability, have led to numerous massive time domain surveys, especially in the optical and infrared bandpasses (see for example [Bakos et al., 2013](#), and references therein). Adding the time dimension to a survey leads to a massive flow of data, and

a need for specialized software to analyze it. This need is expected to grow even larger as new time-domain surveys (e.g., *LSST*, *TESS*, *PLATO*, *HATPI*, etc.) continue to come online.

This paper describes VARTOOLS, a program which provides a suite of tools for processing light curves, and which is designed in particular for handling large datasets. An early version of VARTOOLS, which included only a handful of period-finding tools, was briefly described in [Hartman et al. \(2008\)](#) who used it in analyzing the data from a variability survey of the open cluster M37. Since that time the program has been significantly altered, motivating the more detailed and up-to-date description provided here.

In addition to the numerous standalone software packages available which perform particular analysis tasks on light curves (e.g., transiting planet and eclipsing binary identification and modeling software; [Kovács et al., 2002](#); [Devor, 2005](#)), there are several other software suites available for general astronomical light curve analysis. Notable examples include PyKE ([Still and Barclay, 2012](#)),¹ the PERANSO period analysis program ([Paunzen and Vanmunster, 2016](#)), the PERIOD04 program ([Lenz and Breger, 2005](#)), WQED which was developed for the Whole Earth Telescope project ([Thompson and Mullally, 2009](#)), and the open-source VSTAR program provided by the American Association of Variable Star Observers.²

[☆] This code is registered at the ASCL with the code entry ascl:1208.016.^{*} Corresponding author.E-mail address: jhartman@astro.princeton.edu (J.D. Hartman).¹ <http://keplerscience.arc.nasa.gov/PyKE.shtml>.² <http://www.aavso.org/vstar-overview>.

The VARTOOLS programs differs from these in a few key ways. First, it provides more general purpose analysis routines than the other programs. Second, unlike the other programs, it is designed specifically for processing large numbers of light curves. This includes the ability to seamlessly pass light curves between commands without requiring intermediate temporary files, not requiring any human interaction to process each light curve, and native support for parallel processing of light curves.

General signal processing packages exist for data analysis platforms such as MATLAB, R, IDL, or PYTHON, but these are typically not tailored for astronomical purposes, and in general are collections of functions (like PYKE), requiring the user to solve a variety of bookkeeping problems, and in-so-doing to write a fair amount of code, to apply them to large astronomical time domain datasets. All of these platforms also allow executing system commands, so VARTOOLS may be incorporated into pipelines built for these platforms as well.

The following section provides an overview of the program, including a general description of its operation, and discussions of the processing commands and control options that are provided; in Section 3 several examples are provided; performance tests are presented in Section 4; possible areas for future development are discussed in Section 5. The appendices include comprehensive listings of the input and output syntax and data formats.

2. The VARTOOLS program

The basic operation of VARTOOLS is to read-in a light curve, apply one or more “commands” (i.e. processing routines) to the light curve, and output resulting statistics to an ASCII table for later analysis. The program is written in ANSI C, with regular updates provided on the web.³ This article describes version 1.33 of the program, a static copy of which is preserved on the website. To allow maximum portability, it can be compiled with basic functionality without any non-standard external libraries, however a number of features have library dependencies (including the CFITSIO library, Pence, 1999⁴; the GNU scientific library, or GSL, Galassi et al., 2009⁵; and the JPL NAIF CSPICE library, Acton, 1996⁶). Compilation and installation are carried out using the GNU Build System⁷ and has been successfully carried out on a number of system architectures (Linux, Mac and Windows). While VARTOOLS may be used to analyze an individual light curve of interest, it has been developed primarily to be included in pipelines which conduct batch processing of large numbers of light curves.

As a simple example, listing 1 shows the use of VARTOOLS to calculate the average, standard deviation, and expected standard deviation of the magnitudes in the light curve stored in the file “EXAMPLES/1” (this ASCII text file is included in the distribution of the program).

Listing 1: A simple example of running VARTOOLS. In this example, and others shown in this paper, the text following “prompt>” is typed by the user into the shell. In all examples in this paper we assume a BASH environment. Note that the “\” character at the end of the line continues the command over multiple lines; the “-oneline” argument, which causes one output statistic to be printed per line, is thus also part of the same command. The last five lines

(beginning with “Name”) are the output of the program. Colors are used to highlight commands and options passed to VARTOOLS.

```
1 prompt> vartools -i EXAMPLES/1 -rms \
2           -oneline
3
4 Name           = EXAMPLES/1
5 Mean_Mag_0     = 10.24745
6 RMS_0          = 0.15944
7 Expected_RMS_0 = 0.00101
8 Npoints_0      = 3122
```

Alternatively, we can apply this to a list of light curves stored in “EXAMPLES/lc_list” (each line in this list contains the name of a light curve file to process), and run this using 32 parallel processes (listing 2). A number of more complicated examples are provided in Section 3.

The next subsection describes the general operation of the program, paying special attention to those features relevant for batch processing. The processing algorithms which are included as “commands” in VARTOOLS are discussed in Section 2.2, built-in methods for extending VARTOOLS with a user’s own processing routines are discussed in Section 2.3, and options which may be used to control the operation of the program are discussed in Section 2.4. Performance benchmark tests are provided in Section 4.

2.1. General operation

2.1.1. Input

A user may provide either a single light curve or an ASCII list of light curve files as input to the program. Each light curve is stored in a separate file, either in ascii text format (one measurement per row, with separate white-space delimited columns for the time, magnitude or flux, uncertainties, and other optional information), or as a binary FITS table. The input list file may also be used to provide light-curve dependent data (e.g., this could be the right ascension and declination coordinates of each star) as needed by the various processing commands invoked by the user. There is flexibility for the user to specify the format of the input files. The user may also provide a string giving a set of commands that are passed to the shell and applied to the input light curve files before being read-in by the program (e.g., this may be used to process compressed light curve files without having to store the uncompressed files on the disk).

2.1.2. Data flow

The input light curves are then sent to the commands in the order that they are invoked on the command-line. The user may choose from 51 built-in commands, as well as a potentially unlimited number of user-developed commands. In principle an arbitrary number of commands can be called in a single invocation of VARTOOLS; though in practice a machine-dependent limitation will be set by the amount of available memory. These commands generally have parameters which control their properties (e.g., the range of frequencies to search in a periodogram, or the period, epoch, planet size, etc., for injecting a transit signal into a light curve). Most commands allow a number of options for specifying how the parameters are to be determined. Typically a parameter can be specified on the command-line for all light curves, it can be read-in from a column in the input list (so that a different value can be used for each light curve), it can be set using the results from a previously executed command (e.g., the period found by one command can be used in subtracting a harmonic signal with another command), it can be set to the value of a variable (see Section 2.1.4), or equal to the result from evaluating an analytic expression.

³ <http://www.astro.princeton.edu/~jhartman/vartools.html>.

⁴ <http://heasarc.gsfc.nasa.gov/fitsio/fitsio.html>.

⁵ <http://www.gnu.org/software/gsl/>.

⁶ <http://naif.jpl.nasa.gov/naif/toolkit.html>.

⁷ http://www.gnu.org/software/automake/manual/html_node/GNU-Build-System.html.

Listing 2: Processing a list of light curves in parallel. All quantities for a particular light curve are output on a single line, the “...” indicates that additional output from the program is not being shown here, and the parallel processing causes the output light curves to be in a different order from the input list.

```
prompt> vartools -l EXAMPLES/lc_list -rms \
2      -header -numbercolumns -parallel 32
4 #1_Name 2_Mean_Mag_0 3_RMS_0 4_Expected_RMS_0 5_Npoints_0
   EXAMPLES/2 10.11802 0.03663 0.00102 3313
6 EXAMPLES/1 10.24745 0.15944 0.00101 3122
   EXAMPLES/10 10.87781 0.00236 0.00143 3974
8 ...
```

When batch processing hundreds of thousands (or more) of individual light curve files, often the speed of the process is limited by attempts to read-from or write-to the hard disk, rather than by computation speed. In such cases there is a significant advantage to eliminating redundant attempts to access the disk. This is one advantage of writing a pipeline in VARTOOLS, where a light curve is read into memory once and output to the disk only when explicitly requested (output is treated as a command, which may be invoked at any time in the pipeline). When a similar pipeline is written using a shell script to piece together many standalone processing programs, it is often the case that the programs are not capable of transmitting the light curves via memory, forcing redundant disk access.⁸ Moreover, scripting a pipeline around VARTOOLS is generally simpler than using a variety of different programs which often require different data formats, necessitating messy, and potentially bug-prone data manipulation steps within a script. While pipelines written in an interpreted language such as MATLAB, IDL, or PYTHON may also avoid redundant disk access, they generally have a slower execution than a compiled program like VARTOOLS. Moreover building a pipeline from scratch in an interpreted language may well require more development effort for the user than using VARTOOLS where all of the necessary bookkeeping is handled internally by the program.

2.1.3. Parallelization

One other significant feature of VARTOOLS is its built-in support for parallel-processing of light curves. In the present version this is limited to shared-memory parallel processing, which is appropriate for processing on a single multi-core machine. The parallelization is done using the standard POSIX threads library, included with most C compilers. VARTOOLS creates a number of threads specified by the user and assigns to each thread a light curve to fully process. Once that light curve is processed, the parent thread is notified, and a new light curve is assigned to the child thread. This is repeated until all light curves have been processed. The parallelization of individual commands (e.g. period searches) has not yet been implemented, but is under development. For the common case where the number of light curves exceeds the number of processors, the current parallelization implementation should be adequate. Note that when run in parallel mode, computed statistics are printed in the order that light curves finish processing. We use locks to prevent simultaneously writing out data for multiple light curves, with buffering of the output to minimize occurrences of multiple threads waiting for the same lock. As a result the ordering of light curves in the output table may not be the same as in the input list, and in general will differ each time the program is run.

⁸ In some cases this problem can be overcome by writing temporary files to shared memory rather than a physical disk.

2.1.4. Variables

VARTOOLS supports the use of variables on the command-line to refer to vector or scalar data read-in from the light curves or list files, or computed by commands. These variables can also be used in evaluating analytic expressions with the **-expr** command, in controlling the flow of the program with the **-if**, **-elif**, **-else** and **-fi** commands, for processing multiple data columns within a single light curve with the **-changevariable** command, or in fitting analytic functions to the light curves with the **-linfit** or **-nonlinfit** commands. The special variable names *id*, *t*, *mag*, and *err* are reserved for a string giving the name of the image that a measurement comes from, the time, magnitude (or flux) and magnitude (flux) uncertainty, respectively. Additionally the special variable *NR* refers to the integer record number of a point in a light curve (1 for the first observation, 2 for the second, etc.). Finally a list of special names are reserved for analytic functions, constants, and operators (these are listed in [Appendix C](#)).

2.2. Commands

Here the built-in processing commands supported by VARTOOLS are described. A detailed description of the expected input syntax is provided in [Appendix A](#), while a description of the quantities output by each command is provided in [Appendix B](#). The commands can be separated into several categories: (1) finding periodic signals; (2) calculating light curve statistics; (3) fitting models; (4) filtering; (5) simulating light curves; (6) manipulating light curves; and (7) controlling the flow of data through VARTOOLS. In addition to the built-in commands, VARTOOLS provides support for the dynamic inclusion of commands and functions developed by the user; this feature is described at the end of this section. Note that the choice of commands built-in to the current version of VARTOOLS has been determined primarily by the authors' own research. As such there is a bias towards specialized routines that are useful in dealing with data from transiting planet surveys, while some general routines useful for other types of time-series applications may not be present.

2.2.1. Period-finding algorithms

Several of the commands included in VARTOOLS are used to identify periodic signals in light curves. Each of these routines generates a spectrum giving the significance of a periodic signal as a function of frequency or period. The user can optionally output the spectrum of each light curve to a separate file, VARTOOLS will also identify peaks in the spectrum and report the period or frequency, a measure of the significance, and other relevant information to the output ascii table. Below is a synopsis of each of these commands:

2.2.1.1. -LS. Calculates the Generalized Lomb–Scargle (L–S) periodogram of a light curve ([Zechmeister and Kürster, 2009](#); [Lomb, 1976](#); [Scargle, 1982](#)) using the method of [Press and Rybicki \(1989\)](#) to speed up the calculation of various trigonometric sums (we use

the implementation of this “extirpolation” algorithm due to [Press et al., 1992](#)). The primary motivation for using the L–S method as opposed to directly applying the Fast Fourier Transform (FFT) to the light curve is that L–S handles nonuniformly sampled data (in astronomy this is the rule rather than the exception) without resorting to methods such as zero-padding or interpolation to a uniform grid, which amounts to making assumptions about the light curve at non-observed time instances. For this reason, this tool is widely used for identifying periodic signals in astronomy. The generalized periodogram extends the traditional L–S periodogram by allowing for a floating mean and non-uniform uncertainties (the user may optionally calculate the traditional periodogram instead; this is calculated following [Press et al., 1992](#)). The value of the periodogram at frequency f (in cycles per day) is given by equations 4 and 5 in [Zechmeister and Kürster \(2009\)](#), i.e.:

$$LS(f) = \frac{\chi_0^2 - \chi^2(f)}{\chi_0^2} \quad (1)$$

where χ_0^2 is the value of χ^2 using the weighted mean of the data as the model, and $\chi^2(f)$ is the value of χ^2 using the best-fit sinusoidal signal with frequency f as the model. This statistic varies between 0 (for no signal present at all) and 1 (for a perfectly sinusoidal signal).

In addition to the peak periodogram value, the VARTOOLS implementation of L–S outputs two measures of significance for peaks identified in the periodogram. These are the logarithm of the formal false alarm probability, and the signal-to-noise ratio (S/N).

There are several ways in which the false alarm probability may be estimated. We follow the [Cumming et al. \(1999\)](#) method described in section 3 of [Zechmeister and Kürster \(2009\)](#). In this method the signal is measured relative to the scatter in the residuals from the best-fit model (as opposed to using the scatter in the original light curve, or using the input measurement uncertainties, which are two other popular methods). Namely we take

$$z(f) = \frac{N-3}{2} \frac{LS(f)}{1 - LS_{\text{best}}} \quad (2)$$

where N is the number of observations in the light curve, and LS_{best} is the highest value of LS found in the periodogram. The false alarm probability is then given by

$$\text{FAP} = 1 - [1 - \text{Prob}(z > z_0)]^M \quad (3)$$

where the false alarm probability for an individual trial frequency is

$$\text{Prob}(z > z_0) = \left(1 + \frac{2z_0}{N-3}\right)^{-(N-3)/2} \quad (4)$$

and M is the number of independent frequencies sampled (sometimes called the “bandwidth” penalty). Here we differ slightly from [Zechmeister and Kürster \(2009\)](#) and use the [Horne and Baliunas \(1986\)](#) estimate of

$$M = 2f_{\text{max}}T \quad (5)$$

where f_{max} is the maximum frequency calculated in the periodogram, and T is the time spanned by the light curve. Note that the false alarm probability will change if you scan a larger or smaller frequency range—this is expected behavior. It is also important to note that this false alarm estimate assumes Gaussian white noise. For real data the noise is often correlated in time, or does not follow a Gaussian distribution. In such cases the true false alarm probability is often much higher than the calculated probability, and should be determined through Monte Carlo simulations. Also note that M is a simple estimate that may be off by as much as a few dex in cases where the sampling is highly non-uniform. For

applications requiring a false alarm probability accurate to better than 1 dex, it is necessary to conduct Monte Carlo simulations applying L–S to light curves with simulated noise to calibrate the FAP.

The S/N measure provided by VARTOOLS is determined within the periodogram as

$$S/N = (LS - \bar{LS})/\sigma_{LS} \quad (6)$$

where LS is the value of the periodogram for a given peak, $\bar{LS}$ is the mean across the periodogram, and σ_{LS} is the standard deviation of the periodogram. The user may optionally specify a clipping factor used to remove outlier points (peaks) from the values used in determining $\bar{LS}$ and σ_{LS} . The S/N value provides an alternative method for estimating the significance of a detection, however it is important to keep in mind that the “noise” in the periodogram does not follow a normal distribution, so the S/N itself does not follow a normal distribution (i.e., a spectroscopic S/N of 10 does not correspond to a 10σ detection). The user specifies the number of peaks to identify in the periodogram, these will be sorted based on the peak-height. If searching for multiple peaks, the user has the option to iteratively “whiten” the light curve after finding a peak (i.e., subtract the best-fit periodic sinusoidal signal from the light curve with the period fixed to the value identified in the periodogram), recompute the periodogram, and search for the next highest peak.

2.2.1.2. -GetLSAmpThresh. Though not a method for identifying a periodic signal per se, this command is closely related to **-LS** and so it is discussed in this section. For many applications it is important to know not only that a signal is present in a light curve, but also what the minimum amplitude the signal could have had and still have been detected within the noisy observed data. This information helps in characterizing the selection effects present in a survey, and is thus vital for determining the completeness of a survey. This command fits a multi-harmonic Fourier series to a light curve at the period found by **-LS**. The model is then subtracted from the light curve, its amplitude is scaled by a factor α , and then it is re-added to the residual. The Lomb–Scargle FAP is calculated at the fixed period for this new light curve. The value of α is varied until the resulting FAP is equal to a threshold value chosen by the user; the resulting α times the observed amplitude is reported to the user.

2.2.1.3. -dftclean. Calculates the Discrete Fourier Transform (DFT) of a light curve, and optionally applies the CLEAN deconvolution algorithm ([Roberts et al., 1987](#)). We follow the convention that the complex DFT of the set of points (t_j, m_j) , evaluated at frequency f is given by:

$$F(f) = \sum_{j=1}^N (m_j - \bar{m}) \exp(i2\pi f(t_j - \bar{t})) \quad (7)$$

and use the method of [Kurtz \(1985\)](#) to speed up the calculation. The user may also optionally output the window function (i.e. the DFT of the set of points $(t_i, 1)$ where t_i are the times of observation, and the DFT is applied without subtracting the average from the y values). The DFT procedure takes two parameters: *nbeam* which determines the frequency sampling of the spectrum via $\Delta f = 1/(T \times nbeam)$ where T is the time baseline of the observations, and the maximum frequency to calculate the DFT up to (the default is the Nyquist frequency given by $1/(2\Delta t_{\text{min}})$ where Δt_{min} is the minimum time separation between consecutive points in the light curve). The CLEAN routine is controlled by the gain and the S/N threshold for selecting peaks in the DFT. After the DFT is calculated the spectrum is searched for the most significant peak above the S/N threshold. This peak, times the gain, is added to the cleaned spectrum, while the window function, scaled appropriately and shifted to the peak frequency, is subtracted from the DFT. The procedure iterates until no peaks above the S/N threshold remain.

2.2.1.4. -aov. Calculates the Analysis of Variance (AoV) periodogram of a light curve, as defined by [Schwarzenberg-Czerny \(1989\)](#). The periodogram statistic follows a Fisher distribution of the form $F(x; r - 1, n - r)$ where x is the value of the AoV periodogram, there are n data points in the light curve, and r bins are used in the fit. As with all period searches, one must also take into account the number of independent periods that are tried. We use the [Horne and Baliunas \(1986\)](#) estimate for the band-width penalty. Work requiring a more exact determination of the false alarm probability should conduct Monte Carlo simulations (see for example [Paltani, 2004](#); [Schwarzenberg-Czerny, 2012](#)). Using the Fisher distribution to interpret the AoV statistic also naturally punishes overfitting the data. While the previous two commands use a sinusoid for the model signal, in this case the model which is fitted to the light curve is a discrete set of step functions (i.e., it is the phase-binned light curve). This method is thus comparable to the classical Phase Dispersion Minimization technique ([Stellingwerf, 1978](#)), but with a superior statistic. In principle **-aov** is more sensitive than **-LS** or **-dftclean** to light curves with sharp features, such as eclipsing binaries, for which a sinusoid is a poor approximation. However, because the phase-binned model is quite general (a relatively large number of free parameters must be used to fit typical signals), for most classes of variables the significance of the detection is lower when using **-aov** than another method such as **-aov_harm** or **-LS** (e.g. [Kovacs, 1980](#)).

2.2.1.5. -aov_harm. Similar to **-aov**, this command computes an AoV periodogram for a light curve, however it uses a multi-harmonic Fourier series model for the signal, rather than the phase-binning model used by **-aov**. The fit is done using a projection onto orthogonal complex polynomials as described by [Schwarzenberg-Czerny \(1996, 2012\)](#). Here one may also use the AoV statistic to optimize the number of harmonics to include in the fit.

2.2.1.6. -BLS. Computes a Box-Least Squares (BLS; [Kovács et al., 2002](#)) spectrum for a light curve. Unlike the other period-finding commands, **-BLS** does not search for signals that are continuously variable over the full phase curve, but instead searches for periodic box-shaped dips in a light curve. This algorithm provides an efficient means of identifying detached eclipsing binary light curves and transiting planet signals. The algorithm operates by scanning through frequencies, phasing the light curve at each frequency, and fitting to the phased light curve a model of the form

$$m(\phi) = \begin{cases} \bar{m}, & \phi < \phi_0 \text{ or } \phi > \phi_0 + q \\ \bar{m} + \delta_m, & \phi_0 < \phi < \phi_0 + q \end{cases} \quad (8)$$

for $\phi_0 + q \leq 1$, or

$$m(\phi) = \begin{cases} \bar{m}, & \phi_0 + q - 1 < \phi < \phi_0 \\ \bar{m} + \delta_m, & \phi < \phi_0 + q - 1 \text{ or } \phi > \phi_0 \end{cases} \quad (9)$$

for $\phi_0 + q > 1$. Here ϕ is the phase, $\bar{m}$ is the average magnitude of the light curve, and δ_m , ϕ_0 and q are free parameters representing the transit depth, starting phase of the transit, and transit duration in phase, respectively. The fit for the free parameters is done by binning the phased light curve into N_{bin} bins, trying each of these bins for ϕ_0 and trying discrete values for q of the form i/N_{bin} where i is an integer that varies between $i_{\text{min}} = \text{floor}(q_{\text{min}}N_{\text{bin}})$ and $i_{\text{max}} = \text{ceil}(q_{\text{max}}N_{\text{bin}})$, and q_{min} and q_{max} are set by the user. The user may optionally fit a simple “trapezoid” transit to the light curve for each of the significant periods found in the BLS spectrum. In this case the fractional duration of ingress and egress (i.e., in units of the total transit duration) is an additional free parameter in the fit. The trapezoid model is fit using the Downhill Simplex method (DHSX; [Nelder and Mead, 1965](#)) with initial parameter values for the frequency, transit depth, out-of-transit magnitude, transit time and transit duration taken from the BLS peak.

The basic statistic computed by the BLS algorithm is the Signal Residue (SR) as a function of trial transit frequency, defined by [Kovács et al. \(2002\)](#) as

$$\text{SR}(f) = \max_{\phi_0, q} \left\{ \left[\frac{s^2(\phi_0, q, f)}{r(\phi_0, q, f)(1 - r(\phi_0, q, f))} \right] \right\} \quad (10)$$

where

$$s^2(\phi_0, q, f) = \sum_{\text{transit}} \bar{w}_i \bar{x}_i \quad (11)$$

is the weighted sum of phased, weighted-mean-subtracted magnitudes $\bar{x}_i$ within a trial transit (i.e., points with phase ϕ for a given frequency f , such that $\phi_0 < \phi < \phi_0 + q$ if $\phi_0 + q \leq 1$, or $\phi < \phi_0 + q - 1$ or $\phi > \phi_0$ otherwise), $\bar{w}_i$ is the weight for point i normalized such that the sum of all weights in the light curve is unity, and we have $r(\phi_0, q, f) = \sum_{\text{transit}} \bar{w}_i$. The maximum in Eq. (10) is taken over all pairs of ϕ_0 and q considered for a given transit frequency f . The function $\text{SR}(f)$ is also called the BLS spectrum. [Kovács et al. \(2002\)](#) also define the Signal Detection Efficiency (SDE) for a given peak in the BLS spectrum, given by

$$\text{SDE} = \frac{\text{SR}_{\text{peak}} - \bar{\text{SR}}}{\sigma_{\text{SR}}} \quad (12)$$

where $\bar{\text{SR}}$ is the average value of the BLS spectrum, and σ_{SR} is its standard deviation.

In many cases time-correlated noise leads to a BLS spectrum which slowly rises with decreasing frequency. To account for this, by default the VARTOOLS program ranks peaks by a modified version of the SDE, called the spectroscopic signal to noise ratio (S/N), and calculated as follows. We first define

$$\tilde{\text{SR}}(f) = \text{mean}_{\phi_0, q} \left\{ \left[\frac{s^2(\phi_0, q, f)}{r(\phi_0, q, f)(1 - r(\phi_0, q, f))} \right] \right\} \quad (13)$$

where the mean is computed, for a given frequency, over all trial combinations of ϕ_0 and q , and the quantity being averaged is the same quantity that is maximized in computing SR. A 3σ clipping is first performed before taking the mean. One may interpret $\tilde{\text{SR}}(f)$ as the average Signal Residue due to noise at a given frequency. The S/N is then taken to be

$$\text{S/N}(f) = \frac{\text{SR}(f) - \tilde{\text{SR}}(f)}{\sigma_{\tilde{\text{SR}}}} \quad (14)$$

where in this case $\tilde{\text{SR}}(f)$ is the average value of $\tilde{\text{SR}}$ computed over a frequency range from $f - 100\Delta f$ to $f + 100\Delta f$ with Δf being the frequency sampling of the spectrum, and $\sigma_{\tilde{\text{SR}}}$ is the standard deviation of $\tilde{\text{SR}}$ taken over the full spectrum. We perform an iterative 3σ clipping before computing the average as a function of frequency and the standard deviation. This determination of S/N is similar to the SDE, except that it is the frequency-localized signal enhancement, relative to the global noise in the spectrum. Note that because SR does not follow a Gaussian distribution, this definition of S/N should not be interpreted using such a distribution. The user may optionally use SR and σ_{SR} in Eq. (14) rather than $\tilde{\text{SR}}(f)$ and $\sigma_{\tilde{\text{SR}}}$, in which case the S/N is the same as the SDE, except for the clipping applied in computing the average and standard deviation.

Another statistic which we calculate for each transit, and which may be used to select transit candidates, is the signal-to-pink noise ratio (S/N_{pink}) defined, following [Pont et al. \(2006\)](#), as:

$$\text{S/N}_{\text{pink}} = \frac{\delta_m^2}{(\sigma_w^2/n_t) + (\sigma_r^2/N_t)} \quad (15)$$

where σ_w is the “white noise” r.m.s., σ_r is the “red noise” r.m.s., n_t is the number of points in transit, and N_t is the number of distinct transits sampled. We take σ_w to be equal to the r.m.s. of the light curve after subtracting the transit model, while we estimate σ_r using the expression

$$\sigma_r^2 = \sigma_{\text{bin}}^2 - \sigma_{\text{bin,thy}}^2 \quad (16)$$

where σ_{bin} is the r.m.s. of the residual light curve after binning in time with a bin-size equal to the duration of a transit, and $\sigma_{\text{bin,thy}}$ is the expected r.m.s. of the binned light curve if the noise were uncorrelated in time. In the event that the estimate for σ_r^2 is less than zero, we take $\sigma_r = 0$.

2.2.1.7. -BLSFixDurTc. Runs BLS on a light curve fixing the duration of the transit and the reference time of transit. Optionally the depth and the duration of ingress (for a trapezoidal-shaped transit) may be fixed as well. This may be used, for example, in a case where a single transit is observed with high precision (e.g., from space) and one wishes to search lower-precision data from another instrument for additional transits.

2.2.1.8. -BLSFixPer. Runs BLS on a light curve fixing the period (i.e., it finds the most transit-like signal in the light curve at a specified period). This may be useful for carrying out transit injection/recovery simulations to determine the detection efficiency where one does not wish to perform a full BLS search on the simulated light curves.

2.2.1.9. -wwz. Computes the weighted wavelet transform as defined by Foster (1996). This tool may be used to characterize signals with quasi-periodic behavior that evolve in time (in amplitude and/or frequency). The weighted wavelet transform is a discrete approximation to the continuous wavelet transform for the function $x(t)$ given by

$$W(\omega, \tau; x(t)) = \omega^{1/2} \int x(t) f^*(\omega(t - \tau)) dt \quad (17)$$

for frequency ω , time-shift τ and wavelet kernel $f(z)$. As shown by Foster (1996) the weighted wavelet transform is superior to the well known discrete wavelet transform for non-uniformly sampled time-series. Foster (1996) adopts the abbreviated Morlet wavelet (Goupillaud et al., 1984) for the kernel given by

$$f(z) = e^{i\omega(t-\tau) - c\omega^2(t-\tau)^2} \quad (18)$$

and approximates the integral using a weighted projection of the data onto the sinusoidal basis functions ($e^{i\omega(t-\tau)}$) with weights $e^{-c\omega^2(t-\tau)^2}$. That is, the data are modeled as

$$y(t) = \sum_a y_a \phi_a(t) \quad (19)$$

where the basis functions are

$$\phi_1(t) = 1 \quad (20)$$

$$\phi_2(t) = \cos(\omega(t - \tau)) \quad (21)$$

$$\phi_3(t) = \sin(\omega(t - \tau)) \quad (22)$$

and the coefficients y_a are given by

$$y_a = \sum_b S_{ab}^{-1} \langle \phi_b | x \rangle \quad (23)$$

where the weighted inner product of functions f and g is given by

$$\langle f | g \rangle = \frac{\sum_{\alpha=1}^N w_\alpha f(t_\alpha) g(t_\alpha)}{\sum_{\beta=1}^N w_\beta} \quad (24)$$

with weights

$$w_\alpha = e^{-c\omega^2(t_\alpha - \tau)^2} \quad (25)$$

and the sums are over the observed times. The matrix S_{ab} is given by

$$S_{ab} = \langle \phi_a | \phi_b \rangle. \quad (26)$$

Foster (1996) defines the Weighted Wavelet Transform (WWT) as

$$WWT = \frac{(N_{\text{eff}} - 1)V_y}{2V_x} \quad (27)$$

which is a χ^2 statistic with 2 degrees of freedom with expected value 1. Foster (1996), however, suggests using the Z statistic (or Weighted-Wavelet Z-Transform, or WWZ) given by

$$Z = \frac{(N_{\text{eff}} - 3)V_y}{2(V_x - V_y)} \quad (28)$$

to characterize the significance of a signal with frequency ω and time-shift τ in the observed data. In both of these relations

$$N_{\text{eff}} = \frac{\left(\sum w_\alpha\right)^2}{\sum w_\alpha^2} \quad (29)$$

is the effective number of data-points contributing to the signal, and

$$V_x = \langle x|x \rangle - \langle 1|x \rangle^2 \quad (30)$$

and

$$V_y = \langle y|y \rangle - \langle 1|y \rangle^2 \quad (31)$$

are the weighted variations of the data and model. The amplitude of the signal (dubbed the Weighted Wavelet Amplitude, or WWA) is given by

$$WWA = \sqrt{(y_2)^2 + (y_3)^2}. \quad (32)$$

Given a value of c (by default $c = 1/8\pi^2$ is assumed), the **-wwz** command scans through frequency and time-shift (in keeping with the rest of the program we express frequency in cycles per day, whereas the frequency ω appearing in the equations above is in radians per day) computing the y_a coefficients, WWT, WWZ, WWA and N_{eff} . The user may optionally output this full transform (i.e., each of these quantities for every frequency and time-shift combination that is tested) to a file (as an ascii table, or a multi-extension fits image) for each light curve analyzed. For each trial time-shift the frequency with the maximum value of WWZ is also identified (i.e., $f_{\text{max},\tau}$, and corresponding quantities $WWZ_{\text{max},\tau}$, $WWA_{\text{max},\tau}$, $N_{\text{eff,max},\tau}$, and $y_{a,\text{max},\tau}$), and may be output to a separate file (this gives effectively the maximum signal as a function of time in the light curve). The maximum of $WWZ_{\text{max},\tau}$ across all trial τ values is determined, and its associated frequency, amplitude, N_{eff} and model coefficients are included in the VARTOOLS output ascii table. We also compute the median values of the max quantities across τ and include these in the output ascii table as well.

2.2.2. Light curve statistics

There are eight built-in commands which calculate statistics useful for characterizing the variability of light curves.

2.2.2.1. -alarm. Calculates the statistic proposed by Tamuz et al. (2006) to detect variability. Let $\bar{m}$ be the weighted average

magnitude of the light curve defined by

$$\bar{m} = \frac{\sum_i m_i / \sigma_i^2}{\sum_i 1 / \sigma_i^2} \quad (33)$$

where the sum is over all points in the light curve. Then the alarm a is given by

$$a = \frac{\sum_k a_k^2}{\sum_i ((m_i - \bar{m}) / \sigma_i)^2} - 2.2732395 \quad (34)$$

where a_k is given by

$$a_k = \sum_j (m_j - \bar{m}) / \sigma_j \quad (35)$$

the sum on j is over the k th continuous set of points with the same value of $\text{sign}(m - \bar{m})$, the sum on k is over all such values of a_k , and the sum on i is over all points in the light curve. The constant term 2.2732395 is chosen such that the expected value of a is zero for Gaussian white noise. A large value of a indicates a light curve exhibiting correlated variability.

2.2.2.2. -autocorrelation. Calculates the Discrete Autocorrelation Function (DACF; [Edelson and Krolik, 1988](#)) for a light curve. This variant of the autocorrelation function handles non-uniformly sampled time series by forming all pairs of points m_i and m_j , calculating the correlation between the pairs via

$$C_{ij} = \frac{(m_i - \bar{m})(m_j - \bar{m})}{\sigma_i \sigma_j} \quad (36)$$

and then binning this in lag time τ , such that the DACF at lag τ_k is given by

$$DACF(\tau_k) = \frac{1}{M_k} \sum C_{ij}, \quad (37)$$

where the sum is over all ij pairs with lag $\tau_{ij} = |t_i - t_j|$ satisfying $|\tau_{ij} - \tau_k| < \Delta\tau/2$, for a fixed sampling $\Delta\tau$, and M_k is the number of these pairs. Eq. (36) differs from the corresponding equation in [Edelson and Krolik \(1988\)](#) in that we use the product of the measurement uncertainties $\sigma_i \sigma_j$ in the denominator, rather than $(\sigma^2 - \epsilon^2)$ with σ being the standard deviation of the time series, and ϵ being a “measurement error” associated with the data-set, and included to preserve the normalization. This modification allows for heteroscedastic data, but as a consequence, the autocorrelation at time-lag $\tau = 0$ in general will not equal 1.

An uncertainty on this correlation is estimated by taking the standard error on the mean for the points in a bin:

$$\sigma_{DACF}(\tau_k) = \frac{1}{M_k - 1} \left\{ \sum [C_{ij} - DACF(\tau_k)]^2 \right\}^{1/2}. \quad (38)$$

The DACF will be written out to a separate file for each light curve analyzed. This algorithm is sometimes used to identify variability periods for non-stationary quasi-periodic signals by searching for peaks in the DACF, however it is most useful in characterizing the coherence time scale of features (either true astrophysical signals, or correlated noise) in the light curves. Currently it is left up to the user to separately analyze the output autocorrelation function.

2.2.2.3. -chi2. Calculates

$$\chi^2/\text{d.o.f.} = \frac{1}{N-1} \sum_{i=1}^N \left(\frac{m_i - \bar{m}}{\sigma_i} \right)^2, \quad (39)$$

the reduced χ^2 , for each light curve.

2.2.2.4. -chi2bin. Calculates the reduced χ^2 after applying a moving mean filter to each light curve. This filter operates by replacing each m_i in the light curve by the average value of m within a bin of size Δt . That is, m_i is replaced with

$$\frac{\sum_j m_j}{\sum_j 1} \quad (40)$$

where the sum is over points j such that $|t_j - t_i| < \Delta t$. The light curve uncertainties after applying the filter are reduced by the amount expected for white noise. That is

$$\sigma'_i = \frac{\sqrt{\sum_j \sigma_j^2}}{(\sum_j 1) - 1} \quad (41)$$

where σ'_i is the new value for the uncertainty, and the sum is over the same points as above. One result of this is that if the light curve has time correlated variations, the reduced χ^2 reported by **-chi2bin** will increase as Δt is increased. For pure white noise, the reduced χ^2 will be constant (to within statistical uncertainties) as Δt is increased.

2.2.2.5. -Jstet. Calculates the J light curve variability statistic suggested by [Stetson \(1996\)](#), together with the Kurtosis K and the L statistic also defined by [Stetson \(1996\)](#). The J statistic is similar to **-alarm** in that its value will be greater for a light curve that shows time-correlated variability than for a light curve with the same scatter, but for which the variations are not correlated in time. The J statistic is given by the expression

$$J = \frac{\sum_{k=1}^n w_k \text{sign}(P_k) \sqrt{|P_k|}}{\sum_{k=1}^n w_k} \quad (42)$$

where the sum in the numerator and denominator is of n pairs of observations, $P_k = \delta_{i(k)} \delta_{j(k)}$ for $i(k) \neq j(k)$ or $P_k = \delta_{i(k)}^2 - 1$ for $i(k) = j(k)$, and where

$$\delta_i = \sqrt{N/(N-1)} (m_i - \bar{m}) / \sigma_i \quad (43)$$

is the normalized residual for observation i , there are N observations total, observations $i(k)$ and $j(k)$ form pair k , and w_k is a weight to be assigned to a pair.

The Kurtosis of the light curve magnitudes (i.e., the fourth moment of the distribution) is estimated via

$$K = \frac{\frac{1}{N} \sum_{i=1}^N |\delta_i|}{\sqrt{\frac{1}{N} \sum_{i=1}^N \delta_i^2}} \quad (44)$$

and has a value of $\sqrt{2/\pi} \approx 0.798$ when N is large and the magnitude values are drawn from a Gaussian distribution.

The L statistic is then defined as

$$L = \frac{JK}{0.798} \frac{\sum w}{w_{\text{all}}} \quad (45)$$

where w_{all} is the maximum value of $\sum w$ that any light curve from the survey could have (i.e., a light curve with measurements from all images in a survey). This statistic provides enhanced significance over the J statistic for light curves with non-Gaussian magnitudes, and reduces the significance for light curves with missing observations.

2.2.2.6. -rms. Calculates the standard root-mean-square (RMS) scatter of the light curve given by:

$$\text{RMS} = \sqrt{\sum_{i=1}^N (m_i - \bar{m})^2 / (N - 1)}. \quad (46)$$

Additionally this command will compute the expected RMS of the light curve given the input photometric uncertainties. This is given by

$$\bar{\text{RMS}} = \sqrt{\sum_{i=1}^N (\sigma_i^2) / N}. \quad (47)$$

2.2.2.7. -rmsbin. Calculates the RMS after applying a moving mean filter to each light curve. The filter is the same as used for the **-chi2bin** command.

2.2.2.8. -stats. Calculates one, or more, basic statistics for one, or more, light-curve vectors (i.e., variables read-in from the light curve). The available statistics include: the mean; the mean weighted by the input photometric uncertainties; the median; the weighted median; the standard deviation calculated with respect to the mean; the standard deviation calculated with respect to the median; the median of the absolute deviations from the median (medmeddev); the MAD ($1.483 \times \text{medmeddev}$, which for a Gaussian distribution equals the standard deviation in the limit of large numbers, but is robust against outliers); the kurtosis; the skewness; percentile values (any floating point number between 0 and 100 may be used); weighted percentile values; the maximum value; the minimum value; and the sum of all elements.

2.2.3. Model-fitting

There are seven built-in commands which can be used to fit model signals to light curves.

2.2.3.1. -decorr. Fits a model that is linear in its parameters to a light curve. This command is deprecated in the current version of VARTOOLS, being replaced by the more flexible **-linfit** command, so it will not be discussed in detail here.

2.2.3.2. -Killharm. Fits a harmonic series to a light curve. The model has the form

$$\begin{aligned} m_0 + \sum_{i=1}^{N_p} \left\{ a_{i,1} \sin(2\pi f_i t) + b_{i,1} \cos(2\pi f_i t) \right. \\ + \sum_{k=2}^{N_{\text{harm},i}+1} [a_{i,k} \sin(2\pi k f_i t) + b_{i,k} \cos(2\pi k f_i t)] \\ \left. + \sum_{k=2}^{N_{\text{subharm},i}+1} [c_{i,k} \sin(2\pi f_i t/k) + d_{i,k} \cos(2\pi f_i t/k)] \right\} \end{aligned} \quad (48)$$

where there are N_p fixed frequencies f_i in the fit, each having $N_{\text{harm},i}$ higher-order harmonics and $N_{\text{subharm},i}$ subharmonics, and the parameters m_0 , $a_{i,k}$, $b_{i,k}$, $c_{i,k}$ and $d_{i,k}$ are fitted by the procedure. Note that this same model can also be fit through the more general-purpose **-linfit** command; the separate **-Killharm** command is provided for convenience.

By default the best-fit values for the parameters m_0 , $a_{i,k}$, $b_{i,k}$, $c_{i,k}$ and $d_{i,k}$ are reported in the output table. The user may optionally output amplitudes, phases, or relative amplitudes and phases instead. Here the amplitude of harmonic k is given by

$$A_{i,k} = \sqrt{a_{i,k}^2 + b_{i,k}^2} \quad (49)$$

while the amplitude for sub-harmonic k is given by

$$B_{i,k} = \sqrt{c_{i,k}^2 + d_{i,k}^2}. \quad (50)$$

The amplitudes may also be specified relative to the amplitude of the fundamental frequency via

$$R_{i,k,1} = A_{i,k}/A_{i,1} \quad (51)$$

and

$$Q_{i,k,1} = B_{i,k}/A_{i,1}. \quad (52)$$

The phases are calculated as

$$\phi_{i,k} = \text{atan2}(-b_{i,k}, a_{i,k}) \quad (53)$$

and

$$\psi_{i,k} = \text{atan2}(-d_{i,k}, c_{i,k}) \quad (54)$$

where $\text{atan2}(y, x)$ is the standard 4-quadrant inverse tangent function with output in radians. The relative phases of the harmonics and sub-harmonics are given by

$$\phi_{i,k,1} = \phi_{i,k} - k\phi_{i,1} \quad (55)$$

and

$$\psi_{i,k,1} = \psi_{i,k} - \frac{1}{k}\phi_{i,1}. \quad (56)$$

Using relative amplitudes and phases for the harmonics and sub-harmonics is a common way to characterize the shape of a signal independently of the overall phase and amplitude. Such parameters are useful, for example, in classifying variable star light curves based on their morphology.

The **-Killharm** command also provides a measurement of the peak-to-peak amplitude of each harmonic series that is fit to the data (a separate amplitude is supplied for each period). To determine this amplitude, the model harmonic series is evaluated at $100 \times (N_{\text{harm},i} + N_{\text{subharm},i} + 1)$ phase steps. The minimum and maximum values of the series at the evaluated points are determined. Refined measurements are then performed near these points using the DBRENT algorithm (Press et al., 1992).

2.2.3.3. -linfit. Fits an analytic model that is linear in its parameters to a light curve. The user provides on the command-line an analytic expression together with a list of the names of variables in that expression that are to be varied. The fit is done by singular value decomposition (SVD) as implemented in the `svdfit` function of Numerical Recipes (Press et al., 1992). This function includes a correction which suppresses poorly constrained parameter combinations, with the goal of providing greater numerical stability. Because this can lead to differences from other tools which solve the linear least squares problem by direct inversion of the so-called design matrix, we provide a summary of the `svdfit` method below.

Let the light curve magnitudes be represented as a column vector $\vec{m}$ of length N , and suppose these are to be fit with a model $\vec{mod}$ which can be expressed as

$$\vec{mod} = \sum_i^{N_{\text{param}}} a_i \vec{x}_i, \quad (57)$$

i.e., the linear combination of N_{param} column vectors $\vec{x}_i$. These vectors are supplied by the user, and may be the times of observations (perhaps raised to some power), other columns read-in from the light curve (e.g., the X and/or Y coordinates of the star on each image), or any analytic expression involving such terms. The a_i parameters are the N_{param} variables to be fit by this command by minimizing χ^2 , which is given by

$$\chi^2 = |\mathbf{A}\vec{a} - \vec{b}|^2. \quad (58)$$

Here the $N \times N_{\text{param}}$ design matrix $\mathbf{A}$ has components

$$A_{ij} = a_j x_{ji} / \sigma_i \quad (59)$$

with x_{ji} being the i th component of vector $\vec{x}_j$ and σ_i is the i th measurement uncertainty in the light curve, and the vector $\vec{b}$ has components

$$b_i = m_i / \sigma_i. \quad (60)$$

Applying SVD (via the svdcmp function in Numerical Recipes) to $\mathbf{A}$ yields an $N \times N_{\text{param}}$ column-orthogonal matrix $\mathbf{U}$, a $N_{\text{param}} \times N_{\text{param}}$ orthogonal matrix $\mathbf{V}$, and a $N_{\text{param}} \times N_{\text{param}}$ diagonal matrix $\mathbf{W}$ with positive or zero elements, which together satisfy

$$\mathbf{A} = \mathbf{U}\mathbf{W}\mathbf{V}^T \quad (61)$$

and which can be used to determine the least-squares solution for $\vec{a}$ via

$$\vec{a} = \sum_{i=1}^{N_{\text{param}}} \frac{\mathbf{U}_i \cdot \vec{b}}{w_i} \mathbf{V}_i \quad (62)$$

where $\mathbf{U}_i$ and $\mathbf{V}_i$ are the i th columns of each matrix, and w_i is the i th diagonal element of $\mathbf{W}$. The evaluation of Eq. (62) can be done efficiently by back-substitution (the svbksb function in Numerical Recipes). In cases where degeneracies exist between the different $\vec{x}_j$ vectors, very small values of w_i may be found leading to numerical instabilities in evaluating Eq. (62). This problem is overcome by setting $\frac{1}{w_i} = 0$ in cases where $w_i < \text{TOL} \times w_{\text{max}}$, for some tolerance factor TOL and with w_{max} being the maximum element in $\mathbf{W}$. We adopt $\text{TOL} = 10^{-9}$ in VARTOOLS.

Formal uncertainties on the parameters are also determined based on the covariance matrix—these uncertainties are accurate only if the measurement uncertainties supplied in the light curve are correct, and if the noise is uncorrelated in time and drawn from a normal distribution.

2.2.3.4. -MandelAgolTransit. Fits the analytic model for the transit of a dark spherical planet across a limb-darkened spherical star described by Mandel and Agol (2002). This model is parameterized by the orbital period, an initial epoch of transit center, the ratio of the planet radius to the stellar radius, the ratio of the semi-major axis to the stellar radius, the normalized impact parameter (the minimum projected distance between the stellar and planet centers divided by the sum of the planet and stellar radii), the orbital eccentricity, the argument of periastron, and either two or four limb darkening coefficients (two for a quadratic limb-darkening law, and four for a non-linear limb-darkening law). Note that this choice of the impact parameter differs from some implementations where the normalization is done only to the stellar radius. Normalizing to the sum of the radii has the benefit of limiting the range of possible values to between 0 and 1 to ensure transit. The user may also simultaneously fit a Keplerian orbit to a set of radial velocity data, input as a separate file. In this case the orbital semi-amplitude and center-of-mass velocities are additional parameters. Because the transit model is nonlinear in its parameters, it is necessary to choose initial values from which to begin the fit. The DHSX is then used to search for a local χ^2 minimum near the initial position. The user may either fix the initial values on the command line, or use the results from the -BLS command to automatically initialize the parameters. Note that this routine is not intended to produce publication-quality fits, as it provides no mechanism for error determination, and does not support simultaneous fits of multiple light curves. It is instead useful for rapidly processing many candidate transiting planet systems to identify initial parameter estimates (as might be done in a transit search pipeline).

2.2.3.5. -microlens. Fits a simple microlensing model due to Wozniak et al. (2001) to a light curve. The model is given by

$$M(t) = f_0 + f_1(A(t) - 1) \quad (63)$$

where

$$A(t) = \frac{u^2 + 2}{u\sqrt{u^2 + 4}} \quad (64)$$

and

$$u^2 = u_0^2 + \left(\frac{t - t_{\text{max}}}{t_0} \right)^2. \quad (65)$$

The free parameters in the model include: the zero-point flux f_0 ; the lensed flux f_1 ; the impact parameter u_0 in units of the Einstein radius; the Einstein ring crossing time t_0 ; and the time of maximum lensing t_{max} . The model is fit using DHSX.

2.2.3.6. -nonlinfit. Fits an analytic model to a light curve (non-analytic functions may also be fit through the use of user-developed functions, see Section 2.3.2). The model may be non-linear in its parameters. The user provides on the command-line an analytic expression together with a list of the names of variables in that expression that are to be varied, initial estimates for their best-fit values, and step-sizes to use in initializing the search algorithms. The user may optionally include a separate list of parameters that enter linearly into the model, and which will be optimized using linear least squares at each step in the non-linear model fitting (via Eq. (62)). In addition to specifying the model and the free parameters, the user may also provide prior probability estimates for the parameters and/or constraints on the parameters (a list of analytic inequalities that must be satisfied by the parameters). The routine will attempt to minimize the expression:

$$\chi^2 = \ln(\det(\Sigma)) + (\vec{m} - \vec{mod})^T \Sigma^{-1} (\vec{m} - \vec{mod}) + \sum_{j=1}^{N_p} P_j \quad (66)$$

where $\vec{m}$ and $\vec{mod}$ are column vectors containing the measured and model magnitudes, respectively, Σ is the covariance matrix for the observational data, and P_j are the N_p priors which are analytic functions of the free parameters. By default Σ is taken to be a diagonal matrix with components $\Sigma_{ii} = \sigma_i^2$, where σ_i is the magnitude uncertainty for observation i . In this case the term $\ln(\det(\Sigma))$ is ignored by the routine as it is independent of the model parameters. The user may optionally specify an analytic expression for the uncertainties (this expression may include parameters which are varied in the fit), in which case the term will be included. Options for specifying a non-diagonal covariance matrix are discussed below. The value of χ^2 is minimized over the region satisfying the parameter constraints. The quantity χ^2 is related to the likelihood L via $\chi^2 = -2 \ln L + C$ for some constant C and is equal to χ^2 when using a diagonal covariance matrix and no priors. Note that the P_j functions specified by the user are actually equal to -2 times the natural logarithm of functions proportional to the prior probability densities, the P_j functions are not the prior probability density functions themselves. For example, if the user wishes to specify a Gaussian prior of the form:

$$P \propto \exp \left[-(a - \bar{a})^2 / (2\sigma_a^2) \right] \quad (67)$$

for some parameter a , they would express it as $(a - \bar{a})^2 / (\sigma_a^2)$.

We allow three different forms for describing non-diagonal elements in the covariance matrix (i.e., allowing for time-correlated noise). These are the squared exponential, exponential and Matérn noise models. In the squared exponential model the covariance between times i and j is given by

$$\Sigma_{ij} = \sigma_i^2 \delta_{ij} + a \exp \left(-\frac{(t_i - t_j)^2}{2\rho^2} \right) \quad (68)$$

where σ_i is the formal uncertainty for measurement i (if the user specifies an analytic expression to use for the uncertainty, then σ_i is the expression evaluated at time i), δ_{ij} is the Kronecker delta function, and $a > 0$, and $\rho > 0$ are parameters of the noise model. The exponential model is similar, with

$$\Sigma_{ij} = \sigma_i^2 \delta_{ij} + a \exp\left(-\frac{|t_i - t_j|}{\rho}\right). \quad (69)$$

The Matérn model (Handcock and Stein, 1993) is given by

$$\Sigma_{ij} = \sigma_i^2 \delta_{ij} + a \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu}|t_i - t_j|}{\rho}\right)^\nu K_\nu\left(\frac{\sqrt{2\nu}|t_i - t_j|}{\rho}\right) \quad (70)$$

where Γ is the standard gamma function, K_ν is the modified Bessel function of the second kind, and $a > 0$, $\rho > 0$ and $\nu > 0$ are free parameters. This model allows for covariances that fall off more slowly with separation than the squared exponential model. For $\nu = 1/2$ the Matérn model is proportional to $\exp(-|t_i - t_j|/\rho)$, while for $\nu \rightarrow \infty$ it converges to the squared exponential. When non-diagonal covariances are used it is necessary to compute the inverse of the covariance matrix, which as an $\mathcal{O}(N^3)$ operation becomes computationally expensive when analyzing time series with tens to hundreds of thousands of points. To speed up the calculation we make use of the fact that for the noise models assumed we have $\Sigma_{ij} > \Sigma_{ik}$ for $j \geq i$ and $k > j$ if the light curve is sorted in time. Moreover, because the elements of Σ approach zero very rapidly when moving away from the diagonal, the covariance matrix is typically a sparse, near-diagonal matrix. We therefore compute the expression $\Sigma^{-1}(\mathbf{m} - \mathbf{mod})$ using Cholesky decomposition and back-substitution, truncating the loops within the algorithm when $C_{ij} < \epsilon$ for an assumed tiny value of ϵ . The logarithm of the determinant of Σ is computed from this same decomposition.

To optimize the non-linear parameters the user may choose between using the DHSX method as implemented in the AMOEBA routine by Press et al. (1992), or running a Differential Evolution Markov Chain Monte Carlo (DEMC) procedure.

The AMOEBA routine performs a greedy search for the X^2 minimum, and will tend to find the local minimum nearest to the initial starting position. The search is carried out using a set of $N_{\text{par}} + 1$ vectors of parameters (the so-called simplex, here N_{par} is the number of free parameters). In the VARTOOLS implementation the components of each vector are initially set to the initial parameter values specified by the user, except that for vector j the component $j - 1$ is set to the initial value plus the specified step-size (for the first vector no component is adjusted). The routine evaluates X^2 for each vector of parameters, and then repeatedly adjusts the components of each vector following a prescribed method in an attempt to find the minimum value of X^2 . The routine stops once it achieves:

$$\frac{2|X_{\text{max}}^2 - X_{\text{min}}^2|}{|X_{\text{max}}^2 + X_{\text{min}}^2| + \epsilon} < \text{TOL} \quad (71)$$

where X_{max}^2 is the maximum value of X^2 amongst the vectors in the simplex, X_{min}^2 is the minimum value, $\epsilon = 10^{-10}$ is a small number used to prevent division by zero, and TOL is the convergence tolerance (by default equal to 10^{-10} , but the user may specify a different value on the command line).

The second option is to run a DEMCMC procedure (ter Braak, 2006). This procedure runs $2N_{\text{par}}$ simultaneous Markov chains (a minimum of three chains are used). By default VARTOOLS will first run the AMOEBA downhill simplex to find a local minimum about which the DEMCMC will be executed, though the user may skip this step. One of the chains is then initialized to the best-fit parameter values, for the other chains we add to each parameter a Gaussian random number multiplied by the step-size. The initial

user-supplied step sizes are iteratively adjusted to determine the step-size such that $\Delta X^2 = 1$ for each adjusted parameter, holding all other parameters fixed. We cycle through each of the chains proposing a new link N in the chain j using

$$\vec{a}_{j,N} = \vec{a}_{j,N-1} + \gamma(\vec{a}_{k,N-1} - \vec{a}_{l,N-1}) + \epsilon \mathbf{U} \times \vec{r} \quad (72)$$

where k and l are the indices of two randomly selected chains, $\gamma = 1.7/\sqrt{2N_{\text{par}}}$, ϵ is a small number (0.1 by default, but can be modified by the user), $\vec{r}$ is a vector of Gaussian random numbers and $\mathbf{U}$ is an upper triangular matrix determined from the Cholesky decomposition of the parameter covariance matrix Σ (e.g. Press et al., 1992). We estimate Σ from the existing DEMCMC chains, recomputing $\mathbf{U}$ every 10,000 links. We calculate ΔX^2 , the difference in X^2 between the proposed link and the prior link in the chain, and then accept the proposal with probability $\exp(-\Delta X^2/2)$ (if $\Delta X^2 < 0$, the proposed link is always accepted). If the proposed link is rejected, the previous link is copied to link N . The process terminates once either the total number of links reaches a user specified limit, or the total number of accepted links reaches a limit.

The N_c chains are stored in a single array such that element i in this array is associated to chain $i_{\text{chain}} = i \bmod N_c$. The user has the option of writing this array out to a file. By default all links are written, but the user may write out only a fraction of the links using the “printevery” option. To allow the user to interrupt the process without losing all of the results the links are written to the file buffer immediately after each MCMC step. However, we do not explicitly flush the file buffer, so the data will only be written to disk once the buffer is filled (this may be every few hundred links, but depends on the number of free parameters, and on the specific computer system used).

The full chain is also stored in memory and is used to compute statistics included in the output ascii table. By default these are the median and standard deviation of each free parameter. These may be interpreted as the most likely values and uncertainties for each parameter. The values corresponding to the lowest X^2 model are also included in the output ascii table. The user may change the statistics computed from the chain and may also specify different combinations of the parameters on which to compute the statistics. By default the first 10% of the chain is excluded from calculating the output statistics, however the user may optionally specify a different burn-in fraction.

A limit is also provided on the total amount of memory allowed to be allocated to a particular MCMC command. Once this limit is reached, each time a new link is added to the chain the earliest link in the chain is forgotten. The default memory limit is 4 GB. Note that if VARTOOLS is run in parallel mode, then the limit for each thread is the total limit divided by the number of threads.

2.2.3.7. -SoftenedTransit. Fits an approximate transit model due to Protopapas et al. (2005) to a light curve. The model is given by:

$$M(t) = M_0 + \frac{1}{2} \delta \left\{ 2 - \tanh[c(t' + 0.5)] + \tanh[c(t' - 0.5)] \right\} \quad (73)$$

with

$$t'(t) = \frac{P}{\pi \eta} \sin(\pi(t - T_0)/P) \quad (74)$$

where M_0 is the out of transit magnitude, δ is the depth of the transit, c is a constant controlling the “sharpness” of the transit, P is the period, T_0 is the reference time of mid transit, and η is the transit duration.

2.2.3.8. -Starspot. Fits a single, circular, uniform temperature spot model given by Dorren (1987) to a light curve. This model is given by:

$$M(t) = M_0 - 2.5 \log_{10} [1 - (aA(\alpha, \beta(t)) + bB(\alpha, \beta(t)))] \quad (75)$$

where M_0 is the out of transit magnitude, A and B are functions given by Dorren (1987) that depend on α , the spot angular radius, and β , the angle between the line of sight and the surface normal at the spot center, and a and b are quantities given by

$$a = \frac{(1 - \mu_\star) - (1 - \mu_s) \frac{F_s}{F_\star}}{\pi(1 - \mu_\star/3)} \quad (76)$$

$$b = \frac{\mu_\star - \mu_s \frac{F_s}{F_\star}}{\pi(1 - \mu_\star/3)} \quad (77)$$

with $\mu_\star$ and μ_s being the linear limb darkening coefficients for the unspotted photosphere of the star and for the spot, respectively, and $F_s/F_\star$ being the spot to photosphere flux ratio. Dorren (1987) suggest values of $a = 0.0298$ and $b = 0.08745$ for the Sun at 5000 Å. The angle β is itself a function of the time of the observation, the rotation period of the star, the inclination of the stellar rotation axis, the latitude of the spot, and the longitude of the spot center at a reference time. This procedure is deprecated in the current version of VARTOOLS, users should instead use the **-macula** model which is distributed with VARTOOLS as a user-developed command.

2.2.4. Filtering

The built-in commands listed below can be used to filter light curves. Additionally, filtering may be done through the procedures described in the previous subsection (e.g., **-Killharm** to filter harmonic series from the data, or **-linfit** to apply a linear filter).

2.2.4.1. -clip. Removes points from the light curve. Each point is removed from all vectors associated with the light curve (e.g. the time, magnitude, magnitude uncertainty, and other data read-in from the light curve). The default method is to σ -clip outlier magnitude values. In other words, given an average magnitude $\bar{m}$ measured RMS magnitude σ_m , and specified clipping factor α , any point i with magnitude m_i satisfying $|m_i - \bar{m}| > \alpha \sigma_m$ will be removed from the light curve. The user may optionally run this in iterative mode, in which case $\bar{m}$ and σ_m are both recalculated after clipping, and the clipping is repeated with the new values; this can either be repeated for a specified number of iterations, or until no further points are removed. Finally the user may also simply remove points with formal errors ≤ 0 or undefined magnitude values. This later capability can be used in combination with an **-expr** command to perform a customized clipping procedure (e.g., listing 3).

Listing 3: Example of using the **-clip** command to remove points with magnitude values greater than 10.

```
vartools -i inputfile.txt \
2 ... \
  -expr 'err=(mag>10)*(-1)+(mag<=10)*err' \
4 -clip -1 0 \
  ...
```

2.2.4.2. -medianfilter. For each point i , determines the median magnitude value of all points j that satisfy $|t_i - t_j| < \Delta t$ for fixed Δt , and then either subtracts this value from the magnitude of point i (high-pass filtering the light curve), or replaces the magnitude of point i with this value (low-pass filtering the light curve). The user may optionally use the average, or the uncertainty-weighted average instead of the median.

2.2.4.3. -restricttimes. Filter observations from the light curve(s) based on the time values. The user either indicates observations to include or exclude. This can be done by giving a range of time values, a list of time values to filter, or a list of image *id* values to filter.

2.2.4.4. -SYSREM. Performs the trend-filtering procedure suggested by Tamuz et al. (2005), which effectively amounts to determining the principle components of the light curve dataset, and filtering these from the light curves. The filter operates by attempting to model the light curves indexed by i consisting of points indexed by j as:

$$m_{i,j} = \epsilon_{i,j} + \sum_{k=1}^{N_{trend}} c_{k,i} a_{k,j} \quad (78)$$

where $\epsilon_{i,j}$ is a Gaussian random number drawn from a normal distribution with mean $\bar{m}_i$ and standard deviation $\sigma_{i,j}$; and there are N_{trend} trend vectors $a_{k,j}$, called “airmass-like” terms by Tamuz et al. (2005), to which each light curve has a linear response parameterized by $c_{k,i}$, called “color-like” terms by Tamuz et al. (2005). Initial airmass and/or color-like vectors are assumed (these might be the airmasses of the images, or the x/y pixel positions of the stars, for example). For each assumed airmass-like (color-like) vector the associated color-like (airmass-like) vector is solved for by linear least squares minimization. The so-determined color-like (airmass-like) vector is then fixed, and the airmass-like (color-like) vector is then redetermined by linear least squares minimization. The procedure is repeated for the next vector until all N_{trend} trends are fitted. To prevent outliers from dominating the determination of trends, the user specifies two σ -clipping parameters. The first for excluding points from individual light curves in determining the average magnitudes of the light curves, the second for excluding points from the fits for the airmass-like or color-like terms.

2.2.4.5. -TFA. Applies the Trend Filtering Algorithm suggested by Kovács et al. (2005). This is similar to SYSREM in that the light curves are modeled as a linear combination of trend vectors as described in Eq. (78), however in this case the $a_{k,j}$ “airmass-like” terms are taken to be N_{trend} template light curves chosen randomly from among the full ensemble of observed light curves, and only the $c_{k,i}$ “color-like” terms are determined by fitting. One other practical difference between SYSREM and TFA is that the latter typically uses more trend vectors in modeling the light curves (e.g. for a field containing $\gtrsim 10,000$ stars, each containing $\sim 10,000$ observations, it is not uncommon to use ~ 1000 template light curves in TFA, whereas typically applications of SYSREM may only use a few dozen trend vectors). When the same set of templates is used to filter numerous light curves the filter may be applied efficiently by a combination of Singular Value Decomposition (or alternatively Lower–Upper Diagonal Decomposition) applied once to the matrix of templates, followed by the much faster Singular Value Back Substitution procedure applied to each light curve to be filtered.

2.2.4.6. -TFA_SR. Applies the Trend Filtering Algorithm in “signal-reconstruction” mode. In this case the assumed light curve model is modified from that given in Eq. (78) to:

$$m_{i,j} = \epsilon_{i,j} + S_i(t_j) + \sum_{k=1}^{N_{trend}} c_{k,i} a_{k,j} \quad (79)$$

where $S_i(t_j)$ is a light-curve-dependent signal. In VARTOOLS the allowed signals include: (1) a periodic step-function of the form

$$S(t) = \sum_{u=1}^{N_{bin}} \alpha_u \Theta(\phi - (u-1)/N_{bin}) \Theta(u/N_{bin} - \phi) \quad (80)$$

$$\phi = t/P - \text{floor}(t/P)$$

where $\Theta(X)$ is the Heaviside-step function ($=0$ for $x < 0$; $=1$ for $x > 0$), P is the fixed period of the signal, there are N_{bin} steps used, and the free parameters are the α_u terms; (2) a fixed signal form evaluated at the times of observation (and read-in from a file); or (3) a harmonic series of the form given in Eq. (48). Additionally the user may include a number of light-curve-specific vectors among the series of trend vectors used in the fit. In practice this slows down the filter as it requires a re-inversion of the design matrix used in the linear fit.

2.2.5. Light curve simulation

The following built-in commands may be used in simulating light curves.

2.2.5.1. -addnoise. Adds noise with optional time-correlation to the light curve. Five different methods may be selected from. The first is simple uncorrelated Gaussian white noise, in which case the user specifies the standard deviation of the noise.

The second, third and fourth methods simulate a Gaussian process with a fixed covariance matrix Σ . These methods differ in the form of the covariance matrix. The second method assumes a squared exponential model where the elements of the covariance matrix are given by Eq. (68). The third method assumes an exponential covariance with elements given by Eq. (69). The fourth method uses the Matérn class of covariance matrices with elements given by Eq. (70). In all three cases the Cholesky decomposition of Σ is computed and then multiplied against a vector of random numbers drawn from a normal distribution to produce a light curve drawn from the specified covariance matrix.

In the fifth method the noise is modeled as

$$\epsilon_w + \epsilon_r \quad (81)$$

where ϵ_w is a white-noise component that is a random number drawn from a Gaussian distribution with zero mean and standard deviation σ_w (specified by the user), while ϵ_r is a red-noise component drawn from a distribution with standard deviation σ_r and time-correlations such that the power-spectral density of the noise scales as $1/f^\gamma$ (both γ , which must satisfy $-1 < \gamma < 1$, and σ_r are specified by the user; taking $\gamma = 0$ yields uncorrelated noise). We use the method of McCoy and Walden (1996) to generate the red-noise. The McCoy and Walden (1996) procedure for simulating a series of $N = 2^p$ points uniformly sampled in time, described in Section 4.2 of that paper, works as follows: (1) generate a set of points $d_{m,k}$ with $m = 1, \dots, p$ and $k = 1, \dots, 2^{p-m}$ each drawn from a Gaussian distribution with zero mean and variance S_m , and generate an additional point x_p , 1 with zero mean and variance S_{p+1} (McCoy and Walden, 1996 gives a prescription for determining the variances S_m that depends on σ_r and γ); (2) apply the inverse discrete wavelet transform (we use the implementation in the GNU scientific library) to the vector $(x_{p,1}, d_{p,1}, d_{p-1,1}, d_{p-1,2}, \dots, d_{1,1}, \dots, d_{1,2^{p-1}})$. To generate non-uniformly sampled observations, we first simulate a series of $N = 2^p$ uniformly sampled points as above, taking

$$p = \min(20, \text{ceil}(\lg(T/\delta_t))) \quad (82)$$

where T is the time spanned by the non-uniformly sampled observations, and δ_t is the minimum time between successive observations. We then linearly interpolate this sequence onto the non-uniformly sampled observation times.

2.2.5.2. -copylc. Makes copies of the light curve under analysis. Data read in from the input list file, as well as data computed by commands executed prior to the copy command are also replicated. Each copy is then independently processed through the subsequent commands issued to VARTOOLS. This command is useful, for example, for carrying out simulations in which different

signals are injected into the same light curve to estimate the recovery efficiency, or different pure-noise light curves with the same time sampling are simulated to determine the bandwidth correction for period finding algorithms such as -LS.

2.2.5.3. -injectharm. Adds to the light curve a harmonic series of the form

$$A_1 \cos(2\pi(t/P + \phi_1)) + \sum_{k=2}^{N_{\text{harm}}+1} (A_k \cos(2\pi(kt/P + \phi_k))) + \sum_{k=2}^{N_{\text{subharm}}+1} (B_k \cos(2\pi(t/(kP) + \psi_k))) \quad (83)$$

where P is the period of the signal, A_1 , A_k , and B_k are the amplitudes of the fundamental, harmonic, and subharmonic terms, respectively, and ϕ_1 , ϕ_k , and ψ_k are the phases of the fundamental, harmonic, and subharmonic terms, respectively. The user indicates the number of desired harmonic and subharmonic terms (N_{harm} and N_{subharm}) and how each of these are to be determined (their values may be fixed on the command-line, read-in from the input list of light curves, or randomly drawn from uniform or uniform-log distributions). The amplitudes and phases of the harmonic and subharmonic terms may also be specified relative to the amplitude and phase of the fundamental (i.e. the input is $R_{i1} = A_i/A_1$ rather than A_i , and/or $\phi_{k1} = \phi_k - k\phi_1$ rather than ϕ_k). This latter option is useful if one wants to inject into the light curve a signal with a fixed shape (e.g. a saw-tooth like pulsation), but random overall amplitude and phase, which someone might want to do to determine the completeness of a variability survey to a particular class of variable stars. In this case one would fix the relative phases and amplitudes of the harmonic and/or subharmonic terms, but allow the amplitude and phase of the fundamental to vary.

2.2.5.4. -injecttransit. Adds a transit signal to the light curve. VARTOOLS uses the Mandel and Agol (2002) model for the transit of a nonluminous spherical object in front of a limb-darkened spherical star. The user may specify the model parameters, including: the orbital period, the radius of the planet, the mass of the planet, the phase of the transit at time $T = 0$, $\sin i$ (i is the orbital inclination), the eccentricity and argument of periastron or $e \cos \omega$ and $e \sin \omega$, the mass of the star, the radius of the star, quadratic or non-linear limb darkening coefficients for the star, and an optional light dilution factor. Alternatively some of these parameters may be drawn from random distributions. For the period this can be a uniform distribution in P , $\log P$, $f = 1/P$ or $\log f$. For the planet mass and/or radius this can be a uniform distribution or a log-uniform distribution. For $\sin i$ it is a uniform distribution of orbit orientations in space, subject to the constraint that there must be a transit, which corresponds to $\cos i$ being drawn from a uniform distribution between 0 and C with

$$C = \frac{R_{\text{star}} + R_{\text{pl}}}{a} \frac{1 + e \cos(\pi - \omega)}{1 - e^2}. \quad (84)$$

2.2.6. Light curve manipulation

The following commands can be used to perform common light curve manipulation tasks:

2.2.6.1. -binlc. Bins the light curve in time. The user either specifies the number of bins to use, or the size of the bins in units of the time coordinate, and may also control the starting time of the first bin (by default it is the time of the first observation in the light curve). The user may either use the average, median or error-weighted average for the resulting binned value. All vectors associated with this light curve (i.e. columns read-in from the

light curve file including the magnitudes), except for the time and uncertainty, vectors storing string or character data, or any vectors explicitly indicated by the user, are binned as follows:

$$\bar{x}_i = \begin{cases} \sum_j x_j / N_i, & \text{average} \\ \text{med}(x_j), & \text{median} \\ \sum_j (x_j \sigma_j^{-2}) / \sum_j \sigma_j^{-2}, & \text{weighted average} \end{cases} \quad (85)$$

while the uncertainties are binned following:

$$\bar{\sigma}_i = \begin{cases} \sqrt{\sum_j (\sigma_j^2) / N_i}, & \text{average} \\ 1.253 \sqrt{\sum_j (\sigma_j^2) / N_i}, & \text{median} \\ \sqrt{\left(\sum_j (\sigma_j^{-2}) \right)^{-1}}, & \text{weighted average} \end{cases} \quad (86)$$

where $\bar{x}_i$ is the i th binned value of vector x , and $\bar{\sigma}_i$ is the i th binned uncertainty, the sum on j in each case is over the unbinned points with times that fall within bin i , N_i is the number of points that contribute to bin i , and the uncertainties are the standard errors on the binned quantities assuming Gaussian uncorrelated noise. Bins for which $N_i = 0$ are excluded from the binned light curve. The time reported for each bin is either the time at the center of the bin, the average of the times that fall within the bin (i.e. $\bar{t}_i = \sum_j t_j / N_i$), the median of the times that fall within the bin, or the user may choose not to shrink the size of the light curve, in which case all points within a bin are replaced by their binned value, but the light curve times are not changed. The user may also optionally provide a list of variables which will not be binned in the default manner. For each variable in the list the user also indicates which statistic to use in calculating the binned value. Options are the same as for the **-stats** command.

2.2.6.2. -changeerror. Sets the magnitude uncertainties σ_j in a light curve equal to the measured r.m.s. scatter of the light curve magnitudes.

2.2.6.3. -converttime. Converts the time-system of the light curves between julian date (JD), modified julian date (MJD = JD – 2400000.5), helio-centric julian date (HJD; i.e., JD approximately corrected to the center of the solar system assuming observations are made at the Earth–Moon barycenter, and that this position follows an elliptical orbit about the center of the solar system with linear perturbations to the orbital elements), and bary-centric julian date (BJD; i.e., time corrected to the center of the solar system using the NASA JPL ephemeris to determine the location of the observer relative to the barycenter at a given time); constants may be added to or subtracted from the input and/or output times.

Time conversions between a UTC-based system (i.e., JD values having been calculated directly from UTC without accounting for leap-seconds; this is not formally correct, but a very common practice) and a TDB-based system (barycentric dynamical time, i.e. the UTC time is corrected to a leap-second-free time system before converting to JD) are also supported. In the year 2015 there is approximately a one minute difference between the two systems (see the discussion in Eastman et al., 2010).

Conversions to/from BJD and between UTC and TDB use the NASA JPL NAIF cspice library (Acton, 1996) following the expressions given by Eastman et al. (2010); we do not allow for Shapiro

or Einstein time-delays in converting to BJD (i.e., gravitational redshifts are ignored).

Differences between BJD and HJD are due mostly to the orbits of Jupiter and Saturn (which are ignored for HJD), and can be as large as 4.2 s for observations made between 1980 and 2020.

The user may specify an observatory where the data were collected, or the latitude, longitude, and altitude. If this is not done then the center of the Earth is assumed, which can introduce up to a 21 ms error in the BJD correction. The procedure optionally accounts for proper motion and/or coordinate precession.

Based on comparing time conversions made with VARTOOLS to those computed through the JPL HORIZONS interface⁹ we conclude that this command has an internal precision of ~ 0.1 ms for conversions near J2000.0. At present this command does not support time conversions for observations made off of the surface of the Earth.

2.2.6.4. -difffluxmag. Converts the light curves from differential flux units (which are output, for example, by the popular ISIS image subtraction procedure; Alard and Lupton, 1998) into magnitudes. The expression for converting from differential flux df to magnitude m is given by:

$$m = m_0 + \Delta_m - 2.5 * \log_{10}(f_{\text{ref}} - df) \quad (87)$$

where m_0 is the magnitude of a source yielding a flux of 1, Δ_m is an offset that the user may give to switch to a different magnitude system, and the reference flux f_{ref} is determined from the input reference magnitude m_{ref} via:

$$f_{\text{ref}} = 10^{-0.4(m_{\text{ref}} - m_0)}. \quad (88)$$

2.2.6.5. -ensemblerescalesig. Transforms the magnitude uncertainties of the light curves by the expression

$$\sigma'_{ij} = \sqrt{a\sigma_{ij}^2 + b} \quad (89)$$

where σ'_{ij} is the new uncertainty for light curve i , observation j , after the transformation. Here a and b are determined by fitting a linear relation of the form:

$$y_i = ax_i + b \quad (90)$$

where

$$y_i = (\chi_i^2 / \text{dof}) \bar{RMS}_i^2 \quad (91)$$

for light curve i , and

$$x_i = \bar{RMS}_i^2 \quad (92)$$

for light curve i , and $\bar{RMS}$ is the expected RMS of the light curve (Eq. (47)). The result of this transformation is that, for many datasets, the χ_i^2 / dof values are distributed about unity. The justification for this is given in Appendix D. The parameter a is related to the ratio of the gain and the effective gain, while b is related to a constant magnitude error-term (e.g. Hartman et al., 2005). This routine requires all of the light curves to be read-in simultaneously, and thus is incompatible with the **-parallel** option in the current version of VARTOOLS.

⁹ <http://ssd.jpl.nasa.gov/horizons.cgi>.

2.2.6.6. -expr. Sets a variable equal to an analytic expression. For example, one might give the command “-expr mag=mag/2” which evaluates mag/2 at each point in the light curve and replaces the magnitude with the result. If the variable on the left-hand-side of the equality has not previously been defined, it will be created. Variables which appear on the right-hand-side can be the name of a vector which is read-in from the light curve, a scalar or vector created by another command (e.g. the fitting parameters, or the output model vector, created by the **-linfit** command), or any output parameter from a previously executed command (such as the light curve RMS computed with the **-rms** command). VARTOOLS uses an implementation of the precedence climbing algorithm (Richards and Whitby-Stevens, 1979) for parsing and evaluating analytic expressions. The list of recognized functions is provided in Appendix C, while a method for users to define their own functions which may then be used in such expressions is discussed in Section 2.3.2.

2.2.6.7. -fluxtomag. Converts a light curve from fluxes into magnitudes. The uncertainties are also converted from flux uncertainties into magnitude uncertainties.

2.2.6.8. -Phase. Replaces the time coordinate of a light curve with its phase (from 0 to 1) and sorts the light curve by the phase. The user must specify the period to use for phasing the light curve, this may either be fixed on the command line for all light curves, read-in from the input light curve list, or taken from a previous command (e.g. the period found by **-LS**). The user may also optionally control the reference time for phase 0.

2.2.6.9. -resample. Resamples the light curve onto a new time base. Several methods are allowed for interpolating or extrapolating the data, including setting resampled points to the value of the observation that is closest in time, performing linear interpolation between points, cubic spline interpolation (e.g., Press et al., 1992), cubic spline interpolation with a constraint that the interpolating function be monotonic between points (Steffen, 1990), and Basis-spline interpolation (implemented using GSL routines). The user may optionally adopt different interpolation methods for resampled points that are close to observations and those that are far from observations.

2.2.6.10. -rescalesig. Rescales the magnitude uncertainties for a light curve (i.e. $\sigma = \alpha\sigma$) such that $\chi^2/\text{dof} = 1$ for that light curve.

2.2.7. Control of data flow

The following commands control the flow of data through the pipeline:

2.2.7.1. -changevariable. Changes the variable used for the time, magnitude, magnitude uncertainty, or image identifier in subsequent commands. For example, the command “-changevariable mag mag2” would cause subsequent commands to VARTOOLS to operate on the data stored in the variable *mag2* for the light curve magnitudes. The data stored in the original variable *mag* remains unchanged, and may be used in analytic expressions.

2.2.7.2. -if, -elif, -else, -fi. Can be used to make execution of commands conditional upon the evaluation of expressions, following the typical logic of “if”, “else if”, “else” and “end if”-type conditional statements. The use of the terms **-elif** and **-fi** are as in the Bourne-Again SHell. VARTOOLS supports nested conditionals, but all conditional constructs are ignored by commands which process all light curves simultaneously (e.g. **-SYSREM**) as well as by the **-savelc** and **-restorelc** commands.

2.2.7.3. -o. Outputs the light curve in its present state to a file (or to `STDOUT` if specified by the user). The user can specify a rule for naming the output file, and can also control the format of the output data. By default the light curve will be output as an ascii text file, but may also be output as a binary FITS table.

2.2.7.4. -restorelc and -savelc. Are used to save a light curve (including all associated vectors) and later restore it to this previously saved state.

2.2.8. Miscellaneous commands

The commands listed here do not fall under the other categories.

2.2.8.1. -findblends. This command may be used to determine if a signal detected in a given light curve is likely due to blending with a nearby variable source. The command follows the procedure described by Hartman et al. (2011). Each light curve in the input list is spatially matched to another list of light curves (by default the input light curve list is matched to itself, but the user may optionally provide a different list for the matching). A harmonic series with a specified period (either read-in from the input list file, fixed on the command line, or set equal to the output from a previously executed command) is then fit to both the input light curve, and to all matching light curves. The peak-to-peak amplitude of the fitted harmonic series is then determined for each source and converted to flux units (by default the routine assumes the input is in magnitudes). The light curve with the highest flux amplitude is reported. If this is not the same as the input light curve, then the neighboring object is the most likely source of the variability.

2.3. User-developed extensions

While VARTOOLS includes a wide-range of built-in processing routines, it falls far short of encompassing the full range of routines that one might wish to apply to a light curve. It is therefore essential to enable a user to incorporate his/her own light curve processing algorithms within the program. To that end two mechanisms have been included: one for dynamically loading at run-time a compiled library defining a user-developed command; and another for dynamically loading a library defining a user-developed function which can be used in analytic expressions on the command-line. Each of these are described in turn.

2.3.1. User-developed commands

Each library implementing a user-developed command must contain five functions, with standardized names and calling syntaxes, which are used to interface the command with VARTOOLS. These functions perform such tasks as initializing variables defining standard properties of the command, parsing the command-line when the user calls the command, displaying the syntax and help information for the command, and executing the command on a light curve. These required functions should be written in C, while the algorithm itself may be written in any language callable from C. Templates are provided of the functions required in each library, together with a Makefile for compiling and linking the code into a dynamically loadable shared object library. A template is also provided illustrating how to include a processing algorithm written in FORTRAN in VARTOOLS.

VARTOOLS uses the GNU Libtool package¹⁰ for compiling and dynamically loading libraries in a system-independent fashion. When VARTOOLS encounters an unrecognized term on the command-line,

¹⁰ <http://www.gnu.org/software/libtool>.

it will search for an appropriate library in a data directory defined during the installation of VARTOOLS, load it if available, and execute the command. So long as the libraries are installed in the correct directory, they may be called by the user in the same way that all other commands are executed. To avoid performing a directory search, the user may also use the **-L** option to explicitly load a library file.

A number of example libraries are distributed with VARTOOLS which provide additional commands. These include:

2.3.1.1. -fastchi2. Calculate the Fast χ^2 periodogram following Palmer (2009). This routine scans through a range of frequencies fitting a harmonic series to the data at each trial frequency and reporting the minimum χ^2 . It uses the fast Fourier transform to do the fit quickly in $\mathcal{O}(N \log N)$ time. The user specifies the number of harmonics to include in the series, the minimum and maximum frequencies to search, the order and reference time of a polynomial in time which is removed from the light curve before performing the search, the time-span to use in computing the Nyquist frequency (if not calculating it automatically), an over-sampling factor for determining the resolution of the periodogram, and a tolerance parameter used in performing a higher resolution search around the peaks in the periodogram.

2.3.1.2. -jktebop. Fit or inject a detached eclipsing binary light curve model [in]to a light curve. This command uses the code from the JKTEBOP program due to Southworth et al. (2004), converted into a callable library. The JKTEBOP code is in turn based on the Eclipsing Binary Orbit Program (Popper and Etzel, 1981; Etzel, 1981) and implements the model of Nelson and Davis (1972). The JKTEBOP-based library is written in FORTRAN, so this command can be used as a template for incorporating a FORTRAN algorithm into VARTOOLS. The basic parameters for the model include the orbital period of the binary, a reference primary eclipse epoch, the normalized sum of the component radii $((R_1 + R_2)/a)$, the ratio of the radii (R_2/R_1) , the mass ratio (M_2/M_1) , the surface brightness ratio (J_2/J_1) , the orbital inclination or normalized impact parameter, the eccentricity parameters $e \sin \omega$ and $e \cos \omega$, limb darkening coefficients, gravity darkening coefficients, reflection effect coefficients, third light, and a tidal lead/lag angle. The fit is performed using DHSX.

2.3.1.3. -macula. Fit or inject a Macula spot model [in]to a light curve. The model and FORTRAN code implementing it are both due to Kipping (2012). This model allows for an arbitrary number of starspots which are allowed to evolve over time with a linear growth/decay law. The parameters in the model include: the equatorial rotation period; the inclination of the star; quadratic and quartic differential rotation coefficients; four limb-darkening coefficients for the stellar photosphere and four for the spots; and a blend parameter. Additionally each spot is parameterized by: its longitude at the time of maximum size; its latitude at the time of maximum size; its maximum angular size; the spot-to-star flux ratio; the time of maximum size; the lifetime; the duration of ingress (time to appear); and the duration of egress (time to disappear). The model can be fit either with DHSX, or with the Levenberg–Marquardt algorithm (Press et al., 1992). The latter makes use of partial derivatives that are provided by Kipping (2012).

2.3.1.4. -magadd. Adds a constant to a light curve. This is provided as a trivial example of a user-developed command, and may be used as a template.

2.3.2. User-developed functions

The mechanism for incorporating a user's own functions into VARTOOLS is similar to the mechanism for incorporating user-developed commands. Again the user creates a dynamically loadable library written in C. The library must contain an initialization function with a standardized name which is called when the library is opened. This function is used to indicate the names of all new analytic functions supplied by the library, the number of double-precision arguments expected by each function (at present all functions input and output double-precision variables), and pointers to the C functions in the library (i.e., variables storing their location in memory) which execute the computations associated with each analytic function. These C functions are expected to take a vector of double-precision numbers, with each component in the vector representing a separate argument to the function, and to output a single double-precision quantity as the result. The functions may in turn be wrappers to routines written in other languages callable from C.

In order to use the functions provided by a library, the user must provide the **-F** option on the command-line, followed by the name of the library (excluding filename extensions). An example library is included with the VARTOOLS distribution, which may be used as a template.

2.4. Options

In addition to the commands mentioned above, a number of command-line options are available to change the behavior of VARTOOLS. These options may be given at any point on the command-line (except for within the list of parameters supplied to a command). The available options include:

-basename: only the base filename of each light curve is included in the output table, rather than the full path.

-bufferlines: use this to adjust the number of lines buffered within VARTOOLS before being written to standard out during parallel processing. This buffering is separate from the system buffering done on stdout itself, which may be turned off with the **-nobuffer** option. The **-bufferlines** option only has an effect if the **-parallel** option is used, and will help speed up the processing by preventing threads from waiting on each other to output results. The tradeoff with using a larger buffer is that more memory is used, and results will be output less frequently.

-example: used in conjunction with the name of a command to print out an example of how that command is used.

-functionlist: shows the list of function names that are recognized in analytic expressions.

-header: include column headers in the output ascii table.

-headeronly: prints the output header that a call to VARTOOLS would produce and exits without processing any light curves. This is useful for finding the column names or numbers to use when values computed by one command are used to set parameter values for subsequent commands.

-help: shows detailed help for a command.

-inlistvars: used to specify a format for the input light curve list, and store columns from this list as vector variables.

-inputlformat: used to specify the format of the input light curves. One may use this option to read-in in more than just the default JD, magnitude, and magnitude uncertainty values from the light curve.

-jdtol: sets the tolerance for considering two observations to have come from the same epoch. The default tolerance is 10^{-5} days. This is used by commands such as **-TFA** or **-SYSREM** which match points from different light curves.

-quiet: process the light curves, but do not output the ascii table of statistics.

-L: load a user-compiled library defining a new VARTOOLS command.

-listcommands: shows a terse list of the available commands.

-log-command-line: includes the command-line in the header of the output ascii table.

-matchstringid: use image string identifiers, rather than the time, to match points from different light curves.

-nobuffer: outputs each line in the ascii table as soon as it is generated, rather than buffering the output.

-noskipempty: by default empty light curves are skipped and not included in the output table. To not skip these give this option. This option has no effect if the **-readall** option is used.

-numbercolumns: pre-pend column numbers to the header names in the output ascii table.

-online: display each computed statistic on a separate line for each light curve, rather than in a table format.

-parallel: process multiple light curves in parallel.

-randseed: specifies a seed for the random number generator. If this is not used, then every call to VARTOOLS will produce the same set of random numbers. The system time may be used as a seed to produce quasi-random numbers.

-readall: read-in all of the light curves at once, rather than as they are processed.

-readformat: This option is deprecated in the current version of VARTOOLS, users are encouraged to use the **-inputlcformat** option instead. It is provided to allow compatibility for older processing scripts.

-redirectstats: output the ascii table of statistics to a file rather than to stdout.

-showinputlcformat: displays the expected format of the input light curves, given a call to VARTOOLS. No processing will occur if this option is given, it is provided for assistance in developing scripts using VARTOOLS.

-showinputlistformat: displays the expected format of the input list of light curves, given a call to VARTOOLS. No processing will occur if this option is given, it is provided for assistance in developing scripts using VARTOOLS.

-skipmissing: do not abort if a missing or unreadable light curve file is encountered. Instead skip the light curve and proceed with others in the list.

-tab: outputs the ascii table as a starbase table (tab-delimited columns).

3. Examples

3.1. Searching a set of light curves for periodic variable stars with Lomb–Scargle

For the first example, suppose we have a collection of light curves that we wish to search for periodicity using the L–S method. Further, suppose each light curve is stored in a separate file with the format shown in listing 4, and that we created a list of light curves as shown in listing 5. These light curves may be searched for periodic signals using the Generalized Lomb–Scargle periodogram as shown in listing 6.

Listing 4: Light curve format for example 3.1.

```
1 > cat 1.txt
3 # Time[BJD] Mag Err
5 53725.173920 10.44080 0.00136
5 53725.176540 10.43881 0.00166
5 53725.177720 10.44024 0.00145
7 ...
```

Listing 5: Light curve list format for example 3.1.

```
1 > cat lclist.txt
3 # FileName
5 1.txt
5 2.txt
7 3.txt
7 ...
```

Here we search for periods between 0.01 and 100.0 days sampling the periodogram for each light curve using frequency steps of $0.1/T$, where T is the total time spanned by the light curve. We only output one peak, which is the highest one found, and we do not store the periodograms. The options “-header” and “-numbercolumns” are used to provide the header given in the output, and to also prepend the column number to the name of each column in the header.

As seen in the output the first light curve has a periodic signal of 77.7776 days, detected with false alarm probability 10^{-5710} and spectroscopic S/N 387. In fact, this particular example light curve has a strong linear trend which is well-fit by a periodic signal with a period longer than the time-span of the data. The second light curve also has a periodic signal, this time with a period of 1.2344 days, detected with false alarm probability $10^{-3999.6}$. This example light curve has a strong sinusoidal signal (Fig. 1).

Listing 7 shows a command which will also output the periodograms for each light curve to the directory “PER” and identify 5 peaks in the periodogram, in each case whitening the light curve (i.e., fitting and removing a harmonic function from the light curve) at the previously identified peak before finding the next one. For clarity we truncate the output shown below, the actual output contains 21 columns, with the period, false alarm probability, periodogram value, and S/N reported for each of the five peaks. Fig. 1 shows an example of the periodogram output for the second light curve, while the format of the periodogram file produced by VARTOOLS is shown in listing 8.

3.2. Processing a FITS-format light curve from Kepler

Listing 9 shows an example of processing the quarter 1 binary FITS-format light curve from the NASA *Kepler* mission (Borucki et al., 2010) for the star KIC 1429092.¹¹ The **-inputlcformat** option indicates the table columns to use. The first column is *BJD* — 2,454,833, the eighth column is the PDC-corrected simple aperture photometry flux, and the ninth column is the uncertainty on this flux. We use the **-changevariable** commands to tell VARTOOLS to use the *pdscap_flux* variable for the light curve brightness measurements, and the *pdscap_flux_err* variable for the uncertainties. Of course in this example we could have read columns 8 and 9 directly into the *mag* and *err* variables, but it can be helpful to keep track of what quantities are actually being input when there are different possibilities. The **-fluxtomag** command converts the flux into magnitudes. We then calculate the r.m.s. scatter of the magnitudes and the L–S periodogram. The magnitude-converted light curve is output to the file “tmp.lc” using the specified format. Lines 32 through 35 of listing 9 show the format of the output light curve.

¹¹ This light curve may be downloaded from <https://archive.stsci.edu/kepler/publiclightcurves.html>.

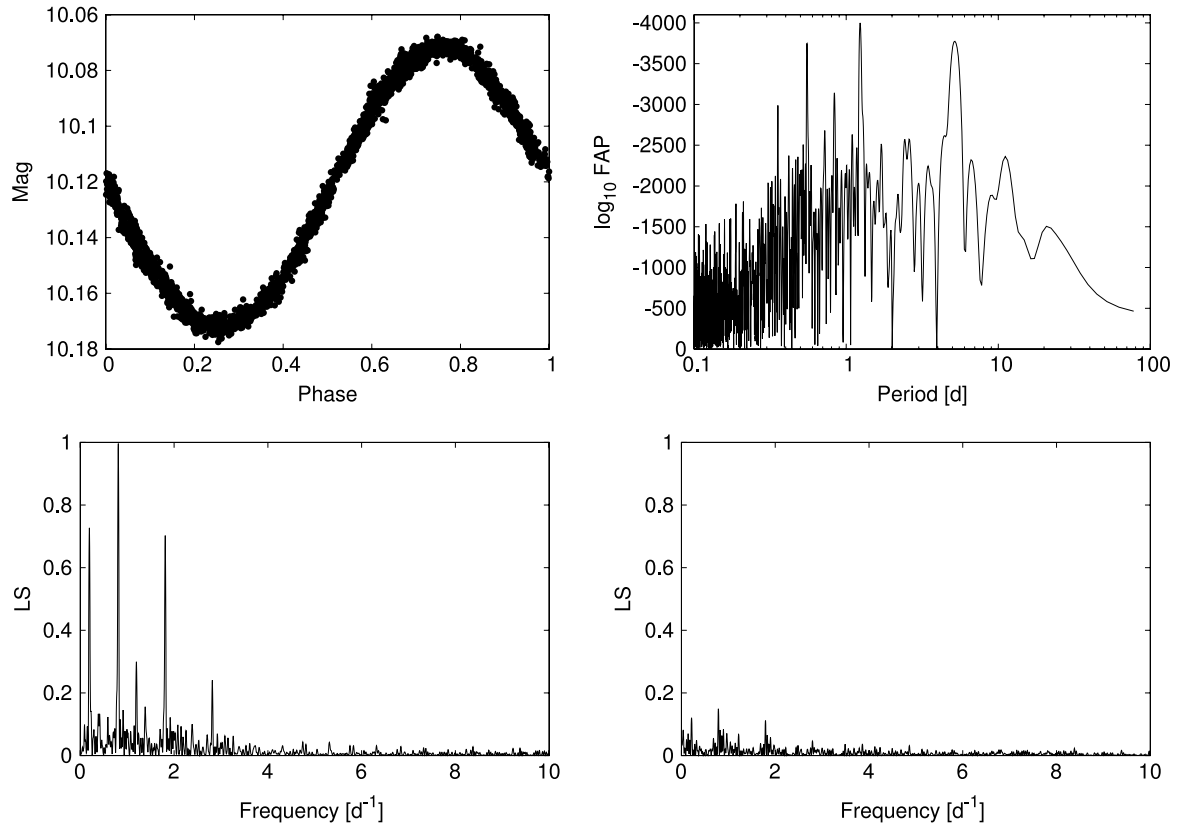


Fig. 1. Periodogram computed in example 3.1 (listing 7) for light curve “2.txt”. The upper left panel shows the light curve phase-folded at the period 1.2344 d found by the `-LS` command. The upper right panel shows the periodogram using period on the horizontal axis and the logarithm of the false alarm probability on the vertical axis. This same periodogram is plotted again in the lower left panel, this time using frequency as the horizontal axis, and the LS statistic (Eq. (1)) on the vertical axis. The lower right panel shows the periodogram after one whitening cycle.

Listing 6: Running the Generalized Lomb-Scargle algorithm on a list of light curves as discussed in example 3.1.

```
1 prompt> vartools -l lclist.txt \
2   -LS 0.01 100.0 0.1 1 0 \
3   -header -numbercolumns
5 #1_Name 2_LS_Period_1_0 3_Log10_LS_Prob_1_0 4_LS_Periodogram_Value_1_0 5_LS_SNR_1_0
6 1.txt 77.76775250 -5709.91013 0.99392 386.76802
7 2.txt 1.23440877 -3999.59411 0.99619 558.03142
8 3.txt 18.29829471 -25.09202 0.03822 38.31823
9 ...
```

Listing 7: Running the Generalized Lomb-Scargle algorithm on a list of light curves, while outputting the periodogram files, and identifying 5 peaks with whitening, as discussed in example 3.1.

```
1 prompt> vartools -l lclist.txt \
2   -LS 0.01 100.0 0.1 5 1 PER whiten \
3   -header -numbercolumns
5 #1_Name 2_LS_Period_1_0 3_Log10_LS_Prob_1_0 4_LS_Periodogram_Value_1_0 \
6   5_LS_SNR_1_0 6_LS_Period_2_0 ...
7 1.txt 77.76775250 -5709.91013 0.99392 683.48184 22.21935786 ...
8 2.txt 1.23440877 -3999.59411 0.99619 825.42315 0.55747493 ...
9 3.txt 18.29829471 -25.09202 0.03822 38.97469 1.15639781 ...
... 
```

Listing 8: Format of a periodogram file produced by the command in listing 7. We truncate the initial header (columns 6 through 11 repeat $P(\omega)$ and the logarithm of the FAP for whitening cycles 2 through 4).

```
prompt> cat PER/2.txt.ls
2
# Column 1 = Frequency in cycles per input light curve time unit.
4 # Column 2 = Unnormalized  $P(\omega)$  (equation 5 of Zechmeister &
# K\urster 2009, A&A, 496, 577). Whitening Cycle 0.
6 # Column 3 = Logarithm of the false alarm probability.
# Whitening Cycle 0.
8 # Column 4 = Unnormalized  $P(\omega)$  (equation 5 of Zechmeister &
# K\urster 2009, A&A, 496, 577). Whitening Cycle 1.
10 # Column 5 = Logarithm of the false alarm probability.
# Whitening Cycle 1.
12 ...
0.012858800310578017 0.0034929037863222533 -463.7738154443843 \
14 0.03192079715170023 -22.681157234188063 0.025954476868188435 \
-16.539118701220865 0.032410425800583273 -20.616443696388661 \
16 0.0022737099428519927 0
...
```

Listing 9: Processing a light curve from the *Kepler* mission, as discussed in example 3.2.

```
prompt> vartools -i kplr001429092-2009166043257_llc.fits \
-inputlcformat t:1,pdcsap_flux:8,pdcsap_flux_err:9 \
3 -changevariable mag pdcsap_flux \
- changevariable err pdcsap_flux_err \
5 -fluxtomag 25.0 0 \
-rms \
7 -LS 0.1 30. 0.1 4 0 \
- o tmp.lc columnformat t,pdcsap_flux:%.10f,pdcsap_flux_err:%.10f \
9 -online

11 Mean_Mag_3 = 14.48415
RMS_3 = 0.00346
13 Expected_RMS_3 = 0.00037
Npoints_3 = 1624
15 LS_Period_1_4 = 8.16381396
Log10_LS_Prob_1_4 = -264.61584
17 LS_Periodogram_Value_1_4 = 0.53223
LS_SNR_1_4 = 7938.63608
19 LS_Period_2_4 = 4.46288496
Log10_LS_Prob_2_4 = -120.94203
21 LS_Periodogram_Value_2_4 = 0.19710
LS_SNR_2_4 = 2939.26680
23 LS_Period_3_4 = 3.84731462
Log10_LS_Prob_3_4 = -11.73784
25 LS_Periodogram_Value_3_4 = 0.01976
LS_SNR_3_4 = 293.78100
27 LS_Period_4_4 = 2.88548597
Log10_LS_Prob_4_4 = -2.35786
29 LS_Periodogram_Value_4_4 = 0.00694
LS_SNR_4_4 = 102.53805
31

prompt> head -3 tmp.lc
33 131.51272409615922 14.4798143174 0.0003752172
131.53315879162255 14.4799784985 0.0003757022
35 131.55359338685957 14.4793099117 0.0003756885
```

3.3. Running the BLS algorithm on a light curve to find transits and eclipses

Listing 10 shows an example of running the BLS algorithm on a light curve containing a transit signal. This example light curve is included with the VARTOOLS distribution, and is produced in a manner similar to that described below in example 3.4.

The parameters are as follows: we consider fractional transit durations between 0.002 and 0.05 in phase, search for periods

between 0.5 and 100 days, use 100,000 frequency steps and 500 phase bins. We use 0 for the timezone (this is used to assign points to an individual night in calculating the fraction of $\Delta\chi^2$ that comes from one night), and report the top five peaks in the periodogram. We output the periodograms to the directory PER, output the best-fit transit model to the directory MODEL, and we do not subtract the model from the light curve. A trapezoid-shaped transit is fit to each peak to refine the transit parameters compared to those found in the full frequency scan. We use the “nobinnedrms” keyword to

Listing 10: Running the BLS algorithm on a light curve, as discussed in example 3.3.

```

1 prompt> vartools -i EXAMPLES/3.transit \
2               -inputlcformat t:1,mag:2,err:3 \
3               -BLS q 0.002 0.05 0.5 100. 100000 500 0 5 1 PER \
4                   1 MODEL 0 fittrap \
5                   nobinnedrms ophcurve MODEL -0.5 0.5 0.001 \
6               -online
7
8 Name                                     = EXAMPLES/3.transit
9 BLS_Period_1_0                           = 2.12316764
10 BLS_Tc_1_0                               = 53727.297790176432
11 BLS_SN_1_0                               = 4.42764
12 BLS_SR_1_0                               = 0.00238
13 BLS_SDE_1_0                              = 4.36608
14 BLS_Depth_1_0                            = 0.01229
15 BLS_Qtran_1_0                            = 0.03609
16 BLS_Qingress_1_0                         = 0.20215
17 BLS_QOTmag_1_0                           = 10.16687
18 BLS_i1_1_0                               = 0.98229
19 BLS_i2_1_0                               = 1.01838
20 BLS_deltaChi2_1_0                        = -24267.20922
21 BLS_fraconenight_1_0                     = 0.43182
22 BLS_Npointsintransit_1_0                 = 166
23 BLS_Ntransits_1_0                        = 4
24 BLS_Npointsbeforetransit_1_0              = 128
25 BLS_Npointsaftertransit_1_0               = 145
26 BLS_Rednoise_1_0                         = 0.00151
27 BLS_Whitenoise_1_0                       = 0.00489
28 BLS_SignaltoPinknoise_1_0                 = 14.54094
29 BLS_Period_2_0                           = 1.85908563
...

```

use $\bar{S}_R$ and σ_{SR} in Eq. (14) for the output BLS_SN columns, rather than $\bar{S}_R(f)$ and $\sigma_{\bar{S}_R}$ (see the discussion in Section 2.2.1.6), and we output a uniformly sampled phase curve for the best-fit transit model to the directory MODEL, to use for plotting.

The results are shown in Fig. 2. The transit is recovered with a period of 2.1232 days, with a spectroscopic S/N of 4.4, and with $S/N_{\text{pink}} = 14.5$ (Eq. (15)). We note that the latter is probably the most useful of the metrics calculated here for selecting transit candidates.

3.4. Simulating and recovering a transiting planet light curve

In this example we simulate a light curve with red-noise, add a transit signal to it, recover the transit with **-BLS**, and then fit a [Mandel and Agol \(2002\)](#) transit model to the light curve. A similar procedure might be repeated millions of times to determine the detection efficiency of a transit survey. The commands are shown in listing 11.

In this case only the times and magnitude uncertainties are read-in from the light curve file, which we use to define the time-base for our simulated observations, and we then initialize all magnitudes to a value of 10 using the **-expr** command.

We use the **-addnoise** command to add time-correlated noise to the light curve, adopting the exponential noise model (Eq. (69)) with $\rho = 0.05$ days, $\sigma_r = 0.0015$ mag, and $\sigma_w = 0.0015$ mag. The **-changererror** command is called to set the magnitude uncertainties equal to the r.m.s. scatter of the light curve, and we use the **-o** command to output the simulated light curve to the file “MODEL/1.simnoise”.

Next comes the **-Injecttransit** command which is used to add the transit signal to the light curve. Here we use the “fix” version of each parameter to specify the value to use on the command line, if one were to carry out a large transit injection/recovery simulation with VARTOOLS, then the “list” or “rand” versions may be used.

The injected signal has a period of 2.123456 days, a planet radius of $1.0 R_J$, a planet mass of $1.0 M_J$, a random phase, a random inclination, e and ω fixed to zero, a stellar mass of $1.0 M_\odot$, a stellar radius of $1.0 R_\odot$, and quadratic limb darkening coefficients of 0.3 and 0.2. We do not output the model, but use the following **-o** command to output the light curve with the transit injected to the file “MODEL/1.injecttransit”.

The **-BLS** command is then used in a similar fashion to the previous example to recover the transit, though in this case we only identify 1 peak in the spectrum. We then use the **-MandelAgolTransit** command to fit a [Mandel and Agol \(2002\)](#) transit model to the light curve, initializing the parameters based on the previous **-BLS** command. We use the same limb darkening coefficients as for the injection, and vary all parameters except for e , ω , and the limb darkening coefficients. We do not attempt to fit an RV curve, and we do not subtract the best-fit model. We output the model together with the model phase curve to the directory “MODEL”. The phase curve (for plotting) runs from phase -0.5 to 0.5 with a phase step of 0.001 .

A selection of the output from this routine is shown in the listing. Fig. 3 shows the simulated noise-only light curve, together with the phase-folded light curve with the transit signal and [Mandel and Agol \(2002\)](#) model overplotted.

3.5. Searching a set of light curves for variable stars, while filtering noise

In this example we perform a general search for variability on a set of light curves after applying two different noise-filtering techniques. Listing 12 shows the format of the light curves for this example. In addition to the usual time, magnitude and uncertainty values provided in the light curve, auxiliary information is provided for each observation including the filename of the image from which the observation is generated, the X and Y

Listing 11: Simulating a light curve with red-noise, adding a transit signal, recovering it with BLS and fitting a Mandel and Agol (2002) transit model to it, as discussed in example 3.4.

```

prompt> vartools -i EXAMPLES/1 \
2      -inputlcformat t:1,err:3 \
      -expr 'mag=10' \
4      -addnoise exp rho fix 0.05 sig_red fix 0.0015 \
      sig_white fix 0.0015 \
6      -changeerror \
      -o MODEL/1.simnoise \
8      -Injecttransit Pfix 2.123456 Rpfix 1.0 Mpfix 1.0 phaserand \
      sinirand eomega efix 0. ofix 0. Mstarfix 1.0 \
10     Rstarfix 1.0 quad ldfix 0.3 0.2 0 \
      -o MODEL/1.injecttransit \
12     -BLS q 0.002 0.05 0.5 100. 100000 500 0 1 1 PER \
      1 MODEL 0 fittrap \
14     nobinnedrms ophcurve MODEL -0.5 0.5 0.001 \
      -MandelAgolTransit bls quad 0.3 0.2 \
16     1 1 1 1 0 0 1 0 0 0 0 1 MODEL \
      ophcurve MODEL -0.5 0.5 0.001 \
18     -oneline

20 Name                      = EXAMPLES/1
   Mean_Mag_2                = 10.00029
22 RMS_2                     = 0.00222
   Npoints_2                 = 3122
24 Injecttransit_Period_4     = 2.12345600
   ...
26 Injecttransit_phase_4      = 0.34066
   Injecttransit_sin_i_4      = 0.99940
   ...
28 BLS_Period_1_6             = 2.12739225
30 BLS_Tc_1_6                 = 53726.949086475011
   BLS_SN_1_6                 = 3.95738
   ...
32 BLS_Depth_1_6              = 0.01207
   ...
34 BLS_Npointsintransit_1_6   = 120
36 BLS_Ntransits_1_6          = 3
   BLS_Npointsbeforetransit_1_6 = 169
38 BLS_Npointsaftertransit_1_6 = 69
   BLS_Rednoise_1_6           = 0.00113
40 BLS_Whitenoise_1_6         = 0.00221
   BLS_SignaltoPinknoise_1_6   = 17.60859
42 BLS_Period_invtransit_6     = 6.45330708
   BLS_deltaChi2_invtransit_6  = -327.05444
44 BLS_MeanMag_6              = 10.00068
   MandelAgolTransit_Period_7  = 2.12519667
46 MandelAgolTransit_T0_7     = 53726.95486357
   MandelAgolTransit_r_7       = 0.09941
48 MandelAgolTransit_a_7      = 6.80453
   MandelAgolTransit_bimpact_7 = 0.26979
50 MandelAgolTransit_inc_7    = 87.72772
   ...
52 MandelAgolTransit_chi2_7    = 0.99461

```

Listing 12: Light curve format for example 3.4.

```

prompt> cat 1.txt
2
# Image  Time[BJD]  Mag    Err    X      Y      FWHM   Ellipticity  PA   Airmass
4  ....
   ....

```

coordinates of the source on the image, the full-width at half maximum (FWHM), ellipticity and position angle of a Gaussian fit to the point spread function (PSF), and the airmass of the observation.

Listing 13 shows the procedure. We will calculate five variability statistics: (1) the r.m.s. scatter of the light curve; (2) the Generalized L–S periodogram; (3) the BLS spectrum; (4) Stetson’s J statistic; and (5) the harmonic AoV periodogram. This will be

done three times for each light curve: on the raw (un-filtered) input light curve; on the light curve after decorrelating against the external parameters X , Y , FWHM, Ellipticity, PA and Airmass; and on the light curve after applying TFA filtering in addition to the decorrelation. To speed up the processing, this is run in parallel on the light curves.

Parameters are chosen for the commands as follows. For L–S we scan periods from 0.1 to 100 days with a subsample factor of 0.1. We only report the top peak, and we do not output the periodogram. For BLS we consider fractional transit durations between 0.002 and 0.05 in phase, search for periods between 0.5 days and 100 days, use 100,000 frequency steps and 500 phase bins. We use 0 for the timezone (this is used to assign points to an individual night in calculating the fraction of $\Delta\chi^2$ that comes from one night), and report the top five peaks in the periodogram. We do not output the periodogram or the best-fit transit model, and we do not subtract the model from the light curve. A trapezoid-shaped transit is fit to each peak to refine the transit parameters compared to those found in the full frequency scan. For Stetson's J we use a timescale of 0.5 days to distinguish between “near” and “far” observations, and “dates.txt” is a file listing all of the dates in the light curves. For the harmonic AoV we use a single harmonic model (i.e., a simple sinusoid), search for periods between 0.1 and 100 days with a sub-sample factor of 0.1. A higher resolution scan, with a subsample of 0.01 is performed around each peak. We only report the top peak in the periodogram, and we do not output the periodogram. Note that the parameters chosen for this example are just to illustrate the use of the program, and are not meant to be best or default choices. In practice all parameters need to be carefully considered based on the properties of the light curves being analyzed.

After calculating the statistics for the raw light curve, we fit a model that is a linear function of all of the external parameters (in practice this is unlikely to be a very good model, one should inspect the light curves to determine the best function to use for filtering). This is then subtracted from the light curve using the ‘correctlc’ keyword. The linear coefficients which we optimize are labeled a_0 through a_8 . The terms ‘ $\sin(2.0\pi x)$ ’ and ‘ $\sin(2.0\pi y)$ ’ give a term that is periodic in one pixel. Light curves generated through aperture photometry commonly have systematic variations that are correlated with terms such as these. Following the ‘-linfit’ call, we use ‘-expr’ to set the light curve mean back to its pre-filtered value (subtracting the linear model causes the mean to be close to zero). This is achieved by adding the difference between the mean magnitude computed by ‘-rms’ before ‘-linfit’ and the mean magnitude computed after it. These are referenced by their output column names (“Mean_Mag_0” and “Mean_Mag_6”). The two calls to “-o” are used to output the light curves after each filtering step. The decorrelated light curves are output to the directory LCEPD, while the EPD+TFA-filtered light curves are output to the directory LCTFA.

To run TFA on the light curves we provide a list of files to be used as TFA templates in ‘template.txt’. These light curves are assumed to have the same format as the input light curves, so we use the readformat option to indicate that column 1 will be used for matching points (this is the image id) and column 3 for the magnitudes. To use the image ids for matching points in the light curves to points in the templates, rather than the JD values, the ‘-matchstringid’ option is given at the end of the call to VARTOOLS. A list of all image ids is given in the file ‘imagelist.txt’. Any template star that is within 10 pixels of the light curve being filtered will be removed from the template for that light curve. The xycol option is used to indicate that the average X and Y positions of the sources are read from columns 2 and 3 of the input light curve list file. The light curves passed on from this command will have the filtering

applied, but we do not output the TFA coefficients or the model trend signals.

Running this command on a set of light curves produces an ascii table with 381 columns, which may be daunting to read. For convenience, Listing 14 shows a method for writing out the column names only, in a human-readable list. In this case one would fill in the ‘...’ with the full command given in listing 13, and the ‘-headeronly’ option causes VARTOOLS to output the header and quit without processing any light curves. The calls to the standard command-line utilities ‘sed’ and ‘awk’ are used to reformat the names into a readable list, including blank lines between the columns produced by each command. We show the first few column names below the command. A simple selection of variables might be to select any light curve with $\log_{10} \text{FAP}_{\text{LS}} < -100$, a BLS signal-to-pink-noise greater than 10, $J_{\text{stet}} > 1.5$, or $\log_{10} \text{FAP}_{\text{AoV}} < -100$. This can be done, for example, by applying a simple ‘awk’ script to the output. In practice these thresholds would need to be adjusted based on the noise properties of the survey (for example, the threshold of $\log_{10} \text{FAP}_{\text{LS}} < -100$ is an extremely low false alarm probability which is a reasonable threshold for highly time-correlated noise, but far too conservative a threshold if the light curves actually have Gaussian white noise).

3.6. Trend-filtering while preserving stellar variability with signal-reconstruction TFA

In this example we show the use of signal-reconstruction-mode TFA (**-TFA_SR**) to filter systematic noise from a light curve while preserving real variability. Here the variability signal is modeled using a harmonic series. The command is shown in listing 15. A periodic signal is first identified with the **-LS** command. We then save the light curve using the **-savevc** command, to allow us to try different filtering techniques in the same call to VARTOOLS. The **-Killharm** command is used to subtract the signal from the light curve, and the call to **-rms** calculates the scatter of the residuals. Calling **-restorelc 1** restores the light curve to its state at the **-savevc** command. We next run **-TFA** to apply the TFA method in non-reconstructive mode. The trends are read from the file “EXAMPLES/trendlist_tfa”, the list of dates from the file “EXAMPLES/dates_tfa”, we use a separation of 25.0 pixels to remove close neighbors from the trend list, we specify the columns in the input list from which to read the X and Y pixel positions of the source, and we subtract the model from the light curve but do not output the trend coefficients or model signal. After **-TFA** we output the filtered light curve, and then subtract the signal with **-Killharm** and calculate the scatter of the residuals with **-rms**. Finally we restore the light curve again to its original state, and then filter using **-TFA_SR**. We use mostly the same options as for **-TFA**, but now we output the coefficients and model trends to the directory “EXAMPLES/OUTDIR1”. We do not apply TFA first, we use a stopping threshold of 0.001 and a maximum number of iterations of 100 (in this case, where we are using a harmonic series to model the signal, the trend and model are fit simultaneously to the data so these parameters have no affect), we use a “harmonic series to model the signal with zero higher-order harmonics and no sub-harmonics, and we take the period from the prior **-LS** command. Again we output the light curve, subtract the signal, and compute the scatter of the residuals.

By comparing the values for “RMS_4”, “RMS_8” and “RMS_13” one can see the affect on the scatter in the residuals from the different filtering techniques. Applying **-TFA** in non-reconstructive model increases the scatter from 2.3 to 7.2 mmag, but when the filtering is run in reconstructive mode the residual scatter decreases to 2.1 mmag. Fig. 4 compares the phase-folded light curves after **-TFA** and after **-TFA_SR** to the original light curve,

Listing 13: VARTOOLS procedure for carrying out a general search for variability on a set of light curves, including different levels of filtering, as discussed in example 3.5.

```

1 prompt> vartools -l inputlist.txt \
2   -inputlcformat id:1,t:2,mag:3,err:4,x:5,y:6,fw:7,el:8,pa:9,air:10 \
3   -rms \
4   -LS 0.1 100. 0.1 1 0 \
5   -BLS q 0.002 0.05 0.5 100. 100000 500 0 5 0 0 0 fittrap \
6   -Jstet 0.5 dates.txt \
7   -aov_harm 1 0.1 100. 0.1 0.01 1 0 \
8   -linfit \
9   'a0+a1*x+a2*y+a3*fw+a4*el+a5*pa+a6*air+a7*sin(2.0*pi*x)+a8*sin(2.0*pi*y)' \
10  a0,a1,a2,a3,a4,a5,a6,a7,a8 correctlc \
11  -rms \
12  -expr 'mag=mag+(Mean_Mag_0-Mean_Mag_6)' \
13  -o LCEPD \
14  -LS 0.1 100. 0.1 1 0 \
15  -BLS q 0.002 0.05 0.5 100. 100000 500 0 5 0 0 0 fittrap \
16  -Jstet 0.5 dates.txt \
17  -aov_harm 1 0.1 100. 0.1 0.01 1 0 \
18  -TFA trendlist.txt readformat 0 1 3 imagelist.txt 10 xycol 2 3 1 0 0 \
19  -o LCTFA \
20  -rms \
21  -LS 0.1 100. 0.1 1 0 \
22  -BLS q 0.002 0.05 0.5 100. 100000 500 0 5 0 0 0 fittrap \
23  -Jstet 0.5 dates.txt \
24  -aov_harm 1 0.1 100. 0.1 0.01 1 0 \
25  -parallel 16 \
26  -header -numbercolumns -matchstringid

```

Listing 14: A technique for printing out the column names for a VARTOOLS procedure in a human-readable list. In this case the call to VARTOOLS is the same as in listing 13, but with the “-headeronly” option used to output only the command headers. The procedure is discussed in example 3.5.

```

1 prompt> vartools -l inputlist.txt \
2   -inputlcformat id:1,t:2,mag:3,err:4,x:5,y:6,fw:7,el:8,pa:9,air:10 \
3   -rms \
4   ...
5   -header -numbercolumns -matchstringid \
6   -headeronly | \
7   sed 's|#||g' | \
8   awk '{c = "";
9     for(i=1; i <= NF; i += 1) {
10       n = split($i,s,"_");
11       if(s[n] != c) print "";
12       c = s[n];
13       print $i;
14     }},
15
16 1_Name
17
18 2_Mean_Mag_0
19 3_RMS_0
20 4_Expected_RMS_0
21 5_Npoints_0
22
23 6_LS_Period_1_1
24 ...

```

where it can be seen that the signal-reconstruction correctly preserves the signal shape. Please note that this example is concocted primarily to illustrate the syntax and operation of the **-TFA_SR** command. The noise-filtering in this example is not terribly effective, but in practice one would likely use a much larger set of template light curves, chosen in an intelligent fashion, which, if the light curves share systematic trends, would reduce the scatter of the residuals even further.

3.7. Using the weighted wavelet Z-transform to characterize a quasi-periodic signal

Listing 17 shows an example of using the **-wwz** command to calculate the Weighted Wavelet Z-Transform of a light curve. The transform is calculated up to a maximum frequency of 2.0 d^{-1} with a frequency step of $0.25/T$ where T is the time base-line of the light curve. We use the “auto” keyword for both “tau0” and “tau1” to consider time-shifts running from the first

Listing 15: Example of filtering a light curve with TFA in non-reconstructive and reconstructive modes, as discussed in example 3.6.

```

prompt> vartools -l EXAMPLES/lc_list_tfa_sr_harm -online -rms \
2      -LS 0.1 10. 0.1 1 0 \
      -savelc \
4      -Killharm ls 0 0 0 \
      -rms -restorelc 1 \
6      -TFA EXAMPLES/trendlist_tfa EXAMPLES/dates_tfa \
      25.0 xycol 2 3 1 0 0 \
8      -o EXAMPLES/OUTDIR1 nameformat 2.test_tfa_nosr \
      -Killharm ls 0 0 0 \
10     -rms -restorelc 1 \
      -TFA_SR EXAMPLES/trendlist_tfa EXAMPLES/dates_tfa \
12     25.0 xycol 2 3 1 \
      1 EXAMPLES/OUTDIR1 1 EXAMPLES/OUTDIR1 \
14     0 0.001 100 harm 0 0 period ls \
      -o EXAMPLES/OUTDIR1 nameformat 2.test_tfa_sr_harm \
16     -Killharm ls 0 0 0 \
      -rms \
18     -online

20 Name                                     = EXAMPLES/2
Mean_Mag_0                               = 10.11802
22 RMS_0                                  = 0.03663
Expected_RMS_0                           = 0.00102
24 Npoints_0                             = 3313
LS_Period_1_1                             = 1.23440877
26 Log10_LS_Prob_1_1                     = -4000.59209
LS_Periodogram_Value_1_1                  = 0.99619
28 LS_SNR_1_1                             = 45.98308
Killharm_Mean_Mag_3                       = 10.12217
30 Killharm_Period_1_3                     = 1.23440877
Killharm_Per1_Fundamental_Sincoeff_3      = 0.05008
32 Killharm_Per1_Fundamental_Coscoeff_3    = -0.00222
Killharm_Per1_Amplitude_3                 = 0.10026
34 Mean_Mag_4                             = 10.11176
RMS_4                                     = 0.00231
36 Expected_RMS_4                         = 0.00102
Npoints_4                                 = 3313
38 TFA_MeanMag_6                           = 10.11766
TFA_RMS_6                                 = 0.03555
40 Killharm_Mean_Mag_7                     = 10.12211
Killharm_Period_1_7                       = 1.23440877
42 Killharm_Per1_Fundamental_Sincoeff_7    = 0.04802
Killharm_Per1_Fundamental_Coscoeff_7      = -0.00268
44 Killharm_Per1_Amplitude_7               = 0.09620
Mean_Mag_8                               = 10.11169
46 RMS_8                                   = 0.00725
Expected_RMS_8                           = 0.00102
48 Npoints_8                               = 3313
TFA_SR_MeanMag_10                         = 10.11788
50 TFA_SR_RMS_10                           = 0.03642
Killharm_Mean_Mag_12                      = 10.12210
52 Killharm_Period_1_12                    = 1.23440877
Killharm_Per1_Fundamental_Sincoeff_12     = 0.04986
54 Killharm_Per1_Fundamental_Coscoeff_12   = -0.00237
Killharm_Per1_Amplitude_12                = 0.09984
56 Mean_Mag_13                             = 10.11166
RMS_13                                   = 0.00210
58 Expected_RMS_13                        = 0.00102
Npoints_13                               = 3313

```

observation to the last, and use a time-shift step of 0.1 days. The full transform is output to the file “EXAMPLES/OUTDIR1/8.wwz” while the projection of the wavelet onto the time-shift axis is output to the file “EXAMPLES/OUTDIR1/8.mwwz”. These are plotted in Fig. 5. We also show the light curve in Fig. 6. For the full 2-d transform we use the “pm3d” keyword to output the file in a format that is convenient for plotting with the GNUPLLOT program. The commands for plotting this are shown at the bottom of

Listing 17. The data has a signal with a frequency of 0.3065 day^{-1} ($P = 3.2632 \text{ days}$) centered at the time $JD - 53725 = 10.174$. The signal is not present at later times, while a lower significance signal with a frequency of $\sim 0.2 \text{ day}^{-1}$ may be present at earlier times. This particular light curve comes from the MMT photometric survey of M37 presented by Hartman et al. (2008), and may be a spotted star with an evolving surface brightness distribution.

Listing 16: Example of calculating the Weighted Wavelet Z-Transform for a light curve, as discussed in example 3.7.

```

1 prompt> vartools -i EXAMPLES/8 -online \
2 -wwz maxfreq 2.0 freqsamp 0.25 tau0 auto tau1 auto dtau 0.1 \
3 outfulltransform EXAMPLES/OUTDIR1/ pm3d \
4 outmaxtransform EXAMPLES/OUTDIR1
5
6 Name = EXAMPLES/8
7 MaxWWZ_0 = 345.87167006071132
8 MaxWWZ_Freq_0 = 0.30645161290322581
9 MaxWWZ_TShift_0 = 53735.173920000001
10 MaxWWZ_Power_0 = 243.91899737448088
11 MaxWWZ_Amplitude_0 = 0.0019655476933700114
12 MaxWWZ_Neffective_0 = 1651.192187564548
13 MaxWWZ_AverageMag_0 = 10.611015587079519
14 Med_WWZ_0 = 135.53839042847693
15 Med_Freq_0 = 0.20967741935483872
16 Med_Power_0 = 116.72893440099938
17 Med_Amplitude_0 = 0.00096335163669205139
18 Med_Neffective_0 = 2188.0750923512305
19 Med_AverageMag_0 = 10.611486630808924
20
21 prompt> gnuplot
22 gnuplot> set pm3d map
23 gnuplot> unset key
24 gnuplot> splot 'EXAMPLES/OUTDIR1/8.wwz' u 1:2:3

```

3.8. Performing an MCMC fit of a non-linear function to a light curve

Listing 17 shows an example of fitting an analytic function that is non-linear in its parameters to a light curve, and using an MCMC procedure to determine the uncertainties. The initial **-stats** command, together with the following **-expr** commands, are used to add a Gaussian function to a light curve. The **-stats** command is used to find the minimum and maximum times in the light curve, which are stored in the variables `STATS_t_MIN_0` and `STATS_t_MAX_0`. The first two calls to **-expr** define the variables `t1` and `Dt` in terms of the minimum and maximum values, and then the third call to **-expr** adds the Gaussian with amplitude 0.1 mag, standard deviation $0.05Dt$ and peak time $t1 + Dt * 0.2$. For our illustrative call to **-nonlinfit** we provide the analytic function to fit to the light curve in terms of the free parameters a , b , c and d . We then set the initial values and uncertainty steps for each of these parameters on line 7 of listing 17. We use the “mcmc” keyword to indicate that the DEMCMC procedure should be run, stopping it after a total of 10,000 links (in practice one would typically run more links, a relatively small number is used here to minimize the computing time for this simple example). The full MCMC chain will be output to the file “EXAMPLES/OUTDIR1/3.mcmc”. The output table from VARTOOLS gives the best-fit (maximum likelihood) values for the free parameters, together with the value of χ^2 for those parameters. Following this, the median and standard deviation of the MCMC chain for each parameter is provided. These can be taken as measures of the most likely values for each parameter, together with their 1σ uncertainties. Following the output from VARTOOLS, lines 28 through 33 of listing 17 show the format of the file containing the full MCMC chain.

3.9. Converting the times in a light curve from UTC to BJD

Listing 18 shows an example of converting the times in a light curve from UTC to TDB-corrected BJD. Lines 1–4 show the format of the input light curve file “EXAMPLES/1.UTC”, where the first column gives the UTC times of the observations. By default VARTOOLS will attempt to read the time column directly into a double-precision variable, which it interprets as the JD. To convert

the input time from UTC to JD, we use the call to **-inputlformat** on line 7 of the listing. Here we indicate that the time will be read from the first column, that it is in “UTC” string format, and we then indicate how the string should be parsed, where “%Y” is interpreted as the year, “%M” is the month, “%D” is the day, “%h” is the UT hour, “%m” is the minute, and “%s” are the seconds. Finally we read the magnitude and uncertainties from columns 2 and 3, respectively. By default these are read in as double-precision variables, so we do not need to specify a variable type or format for these quantities. Lines 8 through 14 of the listing execute the time conversion. We first need to specify the input time system, which we do on line 8 (it is JD, and we use the “inputsys-utc” keyword to indicate that the input times have not been corrected for leap-seconds). We then indicate the output time system on line 9. Here we will convert to BJD, and will subtract 2,400,000 from the result, and we will convert to the TDB system to correct the times for leap-seconds. To convert to BJD we need to tell VARTOOLS the coordinates of the star, which we do on line 10. We also need to provide a set of files from JPL which provide the data for calculating positions in the solar system (“ephemfile”), the list of leap-seconds that have occurred since 1970 (“leapsecfile”) and the detailed position of locations on the surface of the Earth with respect to the center of the Earth–Moon system as a function of time (“planetdatafile”).¹² Line 14 indicates the observatory where the observations were made (in this case, Fred Lawrence Whipple Observatory in Arizona). Finally on line 15 we output the light curve to the file “EXAMPLES/OUTDIR1/1.bjdtodb”, whose format may be seen on lines 17–20 of the listing.

4. Performance tests

Here we present performance tests to compare the time required for I/O between ASCII and Binary format data, and to also determine the execution time for various period-finding commands. We focus here on the periodic signal detection routines

¹² The relevant files may be obtained from ftp://naif.jpl.nasa.gov/pub/naif/generic_kernels/.

Listing 17: Running an MCMC procedure to fit a function that is non-linear in its parameters to a light curve, as discussed in example 3.8.

```

prompt> vartools -i EXAMPLES/3 \
2   -stats t min,max \
   -expr t1=STATS_t_MIN_0 \
4   -expr 'Dt=(STATS_t_MAX_0-STATS_t_MIN_0)' \
   -expr 'mag=mag+0.1*exp(-0.5*((t-(t1+Dt*0.2))/(Dt*0.05))^2)' \
6   -nonlinfit 'a+b*exp(-(t-c)^2/(2*d^2))' \
   'a=10.167:0.0002,b=0.1:0.0008,c=(t1+Dt*0.2):(0.005),d=(Dt*0.05):(0.016)' \
8   mcmc Nlinkstotal 10000 outchains EXAMPLES/OUTDIR1/ \
   -online

Name                               = EXAMPLES/3
12 STATS_t_MIN_0                    = 53725.173920000001
   STATS_t_MAX_0                    = 53756.281021000003
14 Nonlinfit_a_BestFit_4            = 10.16733595824326
   Nonlinfit_b_BestFit_4            = 0.10068199797234947
16 Nonlinfit_c_BestFit_4            = 53731.405627923843
   Nonlinfit_d_BestFit_4            = 1.4997732382607727
18 Nonlinfit_BestFit_Chi2_4         = 90077.83071503813
   Nonlinfit_a_MEDIAN_4            = 10.167339206527549
20 Nonlinfit_a_STDDEV_4             = 2.1634712351800478e-05
   Nonlinfit_b_MEDIAN_4            = 0.10071313795015273
22 Nonlinfit_b_STDDEV_4             = 7.5501039462404719e-05
   Nonlinfit_c_MEDIAN_4            = 53731.40592424116
24 Nonlinfit_c_STDDEV_4             = 0.0010974050260291169
   Nonlinfit_d_MEDIAN_4            = 1.4987003116627102
26 Nonlinfit_d_STDDEV_4            = 0.001632159684888244

prompt> head -3 EXAMPLES/OUTDIR1/3.mcmc
# a b c d -2ln(L)
30 10.167347849788445 0.10070890704157852 53731.406660238252 1.4983339944179459 \
   90080.167413138683
32 10.167330685303858 0.10072766325485784 53731.406153148055 1.5002348151196629 \
   90081.07747374264

```

Listing 18: Converting the times in a light curve from UTC to BJD, as discussed in example 3.9.

```

1 prompt> head -3 EXAMPLES/1.UTC
   2005-12-20T16:10:26.69 10.085 0.00119
3   2005-12-20T16:14:13.06 10.0847 0.00144
   2005-12-20T16:15:55.01 10.0825 0.00123
5
prompt> vartools -i EXAMPLES/1.UTC -quiet \
7   -inputlcformat 't:1:utc:%Y-%M-%DT%H:%m:%s,mag:2,err:3' \
   -converttime input jd inputsys-utc \
9   output bjd outputsubtract 2400000. outputsys-tdb \
   radec fix 88.079166 32.5533 \
11  ephemerfile CSPICEKERNELS/de432s.bsp \
   leapsecfile CSPICEKERNELS/naif0010.tls \
13  planetdatafile CSPICEKERNELS/pck00010.tpc \
   observatory flwo \
15  -o EXAMPLES/OUTDIR1/1.bjdtdb

17 prompt> head -3 EXAMPLES/OUTDIR1/1.bjdtdb
   53725.180285235 10.08500 0.00119
19  53725.182905252 10.08470 0.00144
   53725.184085226 10.08250 0.00123

```

as these are generally the time-limiting components of processing pipelines for large datasets.

The following tests were carried out on a machine with 32 AMD Opteron 6140 64-bit CPUs, each with a maximum clock speed of 2.6 GHz. The machine runs Debian GNU/Linux version 7.8 with Linux kernel version 3.2.0-4-amd64. VARTOOLS was compiled on this machine with gcc version 4.7.2 using O2 optimization, and was linked against all supported external libraries. Here we are primarily interested in the processing time for different

routines, so to minimize I/O overhead light curves were read-in from shared memory (/run/shm/ in the listings below; this is a tmpfs file-system on this machine, and we confirmed that there was sufficient free memory during our tests that the files placed here were stored in RAM rather than swap space) and all output was ignored (redirected to /dev/null; we do not use the **-quiet** option, so time is still spent generating the ASCII table). Except where stated otherwise, all processing was performed on the light curve stored in “EXAMPLES/2.fits”, which is a binary FITS

table containing 3313 observations, and which is included in the VARTOOLS distribution.

4.1. Test 1: ASCII vs. binary I/O

There are four processes which contribute to the time required to input and output a light curve: (1) reading data from disk into memory; (2) parsing the raw data chunks into the appropriate variables within VARTOOLS; (3) transforming data from the vector variables back into the appropriate format for storage on disk; (4) writing the data to disk. If the numerical data are stored in ASCII format then steps (2) and (3), which involve parsing ASCII strings into binary data and vice versa, can be time consuming. If the data is stored in binary format then steps (2) and (3) will be significantly faster (or not applicable, depending on the data organization). Storing the data in binary format also requires less total space than storing in ASCII to the same precision, reducing the time required for steps (1) and (4) as well. The primary benefit to using ASCII format is that the data are easily human-readable with widely available software tools, may be edited with simple text editors, and may be processed using a wide range of tools which expect ASCII format data (e.g., standard shell tools such as AWK or SED).

To help in evaluating whether the time and storage savings from using binary data are worth the possible tradeoffs with data transparency, we carry out tests to evaluate the I/O performance of VARTOOLS for handling both types of data.

The time required to read and write data from/to disk is highly system-dependent, and will vary in time as well, depending, for example, on whether the data have been recently accessed and are stored in cache memory. Regardless of the system, the time difference between ASCII and binary format data for steps (1) and (4) should be proportional to the difference in file-size. Here we focus on testing steps (2) and (3) and attempt to minimize the time required for steps (1) and (4) by using shared memory for the file storage. We first generate an ASCII light curve using the command:

```
prompt> \
2 echo 0 | \
  awk ' \
4   function gauss(u,v) { \
      u = sqrt(-2*log(rand())); \
6     v = 4*atan2(1,0)*rand(); \
      return(u*cos(v)); \
8   } \
  { \
10    for(i=1; i <= 1000000; i += 1) { \
      printf "%.17g %.17g %.17g\n", \
12      2450000 + i/100000, \
      gauss(), gauss(); \
14    } \
  }, \
16 > /run/shm/test_ascii.txt

18 prompt> head -3 /run/shm/test_ascii.txt
2450000.0000100001 -0.43249305127101612 \
20 0.33363186465509398
2450000.0000200002 0.35977598533128757 \
22 0.29950390190544596
2450000.0000300002 0.69568490442241193 \
24 -1.4140794262294938
```

Where the `gauss()` function used in the AWK script is a simple routine for generating Gaussian random numbers, and where we use "%.17g" to write the ASCII data to full double-precision.

To determine the time for ASCII input only we carry out the following:

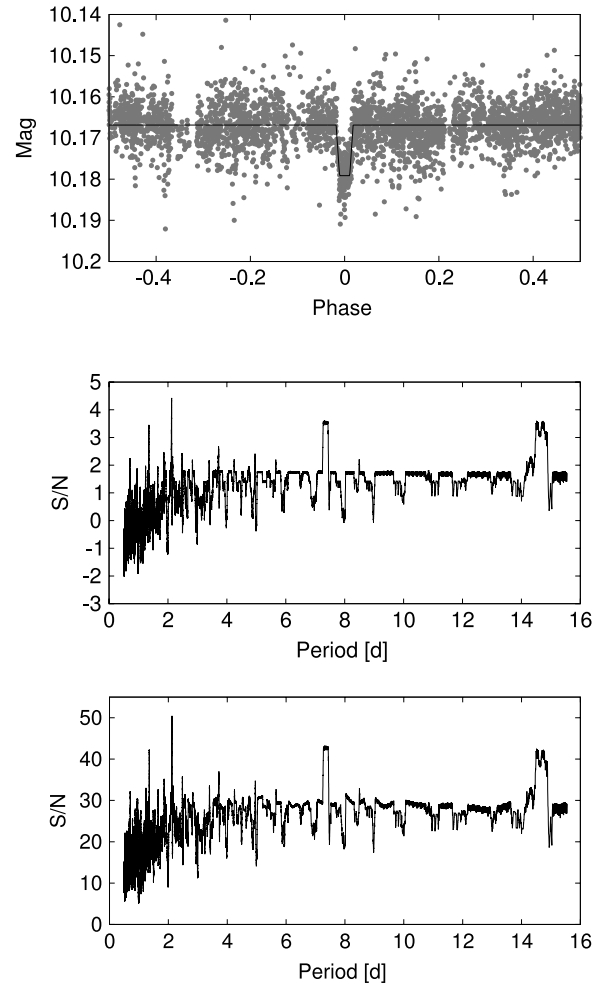


Fig. 2. Phase-folded light curve and BLS spectra for example 3.3 (listing 10). Top: Phase-folded light curve as output by the `-BLS` command, together with the model phase curve (output using the `ophcurve` keyword). Middle: The BLS spectrum using the spectroscopic S/N when the `"nobinnedrms"` keyword is given (i.e., this is equivalent to SR, with the average value subtracted, and divided by the standard deviation). Bottom: The BLS spectrum using the spectroscopic S/N when the `"nobinnedrms"` keyword is not given (Eq. (14)). The primary difference is a change in scale due the difference between $\bar{S}R$ (Eq. (10)) and $\bar{S}R$ (Eq. (13)).

```
prompt> time \
2 vartools -i /run/shm/test_ascii.txt \
  -if 0 -rms -fi > /dev/null
4
6 real    0m2.340s
  user    0m2.240s
  sys     0m0.080s
```

Here the expression `"-if 0 -rms -fi"` is included to allow VARTOOLS to execute without performing any processing of the light curve (if no commands are given to VARTOOLS then it will terminate without reading the light curve, with this expression the light curve will be read-in and parsed into the appropriate variables, but the conditional placed in front of `"-rms"` will prevent it from executing). This test indicates that it takes VARTOOLS ~2.3 s of user + system time on our test machine to parse a 3-column ASCII table with 1,000,000 rows and with each column containing 17 digits. Repeated execution shows a variance of less than 0.1 s in the execution time. Note that running on an empty light curve takes less than 0.05 s so the initialization of VARTOOLS is a negligible

fraction of this time. Using the “-l” option to execute on multiple copies of this same light curve shows that the time scales linearly with the number of light curves. Reducing the light curve size by half also reduces the time by half. If each of the input columns has 8 digit-precision rather than 17 digit-precision, then the process takes 1.5 s. If we use the “-inputlformat t:0,mag:0,err:0” option, in which case the light curve is read-from disk and the number of lines and columns are determined (this is done whenever VARTOOLS reads in a light curve and requires a scan over all ASCII characters in the file), but the ASCII data are not converted to binary, then the process takes 1.2 s to execute. Finally, if we do also include the **-rms** command (i.e., the “-if 0” and “-fi” expressions are removed) the difference in time is negligible (less than the variance between repeated executions of the command), indicating that calculating the r.m.s. scatter of ASCII-format light curves is an I/O limited process, dominated by the ASCII to binary conversion, if not by the disk access itself.

To determine the time for ASCII output we carry out the following:

```
1 prompt> time \
2     vartools -i /run/shm/test_ascii.txt \
3     -o test_ascii_out.txt > /dev/null
4
5 real    0m4.353s
6 user    0m4.216s
7 sys     0m0.116s
```

which, together with the 2.3 s measured above for input, indicates that it takes ~ 2 s on this machine to output an ASCII light curve (primarily step (3)) with 1,000,000 lines to shared memory.

To repeat the input-only test for a binary FITS light curve, we carry out the following:

```
1 prompt> vartools -i test_ascii.txt \
2     -o test_fits.fits
3
4 prompt> for i in $(seq 1 100) ; do \
5     echo test_fits.fits \
6     done > list_fits
7
8 prompt> time \
9     vartools -l list_fits \
10    -if 0 -rms -fi \
11    > /dev/null
12
13 real    0m10.558s
14 user    0m7.092s
15 sys     0m3.436s
```

The first command is used to convert the ASCII light curve to a binary FITS table. The second is used to generate a list with 100 repeated entries of the same light curve (this is done because we found that the input time for one of these light curves is not significantly longer than the time required to start VARTOOLS), and the third performs the time test. Based on this test we find that it takes ~ 0.1 s on this machine to read-in a FITS light curve with 1,000,000 rows and 3 double-precision columns (roughly 2/3 of the time is user time, the other third is system time). If we use the “-inputlformat t:0,mag:0,err:0” option, then only 20 ms is required, with negligible system time.

To determine the output time for FITS we carried out the following test:

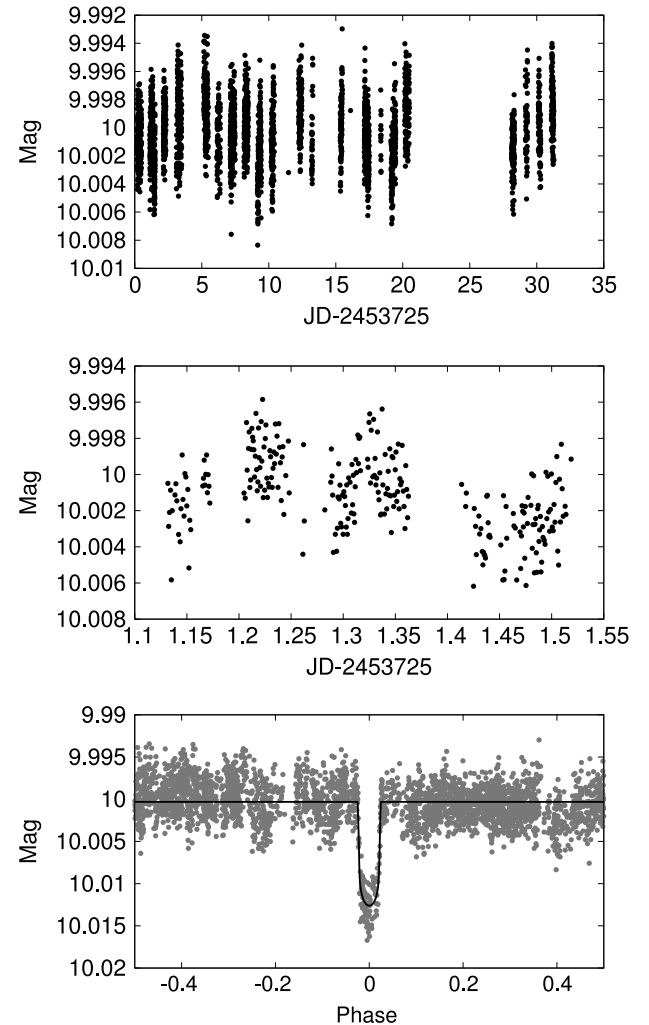


Fig. 3. Top: a simulated light curve with time correlated noise generated in example 3.4. This is the light curve produced by the **-o** command on line 7 of listing 11. Middle: the same light curve in the top panel, zooming in on a single night. Bottom: the phase-folded light curve after injecting the transit signal, recovering it with **-BLS** and modeling it with the **-MandelAgolTransit** command. The best-fit transit model is over-plotted. This is the light curve and model output through the **-MandelAgolTransit** command on line 16 of listing 11.

```
1 prompt> vartools -l list_fits \
2     -o /run/shm/ \
3     nameformat test_fits_out.fits \
4     fits \
5     > /dev/null
6
7 real    0m30.656s
8 user    0m17.133s
9 sys     0m13.469s
```

Where the “nameformat” option is used to output the light curve to a different name than its input file. Subtracting the input time determined above we find that it takes ~ 0.2 s to output the FITS light curve with 1,000,000 rows and 3 double-precision columns, or roughly twice as long as input.

Altogether we estimate that neglecting additional differences in disk access time when the data are stored on a physical hard disk, the input time for an ASCII light curve is approximately 20 times longer than for a FITS-table, while the output time is approximately

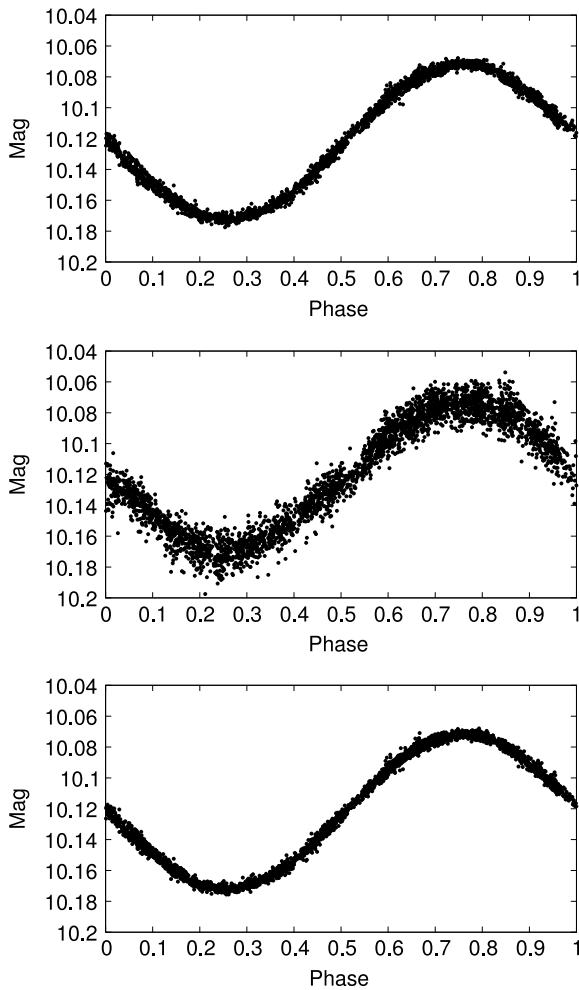


Fig. 4. Top: phase-folded light curve of a simulated variable star before applying noise filtering as discussed in example 3.6. This is the input light curve processed in listing 15. Middle: the phase-folded light curve after processing with the non-reconstructive-mode **-TFA** command (i.e., that output on line 8 of listing 15). Bottom: the phase-folded light curve after processing with the reconstructive-mode **-TFA_SR** command (i.e., that output on line 15 of listing 15). Using the **-TFA_SR** command does a better job of filtering the noise while preserving the signal shape than does the **-TFA** command.

10 times longer for ASCII than for FITS. These scalings assume that the ASCII data consists of 3 columns each stored to 17-digit precision. Changing the precision, and/or the number of columns in the light curve will adjust the scalings. In terms of storage, the ASCII light curve with 1,000,000 data points takes 56 MB whereas the FITS-table with 3 double-precision columns takes 23 MB.

Note that based on applying similar tests to the file “EXAMPLES/2.fits”, which is used for the performance tests discussed below, we conclude that it takes VARTOOLS less than 0.3 ms to load the light curve.

4.2. Test 2: -LS

We now proceed to test the performance of individual VARTOOLS commands. For our first test we executed the **-LS** command on 1000 copies of “EXAMPLES/2.fits”. We choose the period range and

subsample factors such that the periodogram would be evaluated at 16,384 frequencies (due to the use of FFTs within the code the number of frequencies sampled internally increases in powers of 2, and thus the execution time shows discrete jumps as $\text{ceil}(\log_2(N_f))$ increments). Tests were carried out with commands similar to the following:

```
1 prompt> for i in $(seq 1 1000) ; do \
2   echo /run/shm/2.fits; \
3   done > /run/shm/lc_list
4
5 prompt> time \
6   vartools -l /run/shm/lc_list \
7   -LS 0.018980 30.0 0.1 1 0 \
8   > /dev/null
9
10 real    5m26.562s
11 user    5m20.624s
12 sys     0m5.700s
```

Repeated execution shows a variance of a few seconds in the time. The command takes on average 330 ms of user + system time per light curve on this system. When executed in parallel on 10 processors the routine takes 38 ms per light curve processed. Note that here, and in the examples below, we are calculating the periodogram for 10 different light curves in parallel, parallel processing of a single light curve is not currently supported. The total time scales close to linearly with the number of processors, however shared overhead and locks used to prevent conflicts between threads causes the scaling to be not exactly linear. In general the scaling is closer to linear with the number of processors when the execution time for a single process is longer. When 2048, 4096 or 8192 frequencies are scanned the process takes 20 ms, 48 ms, and 110 ms per light curve, respectively. Adjusting the number of points in the light curve, while keeping the number of frequencies fixed, has a negligible effect on the processing time which is dominated by the calculation of FFTs over the sampled frequencies. For very large light curves, however, there may be a difference as the time required for the “extrapolation” routine, which depends on the number of points in the light curve, becomes non-negligible.

4.3. Test 3: -BLS

We performed similar tests on **-BLS** to those conducted for **-LS**, using commands such as:

```
1 prompt> for i in $(seq 1 100) ; do \
2   echo /run/shm/2.fits; \
3   done > /run/shm/lc_list_100
4
5 prompt> time \
6   vartools -l /run/shm/lc_list_100 \
7   -BLS q 0.005 0.05 0.5 30. \
8   10000 200 0 1 0 0 0 \
9   > /dev/null
10
11 real    1m54.527s
12 user    1m54.459s
13 sys     0m5.700s
```

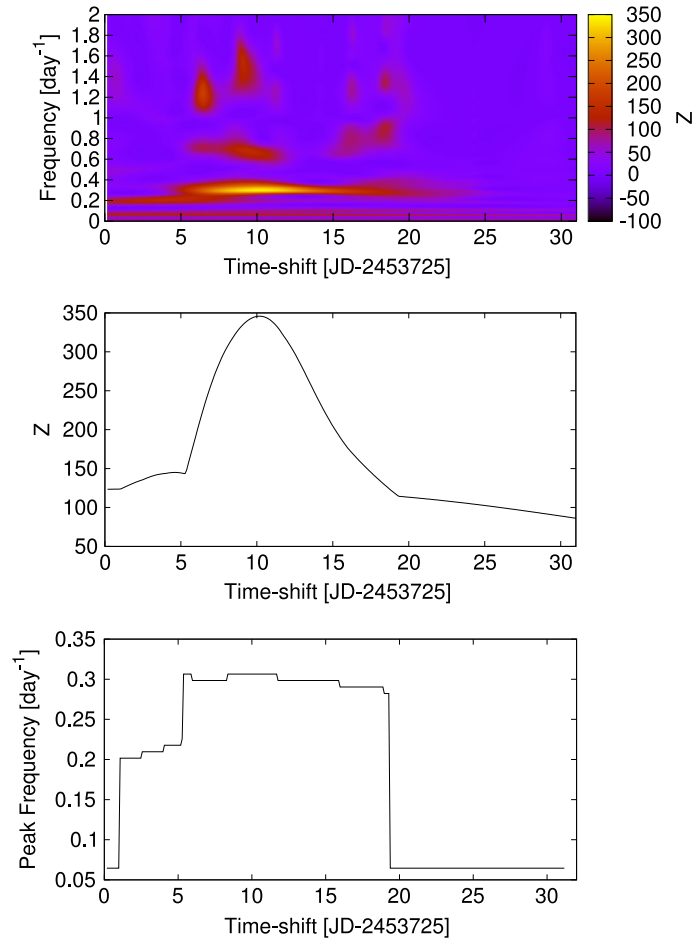



Fig. 5. Top: Weighted Wavelet Z-Transform calculated in example 3.7 (listing 16) for the light curve shown in Fig. 6. Middle: The maximum Z-transform as a function of time-shift. A strong signal is seen near a time of $JD - 2,453,725 = 10.174$. Bottom: the peak frequency in the transform as a function of time-shift. The signal near $JD - 2,453,725 = 10.174$ has a frequency of 0.3065 days^{-1} .

The relevant numbers controlling the execution time in this case are the number of frequencies sampled N_f (the above listing uses 10,000), the number of phase bins N_{bin} (200 above), the maximum number of phase bins for the transit duration $N_{\text{bin},\text{tr}}$ (for “q 0.005 0.05” and $N_{\text{bin}} = 200$ above we have $N_{\text{bin},\text{tr}} = 0.05 \times 200 = 10$), and the number of observations N_{obs} . We expect the execution time to scale roughly as

$$a \times N_f \times N_{\text{obs}} + b \times N_f \times N_{\text{bin}} \times N_{\text{bin},\text{tr}} \quad (93)$$

for some constants a and b . For the above example we find that the execution takes 1.1 s per light curve, or 120 ms of user + system time per light curve if we use 10 parallel processes (if the “nobinnedrms” keyword is used, then the execution is 920 ms per light curve, or 93 ms when running with 10 parallel processes; the difference in time is due to an extra square root being included in the constant b when the keyword is not used). Using 100,000 frequencies increases the execution time by a factor of ten as expected (11 s per light curve), while doubling N_{bin} increases the execution time by a factor of approximately two (or 1.5 for “nobinnedrms”).

4.4. Test 4: -aov_harm

We performed similar tests on **-aov_harm** to those conducted for the other commands. This was done using commands such as:

```
1 prompt> for i in $(seq 1 100) ; do \
2   echo /run/shm/2.fits; \
3   done > /run/shm/lc_list_100
4
5 prompt> time \
6   vartools -l /run/shm/lc_list_100 \
7     -aov_harm 1 0.018980 30.0 \
8     0.1 0.1 1 0 \
9     -parallel 10 \
10    > /dev/null
11
12 real    1m33.439s
13 user    15m41.771s
14 sys     0m0.068s
```

Here we are scanning the same number of frequencies as for **-LS** and using a pure sinusoid for the signal. The processing however is significantly slower than for **-LS**, with 16,384 frequencies taking 9.3 s per light curve when not running in parallel (or 930 ms per light curve when running with 10 parallel processes). Using half as many frequencies reduces the processing time by a factor of two. Using 2 or 3 harmonics (i.e., the fundamental and first higher-order harmonic in the first case, and the fundamental and first two higher-order harmonics in the second case) increases the execution time to 11 s per light curve and 13 s per light curve, respectively (when not running in parallel).

4.5. Test 5: -wwz

We used the following command to test the performance of **-wwz**:

```
prompt> for i in $(seq 1 100) ; do \
2   echo /run/shm/2.fits; \
   done > /run/shm/lc_list_100

prompt> time \
6   vartools -l /run/shm/lc_list_100 \
   -wwz maxfreq 2.0 \
8   freqsamp 0.25 \
   tau0 auto \
10  tau1 auto \
   dtau 0.1 \
12  -parallel 10 \
   > /dev/null

14  real    2m10.350s
16  user    21m54.090s
   sys     0m0.040s
```

Here we are scanning 249 frequencies each at 311 time-shifts. With 10 parallel processes this takes 1.3 s per light curve, or 13 s per light curve per processor. Doubling the number of frequencies sampled increases the execution time by a factor of two, as does doubling the number of time-shifts.

4.6. Test 6: -fastchi2

We used the following command to test the performance of **-fastchi2** which is included as an extension to VARTOOLS:

```
1  prompt> for i in $(seq 1 1000) ; do \
   echo /run/shm/2.fits; \
3   done > /run/shm/lc_list

5  prompt> time \
   vartools -L $LIBDIR/fastchi2.la \
7   -l /run/shm/lc_list \
   -fastchi2 \
9   Nharm fix 1 \
   freqmax fix 52.687 \
11  freqmin fix 0.03333 \
   oversample fix 4 \
13  > /dev/null

15  real    0m54.122s
   user    0m54.011s
17  sys     0m0.076s
```

where we explicitly load the library with the **-L** option to avoid spending time performing a disk search for the appropriate library, and the parameters for **-fastchi2** are chosen to calculate 16,384 frequencies using a simple sinusoid. We find that this process takes 54 ms per light curve, or 5.6 ms per light curve if run on 10 parallel processors. When 2048, 4096 or 8192 frequencies are scanned the process takes 34 ms, 36 ms and 42 ms, respectively. Thus for large number of frequencies the **-fastchi2** procedure is substantially faster than **-LS** (330 ms for 16,384 frequencies), but for 2048 or fewer frequencies the **-LS** command is faster to execute (20 ms per light curve).

5. Future development

There are several areas in which VARTOOLS may be improved to make it more useful for astronomical time series analysis. Several new processing commands are being actively developed, such as Fourier filtering methods, and techniques for stitching together

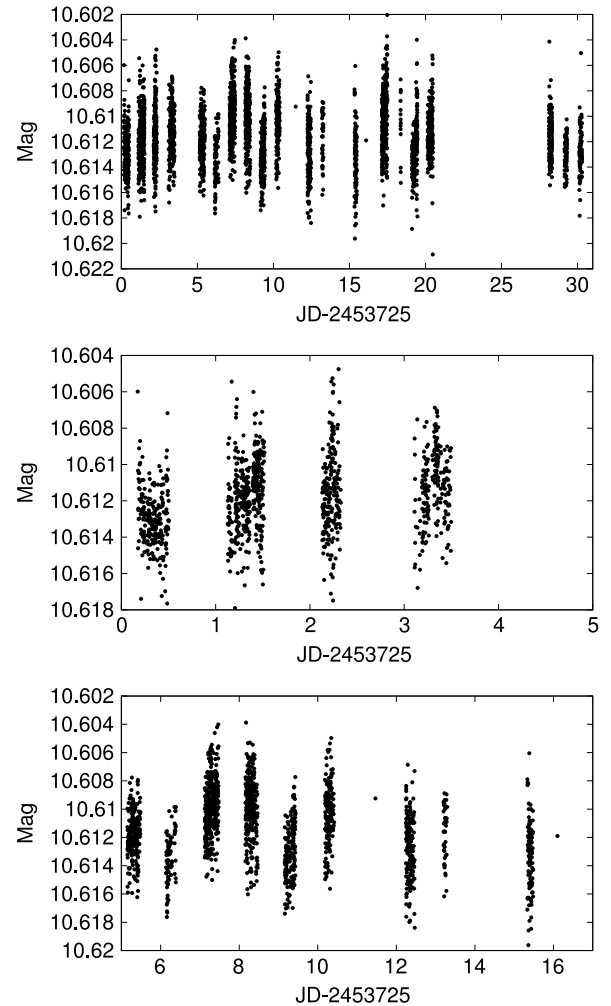


Fig. 6. Top: the light curve analyzed in example 3.7 (listing 17). Middle: the same light curve zoomed-in on early times when no strong signal is detected (though a signal with a frequency of $\sim 0.2 \text{ day}^{-1}$ may be present). Bottom: the same light curve zoomed-in on times where a significant signal with a frequency of 0.3065 day^{-1} is detected.

different light curves for a given source. Other topics for future development, which require more fundamental changes to the organization of the code, include the parallelization of individual commands (currently only parallel processing of multiple light curves is supported), and support for parallel operation in non-shared-memory mode (e.g., using MPICH). The methods for handling analytic expressions may be extended to support vector and matrix operations. We are also working on support for user-developed commands and functions written in PYTHON. As a separate effort we are also working on developing a Graphical User Interface (GUI) for VARTOOLS, written in PYTHON, which may be used to construct processing pipelines and interact with the results.

Acknowledgments

The development of VARTOOLS has been supported by NASA grant NNX14AE87G. We also acknowledge support from NSF/AST-1108686 and NASA grants NNX12AH91H and NNX13AJ15G. We would like to thank the anonymous referee for their review of this paper. We would also like to thank the users of VARTOOLS who have contributed bug reports and other feedback, especially T. Beatty, A. Bonanos, T. Bovaird, D. Flateau, L. Macri, D. Nataf, J. Pepper, J. Rasor, B. Sipőcz, R. Siverd, A. Schwarzenberg-Czerny, K. Stanek, and A. Sweeney.

Appendix A. Command syntax

See Tables A.1–A.45 and B.46–B.96.

Here we describe in detail the expected syntax for each command, including a brief explanation of the input parameters. Commands are listed in alphabetical order. See Section 2.2 for a description of the algorithms and uses of each command. In general a command is executed by typing the name of the command (including the “-” prefix, e.g., “-rms” for the command which calculates the r.m.s. scatter in a light curve) followed by various parameters and options. Terms within angular brackets “ $\langle \rangle$ ” are required parameters. Terms within square brackets “[]” are optional parameters that may be ignored. Within a given set of brackets the pipe “|” is used to distinguish between different choices that are allowed (in general only one of the possible choices may be used). Any term not placed within brackets is required. Terms within quotations indicate keywords that should be typed out exactly as they appear, but without the quotation marks (these are typically used to control optional behavior, or to specify the name of a parameter which then follows). The keywords are case sensitive. Unquoted terms represent parameters for which the user should substitute an appropriate value when calling the command. VARTOOLS obeys a strict ordering of parameters, you must provide them in the exact order indicated. Parameters referred to below as “flags” are used to control the behavior of the command. In these cases the user either substitutes “1” for the flag to turn the option on, or “0” to turn it off.

Many of the commands share syntax for controlling how parameter values should be determined. The syntax

```
1 <"fix" val | "list" ["column" col] | "fixcolumn" <colname | colnum>>
```

is adopted to allow the user to choose between fixing a parameter to a specific value for all light curves (the user types “fix” and then substitutes the appropriate value for *val*), to read the parameter value from the input list of light curve files allowing a different value for each light curve (the user types “list” then optionally indicates which column the parameter is to be read-in from by typing “column” and the column number; if the “column” keyword is not used, then the values are read-in from the next unused column in the file), or to use a value computed by a previously executed command (the user types “fixcolumn” and then gives either the name of the column, which is the column heading when the “-header” option is used, or the number of the column in the output table). In some cases only a subset of these options is allowed.

Below we list the expected syntax for each command, followed by a table explaining the various parameters.

```
1 -addnoise
2   < "white"
3     <"sig_white" <"fix" val | "list" ["column" col]>>
4   | "squareexp"
5     <"rho" <"fix" val | "list" ["column" col]>>
6     <"sig_red" <"fix" val | "list" ["column" col]>>
7     <"sig_white" <"fix" val | "list" ["column" col]>>
8     ["bintime" <"fix" val | "list" ["column" col]>]
9   | "exp"
10    <"rho" <"fix" val | "list" ["column" col]>>
11    <"sig_red" <"fix" val | "list" ["column" col]>>
12    <"sig_white" <"fix" val | "list" ["column" col]>>
13    ["bintime" <"fix" val | "list" ["column" col]>]
14  | "matern"
15    <"nu" <"fix" val | "list" ["column" col]>>
16    <"rho" <"fix" val | "list" ["column" col]>>
17    <"sig_red" <"fix" val | "list" ["column" col]>>
18    <"sig_white" <"fix" val | "list" ["column" col]>>
19    ["bintime" <"fix" val | "list" ["column" col]>]
20  | "wavelet"
21    <"gamma" <"fix" val | "list" ["column" col]>>
22    <"sig_red" <"fix" val | "list" ["column" col]>>
23    <"sig_white" <"fix" val | "list" ["column" col]>>
24  >
```

Table A.1
Input parameters for “-addnoise” command (Section 2.2.5.1).

“white”	—	Use the white-noise only noise model.
“sig_white”	—	Standard deviation of white noise.
“squareexp”	—	Use the square-exponential correlated noise model (Eq. (68)).
“rho”	—	The parameter ρ in Eq. (68).
“sig_red”	—	The standard deviation of the correlated component ($\sqrt{a}$ in Eq. (68)).
“sig_white”	—	The standard deviation of the white noise component (σ_i in Eq. (68)).
“bintime”	—	Optionally divide the input light curve into time bins and simulate the correlated noise independently in each bin. This option speeds up the simulation substantially in cases where the total time spanned by the light curve greatly exceeds the correlation timescale.
“exp”	—	Use the exponential correlated noise model (Eq. (69)).
“rho”	—	The parameter ρ in Eq. (69).
“sig_red”	—	The standard deviation of the correlated component ($\sqrt{a}$ in Eq. (69)).
“sig_white”	—	The standard deviation of the white noise component (σ_i in Eq. (69)).
“bintime”	—	Optionally divide the input light curve into time bins and simulate the correlated noise independently in each bin. This option speeds up the simulation substantially in cases where the total time spanned by the light curve greatly exceeds the correlation timescale.

(continued on next page)

Table A.1 (continued)

"matern"	—	Use the Matérn correlated noise model (Eq. (70)).
"nu"	—	The parameter ν in Eq. (70).
"rho"	—	The parameter ρ in Eq. (70).
"sig_red"	—	The standard deviation of the correlated component ($\sqrt{a}$ in Eq. (70)).
"sig_white"	—	The standard deviation of the white noise component (σ_i in Eq. (70)).
"bintime"	—	Optionally divide the input light curve into time bins and simulate the correlated noise independently in each bin. This option speeds up the simulation substantially in cases where the total time spanned by the light curve greatly exceeds the correlation timescale.
"wavelet"	—	Use a wavelet correlated noise model.
"gamma"	—	The parameter γ discussed in Section 2.2.5.
"sig_red"	—	The standard deviation of the correlated component.
"sig_white"	—	The standard deviation of the white noise component.

—alarm

No parameters or options are available.

```

1 —aov ["Nbin" Nbin] minp maxp subsample finetune Npeaks operiodogram
   [outdir ["whiten"] ["clip" clip clipiter] ["uselog"]
3   ["fixperiodSNR" <"aov" | "ls" | "injectharm" | "fix" period
   | "list" ["column" col]
5   | "fixcolumn" <colname | colnum>>]
```

Table A.2

Input parameters for "—aov" command (Section 2.2.1.4).

"Nbin"	—	The number of phase bins to use. The default is 8.
minp	—	The minimum period to search.
maxp	—	The maximum period to search.
subsample	—	The periodogram will be sampled at a resolution of subsample/ T where T is the time baseline of the light curve.
finetune	—	Peaks found in the initial scan will then be resampled at a resolution of finetune/ T .
Npeaks	—	The number of peaks in the periodogram to report.
operiodogram	—	Flag indicating if the periodogram for each light curve should be output to a separate file.
outdir	—	Required if operiodogram is set. The periodogram will be output to the file \$outdir/\$basename.aov where \$outdir is the value specified for outdir, and \$basename is the base filename of the light curve (any leading directory is stripped).
"whiten"	—	Keyword used to whiten the light curve and recalculate the periodogram for each peak. By default this is not done.
"clip"	—	Adjust the clipping performed on the periodogram in calculating the spectroscopic S/N of a peak. By default iterative 5σ clipping is performed.
clip	—	The sigma-clipping factor to use.
clipiter	—	Flag which if set causes iterative clipping to be performed.
"uselog"	—	Keyword used to calculate the S/N from the natural logarithm of the AoV spectrum. By default the S/N is calculated directly from AoV.
"fixperiodSNR"	—	Report also AoV, the false alarm probability and the S/N for a fixed period.
"aov"	—	Use the peak period from the last executed —aov command for the fixperiodSNR.
"ls"	—	Use the peak period from the last executed —LS command.
"injectharm"	—	Use the period from the last executed —Injectharm command (the first one is used if multiple periods in a given command).

```

1 —aov_harm Nharm minp maxp subsample finetune Npeaks operiodogram [outdir]
   ["whiten"] ["clip" clip clipiter]
3   ["fixperiodSNR" <"aov" | "ls" | "injectharm" | "fix" period
   | "list" ["column" col]
5   | "fixcolumn" <colname | colnum>>]
```

Table A.3

Input parameters for "—aov_harm" command (Section 2.2.1.5).

Nharm	—	The number of harmonics to use. Set this to a value less than 1 to automatically optimize this number. The fundamental mode is counted as 1 harmonic here.
minp	—	The minimum period to search.
maxp	—	The maximum period to search.
subsample	—	The periodogram will be sampled at a resolution of subsample/ T where T is the time baseline of the light curve.
finetune	—	Peaks found in the initial scan will then be resampled at a resolution of finetune/ T .
Npeaks	—	The number of peaks in the periodogram to report.

(continued on next page)

Table A.3 (continued)

operiodogram	—	Flag indicating if the periodogram for each light curve should be output to a separate file.
outdir	—	Required if operiodogram is set. The periodogram will be output to the file \$outdir/\$basename.aov_harm where \$outdir is the value specified for outdir, and \$basename is the base filename of the light curve (any leading directory is stripped).
"whiten"	—	Keyword used to whiten the light curve and recalculate the periodogram for each peak. By default this is not done.
"clip"	—	Adjust the clipping performed on the periodogram in calculating the spectroscopic S/N of a peak. By default iterative 5σ clipping is performed.
clip	—	The sigma-clipping factor to use.
clipiter	—	Flag which if set causes iterative clipping to be performed.
"fixperiodSNR"	—	Report also AoV, the false alarm probability and the S/N for a fixed period.
"aov"	—	Use the peak period from the last executed -aov command for the fixperiodSNR.
"ls"	—	Use the peak period from the last executed -LS command.
"injectharm"	—	Use the period from the last executed -Injectharm command (the first one is used if multiple periods in a given command).

```
1 -autocorrelation start stop step outdir
```

Table A.4

Input parameters for "-autocorrelation" command (Section 2.2.2.2).

start	—	Starting time-lag for sampling the discrete autocorrelation function.
stop	—	Stopping time-lag for sampling the discrete autocorrelation function.
step	—	Time-lag step-size.
outdir	—	Directory for outputting the autocorrelation files. The files will be written to \$outdir/\$basename.autocorr where \$outdir is the value specified for outdir, and \$basename is the base filename of the light curve (any leading directory is stripped).

```
1 -binlc <"average" | "median" | "weightedaverage">
  <"binsize" binsize | "nbins" nbins>
3  ["bincolumns" var1[:stats1][,var2[:stats2],...]]
  ["firstbinshift" firstbinshift]
5  <"tcenter" | "taverage" | "tmedian" | "tnoshrink" ["bincolumnsonly"]>
```

Table A.5

Input parameters for "-binlc" command (Section 2.2.6.1).

"average"	—	Take the average of points in a bin.
"median"	—	Take the median of points in a bin.
"weightedaverage"	—	Take the weighted average of points in a bin.
"binsize"	—	Specify the size of the bin in the time unit of the light curve. Type the "binsize" keyword and then provide the value.
"nbins"	—	Specify the number of bins to use. In this case the binsize is equal to $T/nbins$ where T is the time-span of the light curve. Type the "nbins" keyword and then provide the value.
"bincolumns"	—	By default all light curve vectors except for the time, and uncertainty, will be binned using the "average", "median", or "weightedaverage" as specified by the user. Any vectors that the user wishes to bin in a different manner may be specified with this option. Following the "bincolumns" keyword provide a comma separated list of light curve variables and associated statistics. After a given variable use a ":" and then indicate the statistic to calculate. The options for the statistics are the same as for the -stats command.
"firstbinshift"	—	By default the first bin begins at the initial time in the light curve (t_0). Use this keyword to change this behavior, in which case the first bin will start at $t_0 - \text{firstbinshift}/\text{binsize}$.
"tcenter"	—	Take the binned light curve time to be the time at the center of each bin.
"taverage"	—	Take the binned light curve time to be the average of the times of points that fall in a given bin.
"tmedian"	—	Take the binned light curve time to be the median of the times of points that fall in a given bin.
"tnoshrink"	—	Do not shrink the light curve. In this case all points in the light curve will be replaced by their binned value, but the times will not be changed.
"bincolumnsonly"	—	If this keyword is given, then only the columns specified after the "bincolumns" keyword will be binned. All other light curve vectors will be left unchanged. This option is only available when the "tnoshrink" option is used.

```
1 -BLS <"r" rmin rmax | "q" qmin qmax > minper maxper nfreq nbins
  timezone Npeak outperiodogram [outdir] omodel [model_outdir]
3  correctlc ["fittrap"] ["nobinnedrms"]
  ["ophcurve" outdir phmin phmax phstep]
5  ["ojdcurve" outdir jdstep]
  ["stepP" | "steplogP"]
7  ["adjust-qmin-by-mindt" ["reduce-nbins"]]
```

Table A.6

Input parameters for “-BLS” command (Section 2.2.1.6).

“r”	— Allow for a period dependent limit on the transit durations to search. r corresponds approximately to the stellar radius for lower main sequence stars assuming central transits, circular orbits, and that times are specified in days.
rmin	— The minimum transit duration at period P will be $0.076 \text{ rmin}^{2/3} P^{1/3}$.
rmax	— The maximum transit duration at period P will be $0.076 \text{ rmax}^{2/3} P^{1/3}$.
“q”	— Use a period-independent limit on the fractional transit durations considered.
qmin	— The minimum transit duration at period P will be $qminP$.
qmax	— The maximum transit duration at period P will be $qmaxP$.
minper	— The minimum period to search for transits.
maxper	— The maximum period to search for transits.
nfreq	— The number of frequencies to search (the frequency step-size is $\Delta f = (1/\text{minper} - 1/\text{maxper})/\text{nfreq}$).
nbins	— The number of phase bins to use.
timezone	— The timezone of the observatory (in hours from UTC). This only affects the grouping of points used to determine the fraction of $\Delta\chi^2$ from a single day. If multiple observatories were used, or if the observations were performed from space one can simply provide 0 for this value and ignore the reported $\Delta\chi^2$ from one day.
Npeak	— The number of peaks to identify in the BLS spectrum.
outperiodogram	— A flag used to indicate whether or not the BLS spectra should be output.
outdir	— Required if outperiodogram is set to 1. The BLS spectra will be written to files with the name \$outdir/\$basename.bls \$outdir is the value specified for outdir, and \$basename is the base filename of the light curve (any leading directory is stripped).
omodel	— A flag used to indicate whether or not the best-fit transit model light curves should be output. These light curves are evaluated at the observed times.
model_outdir	— Required if omodel is set to 1. The models files will be written to \$model_outdir/\$basename.bls.model.
correctlc	— A flag used to indicate whether or not the best-fit transit model should be subtracted from the observations before passing the light curve on to the next command.
“fittrap”	— An optional keyword which, if used, causes a trapezoid-shape transit model to be fit to the light curve around each BLS peak identified.
“nobinnedrms”	— An optional keyword used to adjust the way in which the BLS spectroscopic S/N statistic is calculated. If this keyword is used then the average and standard deviation of the spectrum are calculated only from the maximum SR values at each trial frequency. If it is not used, then these are calculated from all SR values considered (including transit epochs and durations that do not optimize the fit at a given frequency). Using this keyword speeds up the BLS calculation, but the S/N will tend to be suppressed for high significance detections.
“ophcurve”	— A keyword used to output best-fit model light curves that are uniformly sampled in phase.
outdir	— The models will be written out to files named \$outdir/\$basename.bls.phcurve.
phmin	— The starting phase to use in the model.
phmax	— The ending phase to use in the model.
phstep	— The phase step-size to use in the model.
“ojdcurve”	— A keyword used to output best-fit model light curves that are uniformly sampled in time between the first and last observed times in the light curve.
outdir	— The models will be written out to files named \$outdir/\$basename.bls.jdcurve.
jdstep	— The time step-size to use in the model.
“stepP”	— Sample the BLS spectrum at uniform steps in period (by default uniform steps in frequency are used).
“steplogP”	— Sample the BLS spectrum at uniform steps in the logarithm of the period (by default uniform steps in frequency are used).
“adjust-qmin-by-mindt”	— Adaptively set the minimum q value to the maximum of $qmin$ or $\Delta t_{\min} \times f$ where $\Delta t_{\min}$ is the minimum time difference between consecutive points in the light curve.
“reduce-bins”	— Adaptively reduce the number of phase bins at each frequency such that there are no more than two bins to sample a transit of phase duration $qmin$.

```

1 -BLSFixPer <"aov" | "ls" | "list" ["column" col]
   | "fix" period | "fixcolumn" <colname | colnum>
3   | "expr" expr>
   <"r" rmin rmax | "q" qmin qmax >
5 nbins timezone omodel [model_outdir] correctlc ["fittrap"]

```

Table A.7

Input parameters for “-BLSFixPer” command (Section 2.2.1.8).

“aov” “ls” “list” “fix” “fixcolumn” “expr”	— Source for the period at which to calculate the BLS model (either from the last -aov command, the last -LS command, from the input light curve list file, fixed to a value given on the command line, set to the output from a previously executed command, or determined by evaluating an analytic expression).
“r”	— Allow for a period dependent limit on the transit durations to search. r corresponds approximately to the stellar radius for lower main sequence stars assuming central transits, circular orbits, and that times are specified in days.
rmin	— The minimum transit duration at period P will be $0.076 \text{ rmin}^{2/3} P^{1/3}$.
rmax	— The maximum transit duration at period P will be $0.076 \text{ rmax}^{2/3} P^{1/3}$.

(continued on next page)

Table A.7 (continued)

“q”	–	Use a period-independent limit on the fractional transit durations considered.
qmin	–	The minimum transit duration at period P will be qmin P .
qmax	–	The maximum transit duration at period P will be qmax P .
nbins	–	The number of phase bins to use.
timezone	–	The timezone of the observatory (in hours from UTC). This only affects the grouping of points used to determine the fraction of $\Delta\chi^2$ from a single day. If multiple observatories were used, or if the observations were performed from space one can simply provide 0 for this value and ignore the reported $\Delta\chi^2$ from one day.
omodel	–	A flag used to indicate whether or not the best-fit transit model light curves should be output. These light curves are evaluated at the observed times.
model_outdir	–	Required if omodel is set to 1. The models files will be written to \$model_outdir/\$basename.blsfixper.model.
correctlc	–	A flag used to indicate whether or not the best-fit transit model should be subtracted from the observations before passing the light curve on to the next command.
“fittrap”	–	An optional keyword which, if used, causes a trapezoid-shape transit model to be fit to the light curve around each BLS peak identified.

```

1  –BLSFixDurTc
   <"duration" <"fix" dur | "fixcolumn" <colname | colnum>
3   | "list" ["column" col]>>
   <"Tc" <"fix" Tc | "fixcolumn" <colname | colnum>
5   | "list" ["column" col]>>
   ["fixdepth" <"fix" depth | "fixcolumn" <colname | colnum>
7   | "list" ["column" col]>
   ["qgress" <"fix" qgress | "fixcolumn" <colname | colnum>
9   | "list" ["column" col]>]]
   minper maxper nfreq timezone
11  Npeak outperiodogram [outdir] omodel [model_outdir]
   correctlc ["fittrap"]
13  ["ophcurve" outdir phmin phmax phstep]
   ["ojdcurve" outdir jdstep]

```

Table A.8

Input parameters for “-BLSFixDurTc” command (Section 2.2.1.7).

“duration”	–	Indicate how the fixed transit duration (in time-units of the light curve) should be determined.
“Tc”	–	Indicate how the fixed transit epoch (in time-units of the light curve) should be determined.
“fixdepth”	–	Optional keyword used to indicate that the transit depth should be fixed (by default it is allowed to vary). If this keyword is used, then one should follow it with an indication of how the fixed depth should be determined.
“qgress”	–	Optional keyword used to indicate that the fractional duration of transit ingress (i.e., the ingress duration divided by the total transit duration) should be fixed (by default it is allowed to vary). This keyword may only be used if “fixdepth” is also used. One should follow the keyword with an indication of how the fixed fractional ingress duration should be determined.
minper	–	The minimum period to search for transits.
maxper	–	The maximum period to search for transits.
nfreq	–	The number of frequencies to search (the frequency step-size is $\Delta f = (1/\text{minper} - 1/\text{maxper})/\text{nfreq}$).
timezone	–	The timezone of the observatory (in hours from UTC). This only affects the grouping of points used to determine the fraction of $\Delta\chi^2$ from a single day. If multiple observatories were used, or if the observations were performed from space one can simply provide 0 for this value and ignore the reported $\Delta\chi^2$ from one day.
Npeak	–	The number of peaks to identify in the BLS spectrum.
outperiodogram	–	A flag used to indicate whether or not the BLS spectra should be output.
outdir	–	Required if outperiodogram is set to 1. The BLS spectra will be written to files with the name \$outdir/\$basename.blsfixdurtc \$outdir is the value specified for outdir, and \$basename is the base filename of the light curve (any leading directory is stripped).
omodel	–	A flag used to indicate whether or not the best-fit transit model light curves should be output. These light curves are evaluated at the observed times.
model_outdir	–	Required if omodel is set to 1. The models files will be written to \$model_outdir/\$basename.blsfixdurtc.model.
correctlc	–	A flag used to indicate whether or not the best-fit transit model should be subtracted from the observations before passing the light curve on to the next command.
“fittrap”	–	An optional keyword which, if used, causes a trapezoid-shape transit model to be fit to the light curve around each BLS peak identified.
“ophcurve”	–	A keyword used to output best-fit model light curves that are uniformly sampled in phase.
outdir	–	The models will be written out to files named \$outdir/\$basename.blsfixdurtc.phcurve.
phmin	–	The starting phase to use in the model.
phmax	–	The ending phase to use in the model.
phstep	–	The phase step-size to use in the model.
“ojdcurve”	–	A keyword used to output best-fit model light curves that are uniformly sampled in time between the first and last observed times in the light curve.
outdir	–	The models will be written out to files named \$outdir/\$basename.blsfixdurtc.jdcurve.
jdstep	–	The time step-size to use in the model.

—changeerror

No parameters or options are available.

1 **—changevariable** <"t" | "mag" | "err" | "id"> var

Table A.9

Input parameters for “—changevariable” command (Section 2.2.7.1).

“t”	—	Change the variable used for the light curve times in subsequent commands.
“mag”	—	Change the variable used for the light curve magnitude values in subsequent commands.
“err”	—	Change the variable used for the light curve uncertainties in subsequent commands.
“id”	—	Change the variable used for the light curve image-ids in subsequent commands.
var	—	The name of the new variable to use for the indicated quantity.

1 **—chi2**

No parameters or options are available.

1 **—chi2bin** Nbin bintime1...bintimeN

Table A.10

Input parameters for “—chi2bin” command (Section 2.2.2.4).

Nbin	—	The number of moving mean filters to use.
bintime1...bintimeN	—	A space-delimited list of filter widths, one for each of the Nbin filters used. The width of each filter is given by $2.0 * \text{bintime}$, where bintime is in minutes, assuming the times in the light curve are in days.

1 **—clip** sigclip iter ["niter" n] ["median"]

Table A.11

Input parameters for “—clip” command (Section 2.2.4.1).

sigclip	—	The σ -clipping factor to use. If a value ≤ 0 is specified, then σ -clipping is not performed, but points with errors ≤ 0 or NaN magnitude values will be clipped from the light curve.
iter	—	A flag that is 1 for iterative clipping (performed continuously until no further points are removed), or 0 to not do continuous iterative clipping.
“niter”	—	An optional keyword used to specify a fixed number of iterations to use in performing the clipping.
“median”	—	By default clipping is done with respect to the mean. Use this keyword to cause the clipping to be done instead with respect to the median.

1 **—converttime**

```

3  <"input" <"mjd" | "jd" | "hjd" | "bjd" >>
4  ["inputsubtract" value] ["inputsys—tdb" | "inputsys—utc"]
5  <"output" <"mjd" | "jd" | "hjd" | "bjd" >>
6  ["outputsubtract" value] ["outputsys—tdb" | "outputsys—utc"]
7  ["radec" <"list" ["column" col] | "fix" raval decval>
8  ["epoch" epoch]]
9  ["ppm" <"list" ["column" col] | "fix" mu_ra mu_dec>]
10 ["input—radec" <"list" ["column" col] | "fix" raval decval>
11 ["epoch" epoch]]
12 ["input—ppm" <"list" ["column" col] | "fix" mu_ra mu_dec>]
13 ["ephemfile" file] ["leapsecf" file] ["planetdatafile" file]
14 ["observatory" <code | "show—codes">
15   | "coords"
16   <"fix" latitude[deg] longitude[deg_E] altitude[m]
17   | "list" ["column" collat collong colalt]
18   | "fromlc" collat collong colalt>]
```

Table A.12

Input parameters for “—converttime” command (Section 2.2.6.3).

“input”	—	Indicate the input time system used. The required “input” keyword must be followed by one of the following keywords.
“mjd”	—	Modified Julian Date (MJD = JD — 2400000.5).
“jd”	—	Julian Date.
“hjd”	—	Helio-centric Julian Date.
“bjd”	—	Bary-centric Julian Date.

(continued on next page)

Table A.12 (continued)

“inputsubtract”	– Optional keyword used to indicate a constant that has been subtracted from the input times (i.e., if the input time is HJD – 2,400,000, one would give “input hjd inputsubtract 2,400,000” on the command-line).
“inputsys-tdb”	– Optional keyword used to indicate that the input times have been corrected for leap-seconds.
“inputsys-utc”	– Optional keyword used to indicate that the input times have not been corrected for leap-seconds (this is assumed by default).
“output”	– Indicate the output time system used. The required “output” keyword must be followed by a keyword indicating the time system, options are the same as for “input”.
“outputsubtract”	– Optional keyword used to indicate a constant that is to be subtracted from the output times.
“outputsys-tdb”	– Optional keyword used to indicate that the output times should be on a system corrected for leap-seconds.
“outputsys-utc”	– Optional keyword used to indicate that the output times should be on a system that is not corrected for leap-seconds (by default the same system as the input times is assumed).
“radec”	– Indicate the source for the RA and Dec coordinates of the source. This is required if converting to/from HJD or BJD. Both coordinates should be in decimal degrees.
“epoch”	– Optionally specify a time epoch in years on the Common Era system for which the RA and Dec coordinates are provided. The default is 2000.0.
“ppm”	– Optionally indicate the source for the RA and Dec proper motions of the source. These should be given in units of milliarcseconds per year. By default sources are assumed to have no proper motion.
“input-radec”	– Optionally specify a different source for the RA and Dec coordinates that were assumed in calculating HJD or BJD for the input times. This may be useful, for example, if the times were converted to HJD or BJD assuming the coordinates for the center of an image, but one wishes now to correct them to the coordinates of the source itself.
“epoch”	– Optionally specify a time epoch in years on the Common Era system for which the input RA and Dec coordinates are provided. The default is 2000.0.
“input-ppm”	– Optionally indicate the source for the input RA and Dec proper motions assumed for the source. These should be given in units of milliarcseconds per year. By the initial times are assumed to have been calculated with no proper motion, or with the proper motion specified using the “ppm” keyword.
“ephemfile”	– Provide the filename for the JPL NAIF ephemeris file used to determine the position of the Earth with respect to the Solar System Barycenter. If not specified VARTOOLS will check if the environment variable CSPICE_EPHEM_FILE has been set, and use it if it has been set.
“leapsecfile”	– Provide the filename for the JPL NAIF leap-second file used to determine the number of leap-seconds since a reference epoch. This can also be set with the environment variable CSPICE_LEAPSEC_FILE.
“planetdatafile”	– Provide the filename for the JPL NAIF planetary physical data file used to determine the location of the observer with respect to the center of the Earth in the J2000.0 inertial frame. This can also be set with the environment variable CSPICE_PLANETDATA_FILE.
“observatory”	– Optionally provide this keyword to specify an observatory used to make the observations. If this is set then one should either give:
code	– A short string used to indicate the observatory, e.g., “maunakea” for Mauna Kea Observatory, Hawaii, “lasilla” for La Silla Observatory, etc.
“show-codes”	– If this keyword is given, then the list of allowed observatory codes will be output, and VARTOOLS will not perform any processing.
“coords”	– Optionally provide the latitude (in degrees), longitude (in degrees east of the prime meridian), and altitude (in meters) of the observatory at which the measurements were performed. One must then indicate the source for these quantities. In addition to the standard “fix” and “list” keywords, one may also use the “fromlc” keyword to read these quantities as columns in the light curve. This may be used if observations carried out from different locations were combined into a single light curve file.

Table A.13

Input parameters for “-copylc” command (Section 2.2.5.2).

Ncopies — The number of light curve copies to make.

1 **-copylc** Ncopies

```
1 -decorr correctlc zeropointterm subtractfirstterm Nglobalterms globalfile1
  order1 ... Nlcterms lccolumn1 lccorder1 ... omodel
3 [model_outdir]
```

Table A.14

Input parameters for “-decorr” command (Section 2.2.3.1).

correctlc	– A flag indicating whether or not the light curves passed on to the next command will be decorrelated. If it is set to 0 then the value of χ^2 from the decorrelation together with the decorrelation coefficients will be output to the table, but the light curves themselves will not be affected by the command.
zeropointterm	– A flag indicating whether or not a zero-point offset should be included in the fit.
subtractfirstterm	– A flag indicating whether or not the first terms in the parameter sequences used in the decorrelation should be subtracted. This may be useful, for example, if one is decorrelating against the JD, in which case one can set subtractfirstterm to 1 to use $JD - JDO$ to prevent accumulating large round-off errors.

(continued on next page)

Table A.14 (continued)

Nglobalterms	—	The number of global parameter sequences to read in from separate files. The two terms below must be repeated for each of the Nglobalterms files.
globalfile1	—	The filename for the first global parameter sequences to decorrelate against. The file should have the format: JD signal_value, or ID signal_value if the -matchstringid option to vartools has been set.
order1	—	The order of the polynomial to use in decorrelating against this sequence.
Nlterms	—	The number of sequences read from each light curve to use in the decorrelation. The two terms below must be repeated Nlterms times.
lccolumn1	—	The column in the light curve to use for the first parameter sequence.
lorder1	—	The order of the polynomial to use in decorrelating against this sequence.
omodel	—	A flag indicating whether or not the model decorrelation light curves should be output.
model_outdir	—	This parameter is required if omodel is set to 1. The model light curves will be written to files named \$model_outdir/\$basename.decor.model where \$model_outdir is the value supplied for model_outdir, and \$basename is the base filename of the light curve (i.e., the name stripped of any directories).

```

1 -dftclean nbeam ["maxfreq maxf"] ["outspec" dspec_outdir]
  ["finddirtytypepeaks" Npeaks ["clip" clip clipiter]]
3  ["outwfunc" wfunc_outdir]
  ["clean" gain SNlimit ["outbeam" cbeam_outdir]
5  ["outspec" cspec_outdir]
  ["findcleanpeaks" Npeaks ["clip" clip clipiter]]]
7  ["useampspec"] ["verboseout"]

```

Table A.15

Input parameters for “-dftclean” command (Section 2.2.1.3).

nbeam	—	The number of points per 1/T frequency element to include in the calculated power spectrum, where T is the time base-line of the input light curve.
“maxfreq”	—	An optional keyword which should be followed by the maximum frequency (in cycles per day) at which to calculate the power spectrum. If not specified, the power spectrum will be calculated up to the Nyquist frequency.
“outspec”	—	An optional keyword used to output the raw Discrete Fourier Transform (i.e., before applying the CLEAN algorithm) to a file.
dspec_outdir	—	Required if the “outspec” keyword is set. The raw DFT will be output to a file named \$dspec_outdir/\$basename.dftclean.dspec where \$basename is the base filename of the input light curve (stripped of any leading directory names).
“finddirtytypepeaks”	—	An optional keyword used to search the raw DFT for peaks.
Npeaks	—	The number of peaks to report from the raw DFT power spectrum. This parameter is required if the “finddirtytypepeaks” keyword is given.
“clip”	—	Optional keyword to change the clipping method used in calculating the average and standard deviation of the raw power spectrum for determining the spectroscopic S/N of each peak. If this keyword is given then follow it with the σ -clipping factor and a flag to indicate whether or not continuous iterative clipping should be performed. By default iterative 5σ clipping is performed.
“outwfunc”	—	An optional keyword used to output the window function to the file \$wfunc_outdir/\$basename.dftclean.wfunc.
“clean”	—	If this keyword is specified then the CLEAN deconvolution algorithm will be executed on the power spectrum.
gain	—	The gain factor used in cleaning the power spectrum. It should have a value between 0.1 and 1.
SNlimit	—	Cleaning will proceed until the last peak has a value that is less than SNlimit times the standard deviation.
outbeam	—	Optional keyword to output the “clean beam” used in producing the final power spectrum. This will be output to the file named \$cbeam_outdir/\$basename.dftclean.cbeam.
findcleanpeaks	—	An optional keyword used to search the cleaned DFT for peaks. It takes the same additional parameters as “finddirtytypepeaks”.
“useampspec”	—	Give this keyword to use the amplitude spectrum for calculating the S/N of peaks rather than the power spectrum, which is used by default.
“verboseout”	—	Give this keyword to output the average and standard deviation of the spectrum before and after clipping, in addition to the final S/N value.

```

1 -difffluxtomag mag_constant offset ["magcolumn" col]

```

Table A.16

Input parameters for “-difffluxtomag” command (Section 2.2.6.4).

mag_constant	—	The magnitude of a source with a flux of 1 ADU.
offset	—	An additive constant to apply to the output light curves.
“magcolumn”	—	An optional keyword used to specify the column from the input light curve list with the reference magnitude of the source. If not given, then the next available column in the light curve list file will be used.

1 **-ensemblescalesig** sigclip

Table A.17

Input parameters for “-ensemblescalesig” command (Section 2.2.6.5).

sigclip	—	σ -clipping factor used in clipping outliers from the $r.m.s.^2_{\text{expected}}$ vs. $(\chi^2/d.o.f.)r.m.s.^2_{\text{expected}}$ distribution in determining the light curve uncertainty rescaling factor.
---------	---	---

1 **-expr** var="expression

Table A.18

Input parameters for “-expr” command (Section 2.2.6.6).

var="expression	—	Set the variable on the left hand side of the expression equal to the result of evaluating the analytic expression on the right hand side.
-----------------	---	--

```
1 -findblends matchrad ["radec"]
  ["xycol" xcol ycol]
3 <"fix" period | "list" ["column" col]
  | "fixcolumn" <colname | colnum>>
5 ["starlist" starlistfile] ["zeromag" zeromagval] ["nofluxconvert"]
  ["Nharm" Nharm] ["omatches" outputmatchfile]
```

Table A.19

Input parameters for “-findblends” command (Section 2.2.8.1).

matchrad	—	Matching radius used in determining if a given light curve might be blended with a source from the star list file.
“radec”	—	If this keyword is specified then matchrad is assumed to be in arcseconds and the X and Y positions of the sources are RA and Dec. in decimal degrees. If this keyword is not given, then rectangular matching will be performed on the X and Y coordinates, and matchrad should be in the same units.
“xycol”	—	An optional keyword for specifying the columns from the input light curve list file to use for determining the X and Y coordinates of the each of the light curves. If not given, then the next unused columns in the list file will be assumed.
“fix” “list” “fixcolumn”	—	Source for the period to use for the light curve.
“starlist”	—	By default the input light curve list is matched to itself. Use this keyword to match instead to the list in the file starlistfile. The file should have 3 white-space delimited columns, the first being a string identifier, the second and third being the X and Y coordinates of the source.
“zeromag”	—	Optional keyword for specifying the zero-point magnitude for converting from magnitudes into fluxes. The default value is 25.0.
“nofluxconvert”	—	If this optional keyword is given then the magnitude to flux conversion is not performed.
“Nharm”	—	Optional keyword to change the number of harmonics to include in the Fourier series fit to each light curve to determine its amplitude of variability. If it is 0 then only a sinusoid will be fit. The default value is 2.
“omatches”	—	Optional keyword to output the names and flux amplitudes of all stars matching to each potential variable. outputmatchfile is the name of the file to output this information to.

-fluxtomag mag_constant offset

Table A.20

Input parameters for “-fluxtomag” command (Section 2.2.6.7).

mag_constant	—	The magnitude of a source with a flux of 1 ADU.
offset	—	A constant to add to the output light curve magnitudes.

```
1 -GetLSAmpThresh <"ls" | "list" ["column" col]> minp thresh
  <"harm" Nharm Nsubharm | "file" listfile> ["noGLS"]
```

Table A.21

Input parameters for “-GetLSAmpThresh” command (Section 2.2.1.2).

“ls” “list”	—	The source for the period for which to calculate the minimally detectable amplitude. If the keyword “ls” is given then the source is the highest peak found by the most recent -LS command. If it is “list” then the period is read-in from the input light curve list.
minp	—	The minimum period considered by the L-S run. This is needed for determining the false alarm probability, which depends on the period range scanned.

(continued on next page)

Table A.21 (continued)

thresh	—	The maximum value of $\log_{10}$ FAP that the light curve have had and still have been considered a detection.
“harm”	—	Use this keyword to fit a harmonic series to the light curve to determine its amplitude.
Nharm	—	The number of harmonics to include in the series.
Nsubharm	—	The number of sub-harmonics to include in the series.
“file”	—	Use this keyword to read-in the model signal from a file (in general a separate file is used for each light curve).
listfile	—	The name of a file listing the files containing the model signals for each of the light curves. This list file has two columns of the form: signal_file signal_amp, with one line for each light curve being processed. Each signal_file should contain the signal magnitude in the third column. signal_amp is the amplitude in magnitudes of the signal (to allow for cases where the signal amplitude is greater than the difference between the minimum and maximum values in the file).
“noGLS”	—	If this optional keyword is given then the traditional L–S periodogram is used for determining the false alarm probability of a signal. By default the Generalized L–S is used.

```

-if <expression> [–command1 ... –commandN]
2  [ –elif <expression> [–command1 ... –commandN]]
...
4  [ –elif <expression> [–command1 ... –commandN]]
   [ –else [–command1 ... –commandN]]
6  -fi

```

Table A.22

Input parameters for “-if” command (Section 2.2.7.2).

expression	—	An analytic expression evaluated for a given light curve. If this evaluates to a number different from 0 the expression will be treated as “true”.
–command1 ... –commandN	—	A set of commands that will be executed conditional upon testing the expression.

```

-Injectharm <"list" ["column" col] | "fix" per
2  | "rand" minp maxp
  | "logrand" minp maxp | "randfreq" minf maxf
4  | "lograndfreq" minf maxf>
Nharm (<"amplist" ["column" col]
6  | "ampfix" amp | "amprand" minamp maxamp
  | "amplogrand" minamp maxamp> ["amprel"]
8  <"phaselist" ["column" col]
  | "phasefix" phase | "phaserand"> ["phaserel"]0...Nharm Nsubharm
10 (<"amplist" ["column" col] | "ampfix" amp
  | "amprand" minamp maxamp
12 | "amplogrand" minamp maxamp> ["amprel"]
  <"phaselist" ["column" col]
14 | "phasefix" phase | "phaserand"> ["phaserel"]1...Nsubharm
omodel [modeloutdir]

```

Table A.23

Input parameters for “-Injectharm” command (Section 2.2.5.3).

“list”	—	Take the period of the harmonic signal to add to the light curve from the input light curve list file, optionally specifying the column number.
“fix”	—	Fix the period to the specified value.
“rand”	—	Choose a random number for the period between minp and maxp.
“logrand”	—	Choose a random number for the period from a distribution that is uniform in logarithm between minp and maxp.
“randfreq”	—	Choose a random number for the period from a distribution that is uniform in frequency between minf and maxf (in cycles per day).
“lograndfreq”	—	Choose a random number for the period from a distribution that is uniform in the logarithm of the frequency between minf and maxf (in cycles per day).
Nharm	—	The number of harmonics to include in the harmonic series. If this is 0 then only the fundamental mode is included (i.e., a simple sine curve). For each of the Nharm + 1 modes one must specify the source for the amplitude and phase. This done with the following keywords:
“amplist”	—	Take the amplitude from the input light curve list file. Optionally specify the column to use with the “column” keyword.
“ampfix”	—	Fix the amplitude to the specified value.
“amprand”	—	Use a random number drawn from a uniform distribution between minamp and maxamp.
“amplogrand”	—	Use a random number drawn from a uniform-log distribution between minamp and maxamp.
“amprel”	—	If specified, amplitudes for harmonics 1 through Nharm are given relative to the amplitude of the fundamental mode.
“phaselist”	—	Take the phase from the input light curve list file. Optionally specify the column to use with the “column” keyword.

(continued on next page)

Table A.23 (continued)

"phasefix"	—	Fix the phase to the specified value.
"phaserand"	—	Draw a random number between 0 and 1.
"phaserel"	—	If specified, phases for harmonics 1 through Nharm are given relative to the fundamental mode.
Nsubharm	—	The number of subharmonics to use. For each of the Nsubharm subharmonics one must specify the source for the amplitude and phase as above, the "amprel" and "phaserel" keywords may also be provided to make the amplitude and/or phase relative to the values for the fundamental mode.
omodel	—	Flag indicating whether or not to output the model harmonic series evaluated at the times of observation in the light curve.
modeloutdir	—	This parameter is required if the omodel flag is set. The model will be output to the file \$modeloutdir/\$basename.injectharm.model where \$basename is the base filename of the light curve (with directory names stripped).

```

1 -Injecttransit <"Plist" ["column" col] | "Pfix" per
    | "Pexpr" expr | "Prand" minp maxp
3    | "Plogrand" minp maxp | "randfreq" minf maxf
    | "lograndfreq" minf maxf>
5 <"Rplist" ["column" col] | "Rpfix" Rp | "Rpexpr" expr
    | "Rprand" minRp maxRp | "Rplogrand" minRp maxRp>
7 <"Mplist" ["column" col] | "Mpfix" Mp | "Mpexpr" expr
    | "Mprand" minMp maxMp | "Mplogrand" minMp maxMp>
9 <"phasetlist" ["column" col] | "phasefix" phase
    | "phaseexpr" expr | "phaserand">
11 <"sinilist" ["column" col] | "sinifix" sin_i
    | "siniexpr" expr | "sinirand">
13 <"eomega" <"elist" ["column" col] | "efix" e | "eexpr" expr | "erand">
    <"olist" ["column" col] | "ofix" omega | "oexpr" expr | "orand">
15 | "hk" <"hlist" ["column" col] | "hfix" h | "hexpr" expr | "hrand">
    <"klist" ["column" col] | "kfix" k | "kexpr" expr | "krand">>
17 <"Mstarlist" ["column" col] | "Mstarfix" Mstar | "Mstarexpr" expr>
    <"Rstarlist" ["column" col] | "Rstarfix" Rstar | "Rstarexpr" expr>
19 <"quad" | "nonlin"> <"ldlist" ["column" col]
    | "ldfix" ld1 ... ldN | "ldexpr" ld1 ... ldN>
21 ["dilute" <"list" ["column" col] | "fix" dilute | "expr" diluteexpr>]
    omodel [modeloutdir]

```

Table A.24

Input parameters for "-Injecttransit" command (Section 2.2.5.4).

"Plist"	—	Take the period of the injected transit from the input light curve list file, optionally specifying the column number.
"Pfix"	—	Fix the period of the injected transit to the specified value.
"Pexpr"	—	Evaluate an analytic expression for each light curve to determine the period.
"Prand"	—	Set the period of the inject transit to a random number drawn from a uniform distribution between minp and maxp.
"Plogrand"	—	Choose a random number for the period from a distribution that is uniform in logarithm between minp and maxp.
"randfreq"	—	Choose a random number for the period from a distribution that is uniform in frequency between minf and maxf (in cycles per day).
"lograndfreq"	—	Choose a random number for the period from a distribution that is uniform in the logarithm of the frequency between minf and maxf (in cycles per day).
"Rplist"	—	Take the radius of the planet (in Jupiter radius units) from the input light curve list file, optionally specifying the column number.
"Rpfix"	—	Fix the radius of the planet to the specified value.
"Rpexpr"	—	Evaluate an analytic expression for each light curve to determine the planet radius.
"Rprand"	—	Set the radius of the planet to a random number drawn from a uniform distribution between minRp and maxRp.
"Rplogrand"	—	Set the radius of the planet to a random number drawn from a uniform-log distribution between minRp and maxRp.
"Mplist"	—	Take the mass of the planet (in Jupiter mass units) from the input light curve list file, optionally specifying the column number.
"Mpfix"	—	Fix the mass of the planet to the specified value.
"Mpexpr"	—	Evaluate an analytic expression for each light curve to determine the planet mass.
"Mprand"	—	Set the mass of the planet to a random number drawn from a uniform distribution between minMp and maxMp.
"Mplogrand"	—	Set the mass of the planet to a random number drawn from a uniform-log distribution between minMp and maxMp.
"phasetlist"	—	Take the phase of the orbit (phase = 0 corresponds to transit center) at time $T = 0$ from the input light curve list file, optionally specifying the column number.
"phasefix"	—	Fix the phase at $T = 0$ to the specified value.
"phaseexpr"	—	Evaluate an analytic expression for each light curve to determine the phase.
"phaserand"	—	Set the phase at $T = 0$ to a random number between 0 and 1.
"sinilist"	—	Take the $\sin(i)$, where i is the inclination of the orbit, from the input light curve list file, optionally specifying the column number.
"sinifix"	—	Fix $\sin(i)$ to the specified value.
"siniexpr"	—	Evaluate an analytic expression for each light curve to determine $\sin(i)$.
"sinirand"	—	Choose a random number for $\sin(i)$ drawn from a uniform orientation distribution with the constraint that there must be a transit.
"eomega"	—	Keyword to indicate that the eccentricity and argument of periastron will be specified.
"elist"	—	Take the eccentricity from the input light curve list file, optionally specifying the column number.

(continued on next page)

Table A.24 (continued)

"efix"	—	Fix the eccentricity to the specified value.
"eexpr"	—	Evaluate an analytic expression for each light curve to determine the eccentricity.
"erand"	—	Adopt a random number for the eccentricity.
"olist"	—	Take the argument of periastron (in degrees) from the input light curve list file, optionally specifying the column number.
"ofix"	—	Fix the argument of periastron to the specified value.
"oexpr"	—	Evaluate an analytic expression for each light curve to determine the argument of periastron.
"orand"	—	Adopt a random number for the argument of periastron.
"hk"	—	Keyword to indicate that $e \sin \omega \equiv h$ and $e \cos \omega \equiv k$ will be specified rather than the eccentricity and argument of periastron.
"hlist"	—	Take $e \sin \omega$ from the input light curve list file, optionally specifying the column number.
"hfix"	—	Fix $e \sin \omega$ to the specified value.
"hexpr"	—	Evaluate an analytic expression for each light curve to determine $e \sin \omega$.
"hrand"	—	Adopt a random number for $e \sin \omega$.
"klist"	—	Take $e \cos \omega$ from the input light curve list file, optionally specifying the column number.
"kfix"	—	Fix $e \cos \omega$ to the specified value.
"kexpr"	—	Evaluate an analytic expression for each light curve to determine $e \cos \omega$.
"krand"	—	Adopt a random number for $e \cos \omega$.
"Mstarlist"	—	Take the mass of the star (in solar masses) from the input light curve list file, optionally specifying the column number.
"Mstarfix"	—	Fix the mass of the star to the specified value.
"Mstarexpr"	—	Evaluate an analytic expression for each light curve to determine the stellar mass.
"Rstarlist"	—	Take the radius of the star (in solar radii) from the input light curve list file, optionally specifying the column number.
"Rstarfix"	—	Fix the radius of the star to the specified value.
"Rstarexpr"	—	Evaluate an analytic expression for each light curve to determine the stellar period.
"quad" "nonlin"	—	Keywords to indicate whether a 2-parameter quadratic limb darkening law or a 4-parameter non-linear limb darkening law should be used.
"ldlist"	—	Take the limb darkening coefficients from the input light curve list file, optionally specifying the column number of the first coefficient (other coefficients must be in the subsequent columns).
"ldfix"	—	Fix the limb darkening coefficients to the specified values. Two values must be supplied if the "quad" keyword was used, or four values if the "nonlin" keyword was used.
"ldexpr"	—	Evaluate an analytic expression to determine the limb darkening coefficients. Two expressions must be supplied if the "quad" keyword was used, or four expressions if the "nonlin" keyword was used.
"dilute"	—	An optional keyword indicating that the transit signal should be scaled by a value between 0 and 1. One must then either use "list" to indicate that the scaling factor should be read from the input light curve list, "fix" to specify on the command line the value to use for all light curves, or "expr" to provide an analytic expression to be evaluated for each light curve.
"omodel"	—	An optional keyword used to output the model transit light curve, evaluated at the times of observation in the input light curve. The model will be written to a file of the name \$modeloutdir/\$basename.injecttransit.model where \$basename is the base filename of the input light curve, stripped of any directories.

—Jstet timescale dates

Table A.25

Input parameters for "-Jstet" command (Section 2.2.2.5).

timescale	—	The time, in minutes (assuming input times are in days), that distinguishes between "near" and "far" observations.
dates	—	A file containing a list of all times that appear in the light curves in the first column. This is used to calculate the maximum possible weight for a light curve.

```

1 -Killharm <"aov" | "ls" | "both" | "injectharm"
  | "fix" Nper per1 ... perN
3 | "list" Nper ["column" col1]> Nharm Nsubharm
  omodel [model_outdir] ["fitonly"]
5 ["outampphase" | "outampphase" | "outRphi" | "outRadphi"]
  ["clip" val]

```

Table A.26

Input parameters for "-Killharm" command (Section 2.2.3.2).

"aov"	—	Take the period for the harmonic series from the most recently executed "-aov" or "-aov_harm" command.
"ls"	—	Take the period for the harmonic series from the most recently executed "-LS" command.
"both"	—	Use two periods, taking one from the most recently executed "-aov" or "-aov_harm" command and the other from the most recently executed "-LS" command.
"injectharm"	—	Take the period for the harmonic series from the most recently executed "-Injectharm" command.
"fix"	—	Fix the period(s) to the specified value(s). Nper is the number of periods to use, and per1 through perN is a space-delimited list of the Nper periods.
"list"	—	Take the period(s) from the input light curve list. Nper is the number of periods to use, and they must be in consecutive columns within the list file. You may optionally specify the column for the first period using the "column" keyword.

(continued on next page)

Table A.26 (continued)

Nharm	– The number of higher-harmonics to use in the harmonic series that is fit for each period (frequencies of $2f_0, 3f_0$, up to $(N_{\text{harm}} + 1)f_0$ will be used for these harmonics, where f_0 is the fundamental frequency). Set this to zero to fit only a sinusoid.
Nsubharm	– The number of sub-harmonics to use in the harmonic series that is fit for each period (frequencies of $f_0/2, f_0/3$ up to $f_0/(N_{\text{subharm}} + 1)$). Set this to zero to fit only a sinusoid.
omodel	– A flag indicating whether or not to output the model harmonic series light curves.
model_outdir	– This parameter is required if omodel is set to 1. The model harmonic series light curves will be written to files named $\$model_outdir/\$basename.killharm.model$ where $\$basename$ is the base filename of the input light curve (stripped of directories).
“fitonly”	– This optional keyword, if used, will cause VARTOOLS to not subtract the harmonic series from the light curve, and instead only perform the fit reporting the coefficients. By default the harmonic series will be subtracted from the light curve before passing it on to the next command.
“outampphase”	– By default the a_k and b_k coefficients of the sin and cos functions in the harmonic series for harmonic number k are output. If the “outampphase” keyword is given, then the amplitudes of the harmonics $A_k = \sqrt{a_k^2 + b_k^2}$ and phases between 0 and 1 ($\phi_k = \text{atan2}(-b_k, a_k)/2\pi$) will be output.
“outampradphase”	– If this keyword is given, then the amplitudes of the harmonics $A_k = \sqrt{a_k^2 + b_k^2}$ and phases in radians $\phi_k = \text{atan2}(-b_k, a_k)$ will be output.
“outRphi”	– If this keyword is given, then the amplitudes of the harmonics relative to the fundamental $R_{k1} = A_k/A_1$ and relative phases $\phi_{k1} = \phi_k - k\phi_1$ (between 0 and 1) will be output. Note for sub-harmonics $k = 1/2, 1/3$, etc. For the fundamental mode the amplitude A_1 and phase ϕ_1 will be given.
“outRradphi”	– Similar to “outRphi”, in this case phases are in radians.
“clip”	– If this optional keyword is given then the model will be fit, σ -clipping with a clipping factor given by val will be applied to the residuals, and the model will be refit to the points which passed the clipping.

```

1  -linfit function paramlist ["modelvar" varname]
2    ["correctlc"]
    ["omodel" model_outdir ["format" nameformat]]

```

Table A.27

Input parameters for “-linfit” command (Section 2.2.3.3).

function	– Analytic expression to fit to the light curve magnitudes. This expression should be linear in the free parameters. An example is ‘a*t^2+b*t+c’ to fit a quadratic function in time to the light curve, with ‘a’, ‘b’ and ‘c’ being free parameters.
paramlist	– A comma-delimited list of variables which are the free parameters to optimize in the function. These must all enter linearly into the function. For the previous example one would supply ‘a, b, c’ for this parameter.
“modelvar”	– Optional keyword used to store the best-fit model light curve in a variable indicated by varname.
“correctlc”	– If this keyword is given then the best-fit model will be subtracted from the light curve before passing it on to the next command. By default the model is not subtracted from the light curve.
“omodel”	– Optional keyword used to indicate that the best-fit model light curve should be output to a file. The name of the file is given by $\$model_outdir/\$basename.linfit.model$ where $\$basename$ is the base filename of the input light curve stripped of directories. The formatkeyword may be used to change the rule for naming the file as indicated below.
“format”	– Optional keyword used in conjunction with the “omodel” keyword to change the rule for naming the output model light curve files. In this case the file will be named $\$model_outdir/\$nameformat$ where instances of %s in $\$nameformat$ are replaced by $\$basename$, instances of %d are replaced by the light curve number (starting with 1), instances of %0nd where n is an integer are replaced with the formatted light curve number, and instances of %% are replaced with %.

```

1  -LS minp maxp subsample Npeaks operiodogram [outdir] ["noGLS"] ["whiten"]
    ["clip" clip clipiter] ["fixperiodSNR" <"aov" | "ls" | "injectharm"
2    | "fix" period | "list" ["column" col]
3    | "fixcolumn" <colname | colnum>>]

```

Table A.28

Input parameters for “-LS” command (Section 2.2.1.1).

minp	– The minimum period to search (in the same time units as the input light curve).
maxp	– The maximum period to search (in the same time units as the input light curve).
subsample	– The periodogram will be scanned at a frequency resolution of $\Delta f = \text{subsample}/T$ where T is the time base-line of the input light curve.
operiodogram	– Flag used to indicate whether or not the periodograms should be output to separate files.
outdir	– This parameter is supplied only if operiodogram is set to 1. The periodograms will be output to files named $\$outdir/\$basename.ls$ where $\$basename$ is the base filename of the input light curve stripped of any directories.
“noGLS”	– If this keyword is given then the traditional L–S periodogram will be calculated. By default the Generalized L–S periodogram is calculated.
“whiten”	– Keyword used to whiten the light curve and recalculate the periodogram for each peak. By default this is not done.

(continued on next page)

Table A.28 (continued)

“clip”	–	Adjust the clipping performed on the periodogram in calculating the spectroscopic S/N of a peak. By default iterative 5σ clipping is performed.
clip	–	The sigma-clipping factor to use.
clipiter	–	Flag which if set causes iterative clipping to be performed.
“fixperiodSNR”	–	Report the periodogram value, the false alarm probability and the S/N for a fixed period in addition to the highest peaks found.
“aov”	–	Use the peak period from the last executed -aov or -aov_harm command for the fixperiodSNR.
“ls”	–	Use the peak period from the last executed -LS command.
“injectharm”	–	Use the period from the last executed -Injectharm command (the first one is used if multiple periods were injected).

```

–MandelAgolTransit <"bls" | "blsfixper"
2 | P0 T00 r0 a0 <"i" inclination | "b" bimpact> e0 omega0 mconst0>
<"quad" | "nonlin"> ldcoeff{1}_{0} ... ldcoeffn_0 fitephem
4 fitr fita fitinclterm fite fitomega fitmconst fitldcoeff1 ... fitldcoeffn
fitRV [RVinputfile RVmodeloutfile K0 gamma0 fitK fitgamma]
6 correctlc omodel [model_outdir]
["modelvar" var] ["ophcurve" curve\_outdir phmin phmax phstep]
8 ["ojdcurve" curve\_outdir jdstep]

```

Table A.29

Input parameters for “-MandelAgolTransit” command (Section 2.2.3.4).

“bls”	–	Initialize the parameters based on the highest peak found with the most recent prior “-BLS” command.
“blsfixper”	–	Initialize the parameters based on the highest peak found with the most recent prior “-BLSFixPer” command.
P0	–	Initial orbital period.
T00	–	Initial transit epoch (time at the center of a transit).
r0	–	Initial ratio of the planet radius to the star radius (R_p/R_*).
a0	–	Initial ratio of the semi-major axis to the stellar radius (a/R_*).
“i”	–	Keyword used to indicate that the next parameter given is the initial orbital inclination angle, in degrees.
“b”	–	Keyword used to indicate that the next parameter given is the initial normalized impact parameter. This is the distance between the projected centers of the planet and star at conjunction divided by the sum of the planet and stellar radii (note, not just the stellar radius).
“e0”	–	Initial orbital eccentricity.
“omega0”	–	Initial argument of periastron in degrees.
“mconst0”	–	Initial out-of-transit magnitude. If a negative value is specified, then the optimal value will be determined automatically.
“quad”	–	Keyword used to indicate that a two-parameter quadratic limb darkening law is used.
“nonlin”	–	Keyword used to indicate that a four-parameter non-linear limb darkening law is used.
ldcoeff1_0 ... ldcoeffn_0	–	A space separated list of initial limb darkening parameters (2 for the quadratic law, 4 for the non-linear law).
fitephem	–	Flag indicating whether or not the ephemeris (P and $T00$) should be varied in the fit.
fitr	–	Flag indicating whether R_p/R_* should be varied in the fit.
fita	–	Flag indicating whether a/R_* should be varied in the fit.
fitinclterm	–	Flag indicating whether the inclination term (either the inclination angle itself, or the impact parameter, whichever was specified) should be varied in the fit.
fite	–	Flag indicating whether the eccentricity should be varied in the fit.
fitomega	–	Flag indicating whether the argument of periastron should be varied in the fit.
fitmconst	–	Flag indicating whether the out-of-transit magnitude should be varied in the fit.
fitldcoeff1 ... fitldcoeffn	–	Flags indicating whether the respective limb-darkening coefficients should be varied in the fit.
fitRV	–	Flag indicating whether a separate RV curve should be fit together with the light curve.
RVinputfile	–	The name of the file containing the RV data to fit. The expected file format is: JD, RV, RV uncertainty. This parameter should be given if and only if fitRV is set to 1.
RVmodeloutfile	–	The name of the file to which the best-fit RV orbit model (evaluated at the input times) should be output. This parameter should be given if and only if fitRV is set to 1.
K0	–	The initial RV orbital semi-amplitude. This parameter should be given if and only if fitRV is set to 1.
gamma0	–	The initial RV zero-point. This parameter should be given if and only if fitRV is set to 1.
fitK	–	Flag indicating whether the RV semi-amplitude should be varied in the fit. This parameter should be given if and only if fitRV is set to 1.
fitgamma	–	Flag indicating whether the RV zero-point should be varied in the fit. This parameter should be given if and only if fitRV is set to 1.

(continued on next page)

Table A.29 (continued)

correctlc	–	Flag indicating whether the best-fit model should be subtracted from the light curve before passing it on to the next command.
omodel	–	Flag indicating whether the best-fit model light curve should be output to a file. The model will be evaluated at the times of observation.
model_outdir	–	The model will be written to the file given by \$model_outdir/\$basename.mandelagoltransit.model where \$basename is the base filename of the input light curve, stripped of directories. This parameter should be given if and only if omodel is set to 1.
“modelvar”	–	Optional keyword used to store the best-fit model light curve in a variable indicated by <i>var</i> .
“ophcurve”	–	A keyword used to output best-fit model light curves that are uniformly sampled in phase.
curve_outdir	–	The models will be written out to files named \$outdir/\$basename.mandelagoltransit.phcurve.
phmin	–	The starting phase to use in the model.
phmax	–	The ending phase to use in the model.
phstep	–	The phase step-size to use in the model.
“ojdcurve”	–	A keyword used to output best-fit model light curves that are uniformly sampled in time between the first and last observed times in the light curve.
curve_outdir	–	The models will be written out to files named \$outdir/\$basename.mandelagoltransit.jdcurve.
jdstep	–	The time step-size to use in the model.

–medianfilter time ["average" | "weightedaverage"] ["replace"]

Table A.30

Input parameters for “-medianfilter” command (Section 2.2.4.2).

time	–	The time separation used by the filter. By default, a high-pass filter is applied, whereby the median magnitude of all points within ‘time’ of a given observation is subtracted from the magnitude for that observation.
“average”	–	Optional keyword used to take the average of the magnitudes, rather than the median.
“weightedaverage”	–	Optional keyword used to take the weighted average of the magnitudes, rather than the median.
“replace”	–	Optional keyword used to perform a low-pass filter, whereby the magnitude of a given observation will be replaced by the median (or average, or weighted average) magnitude of all points within ‘time’ of that observation.

```

1  –microlens
   ["f0"
3     ["fix" fixval | "list" ["column" col]
       | "fixcolumn" <colname | colnum>
5     | "auto"]
       ["step" initialstepsize] ["novary"]]
7  ["f1"
   ["fix" fixval | "list" ["column" col]
9     | "fixcolumn" <colname | colnum>
       | "auto"]
11    ["step" initialstepsize] ["novary"]]
13  ["u0"
   ["fix" fixval | "list" ["column" col]
15    | "fixcolumn" <colname | colnum>
       | "auto"]
17    ["step" initialstepsize] ["novary"]]
19  ["t0"
   ["fix" fixval | "list" ["column" col]
21    | "fixcolumn" <colname | colnum>
       | "auto"]
23    ["step" initialstepsize] ["novary"]]
25  ["tmax"
   ["fix" fixval | "list" ["column" col]
27    | "fixcolumn" <colname | colnum>
       | "auto"]
       ["step" initialstepsize] ["novary"]]
["correctlc"] ["omodel" outdir]
```

Table A.31

Input parameters for “-microlens” command (Section 2.2.3.5).

“f0”	–	Keyword used to change the method for initializing and fitting the f_0 parameter in Eq. (63). If this keyword is not used, the parameter will be automatically initialized and then varied in the fit.
“fix”	–	Keyword indicating that the initial value should be fixed to the value supplied for fixval.
“list”	–	Keyword indicating that the initial value should be taken from the input light curve list file (the column to read it from may also be optionally specified).
“fixcolumn”	–	Keyword indicating that the initial value should be taken from the output of a previously executed command.
“auto”	–	Keyword indicating that the initial value should be determined automatically (the default behavior).

(continued on next page)

Table A.31 (continued)

"step"	— Keyword used to change the initial step-size for varying the parameter. By default the step-size is determined automatically.
"novary"	— If this keyword is specified then the parameter will not be varied in the fit.
"f1"	— Keyword used to change the method for initializing and fitting the f_1 parameter in Eq. (63). If this keyword is not used, the parameter will be automatically initialized and then varied in the fit. The syntax following this keyword is the same as for "f0".
"u0"	— Keyword used to change the method for initializing and fitting the u_0 parameter in Eq. (65). If this keyword is not used, the parameter will be automatically initialized and then varied in the fit. The syntax following this keyword is the same as for "f0".
"t0"	— Keyword used to change the method for initializing and fitting the t_0 parameter in Eq. (65). If this keyword is not used, the parameter will be automatically initialized and then varied in the fit. The syntax following this keyword is the same as for "f0".
"tmax"	— Keyword used to change the method for initializing and fitting the $t_{\max}$ parameter in Eq. (65). If this keyword is not used, the parameter will be automatically initialized and then varied in the fit. The syntax following this keyword is the same as for "f0".
"correctlc"	— If this keyword is given then the best-fit model will be subtracted from the light curve before passing it on to the next command. By default the light curve is not modified by this command.
"omodel"	— If this keyword is given the best-fit model, evaluated at the observed times in the light curve, will be output to a file whose name is given by \$outdir/\$basename.microlens where \$basename is the base filename of the light curve, stripped of directories.

```

1 -nonlinfit function paramlist ["linfit" linfitparams]
  ["errors" error_expr]
3  ["covariance"
   <"squareexp" amp_var rho_var
5   | "exp" amp_var rho_var
   | "matern" amp_var rho_var nu_var>]
7  ["priors" priorlist] ["constraints" constraintlist]
  <"amoeba" ["tolerance" tol] ["maxsteps" steps]
9  | "mcmc" ["Naccept" N | "Nlinkstotal" N]
   ["fracburnin" frac] ["eps" eps] ["skipamoeba"]
11  ["chainstats" exprlist statslist]
   ["maxmemstore" maxmem]
13  ["outchains" outdir ["format" format] ["printevery" N]] >
["modelvar" varname] ["correctlc"]
15 ["omodel" model_outdir ["format" nameformat]]

```

Table A.32

Input parameters for "-nonlinfit" command (Section 2.2.3.6).

function	— Analytic expression to fit to the light curve magnitudes. For example this might be 'a*exp(-(t-t0)^2/2/sigma^2)+b' for a Gaussian function in time with free parameters 'a', 't0', 'sigma' and 'b'.
paramlist	— A comma-delimited list of parameters to vary in the fit. For each parameter you must specify the initial guess and step-size using the format "var=init:step". For example, if the initial value for 't0' is 5.0, and its step-size is 2.0, and if the initial value for 'sigma' is 10.0 and its step-size is 8.0, you would use the expression 't0=5.0:2.0,sigma=10.0:8.0'. The initial values and step-sizes may also be more complicated analytic expressions using variables, functions, etc. Note that the free parameters should have names that are not used by any vector variables (e.g., 't', 'mag', 'err', or other variables defined by the -expr command or -inputlcf option). Note also that these variables may be used by other commands as well (e.g., on the right-hand-side of the expression provided to the -expr command).
"linfit"	— Optional keyword used to indicate any parameters in the function which enter linearly and should be optimized using linear least squares. In this case linfitparams is a comma-separated list of the linear parameters. For example, in the Gaussian case this could be 'linear a, b'.
"errors"	— Optional keyword used to change the magnitude uncertainties used in calculating the likelihood function (Eq. (66)). error_expr is an analytic expression. Parameters which are varied in the fit may also be included in this expression.
"covariance"	— Optional keyword used to indicated correlated uncertainties (by default errors are assume to be uncorrelated).
"squareexp"	— Use a square-exponential covariance matrix (Eq. (68)).
amp_var	— The name of the VARTOOLS variable which should be used as a in Eq. (68). If this variable does not appear in paramlist then it should be specified here as 'amp_var=expr' where amp_var is the name of the variable to use, and expr is an expression used to determine the fixed value to be used for this parameter.
rho_var	— The name of the VARTOOLS variable which should be used as ρ in Eq. (68). If this variable does not appear in paramlist then it should be specified here as 'rho_var=expr' where rho_var is the name of the variable to use, and expr is an expression used to determine the fixed value to be used for this parameter.
"exp"	— Use an exponential covariance matrix (Eq. (69)).
amp_var	— The name of the VARTOOLS variable which should be used as a in Eq. (69). If this variable does not appear in paramlist then it should be specified here as 'amp_var=expr' where amp_var is the name of the variable to use, and expr is an expression used to determine the fixed value to be used for this parameter.
rho_var	— The name of the VARTOOLS variable which should be used as ρ in Eq. (69). If this variable does not appear in paramlist then it should be specified here as 'rho_var=expr' where rho_var is the name of the variable to use, and expr is an expression used to determine the fixed value to be used for this parameter.
"matern"	— Use a Matérn covariance matrix (Eq. (70)).

(continued on next page)

Table A.32 (continued)

amp_var	—	The name of the VARTOOLS variable which should be used as a in Eq. (70). If this variable does not appear in paramlist then it should be specified here as 'amp_var=expr' where amp_var is the name of the variable to use, and expr is an expression used to determine the fixed value to be used for this parameter.
rho_var	—	The name of the VARTOOLS variable which should be used as ρ in Eq. (70). If this variable does not appear in paramlist then it should be specified here as 'rho_var=expr' where rho_var is the name of the variable to use, and expr is an expression used to determine the fixed value to be used for this parameter.
nu_var	—	The name of the VARTOOLS variable which should be used as ν in Eq. (70). If this variable does not appear in paramlist then it should be specified here as 'nu_var=expr' where nu_var is the name of the variable to use, and expr is an expression used to determine the fixed value to be used for this parameter.
"priors"	—	Keyword used to indicate a list of priors to place on the variables. priorlist is a comma-separated list, where each entry in the list is an analytic expression that should evaluate to $-2 \ln(P)$ where P is the prior probability for a variable given its value. For example, to place a Gaussian prior on 't0' with mean 4.0 and standard deviation 3.0 you would use 'prior (t0-4.0)^2/3.0^2'.
"constraints"	—	Use this keyword to place constraints on the parameters. constraintlist is a comma-separated list of expressions. For example, to require a positive 'sigma' you can use 'constraints sigma>0'.
"amoeba"	—	Use the down-hill simplex optimization algorithm.
"tolerance"	—	Optional keyword to change the convergence tolerance. Here 'tol' corresponds to TOL in Eq. (71).
"maxsteps"	—	Optional keyword to change the maximum number of amoeba iterations that are tried before giving up.
"mcmc"	—	Run a Differential Evolution Markov Chain Monte Carlo procedure.
"Naccept"	—	Optionally specify the number of accepted links to run in a given fit. Once this number is reached, the MCMC will terminate.
"Nlinkstotal"	—	Optionally specify a maximum total number of links to run. Once this number is reached, the MCMC will terminal. This option is assumed by default, with a limit of 100,000 links.
"fracburnin"	—	Optionally change the initial fraction of the chain that is ignored in computing statistics to be reported from the posterior distribution (the default is 0.1).
"eps"	—	Optionally change the value of ϵ to use in Eq. (72). The default value is 0.001.
"skipamoeba"	—	By default the down-hill simplex algorithm is run initially before starting the mcmc procedure. This can be skipped by using this keyword.
"chainstats"	—	By default the median and standard deviation for each of the parameters varied in the MCMC will be included in the output table. You may use this keyword to change the statistics, and/or the quantities that are used. Here 'explist' is a comma-separated list of analytic expressions to calculate from the chain, and 'statslist' is a comma-separated list of statistics to report for each of the expressions. The available statistics are the same as for the -stats command.
"maxmemstore"	—	Change the maximum limit on the total amount of memory to be used by this MCMC command. This will limit the length of the chain used for calculating the output statistics. 'maxmem' is the limit in GB. The default is 4.0.
"outchains"	—	Use this keyword to output the MCMC chains. The chains will be output to files named \$model_outdir/\$basename.mcmc, where \$basename is the base filename of the input light curve file, stripped of directories.
"format"	—	Use this keyword to change the naming convention for the output MCMC files. In this case the file will be named \$model_outdir/\$nameformat where instances of %s in \$nameformat are replaced by \$basename, instances of %d are replaced by the light curve number (starting with 1), instances of %0nd, where n is an integer, are replaced with the formatted light curve number, and instances of %% are replaced with %.
"printevery"	—	Use this keyword to change the number of links that are written to the output file. By default every link is written out. If this keyword is used, then every 'N'th link is written out.
"modelvar"	—	Optional keyword used to store the best-fit model light curve in a variable indicated by varname.
"correctlc"	—	Use this keyword to subtract the best-fit model light curve from the input light curve before passing it on to the next command.
"omodel"	—	If this keyword is given, the best-fit model, evaluated at the observed times in the light curve, will be output to a file whose name is given by \$model_outdir/\$basename.nonlinfit.model where \$basename is the base filename of the light curve, stripped of directories.
"format"	—	Use this keyword to change the naming convention for the output model light curve. In this case the file will be named \$model_outdir/\$nameformat where instances of %s in \$nameformat are replaced by \$basename, instances of %d are replaced by the light curve number (starting with 1), instances of %0nd, where n is an integer, are replaced with the formatted light curve number, and instances of %% are replaced with %.

```

1 -o <outdir | outname> ["nameformat" formatstring]
   ["columnformat" formatstring]
3   ["fits"] ["noclobber"]

```

Table A.33

Input parameters for "-o" command (Section 2.2.7.3).

outdir	—	The directory to which the light curve files will be written. By default the output filenames will be \$outdir/\$basename where \$outdir is the value supplied for 'outdir' and \$basename is the base filename of the light curve, stripped of directories. The supplied parameter will be interpreted as the output directory only if a light curve list was used for input (-l).
outname	—	The name of the output light curve file. This will be used if an individual light curve file was used for input (-i).
"nameformat"	—	Use this keyword to change the naming convention for the output light curves. In this case the file will be named \$outdir/\$formatstring where instances of %s in \$formatstring are replaced by \$basename, instances of %d are replaced by the light curve number (starting with 1), instances of %0nd, where n is an integer, are replaced with the formatted light curve number, and instances of %% are replaced with %.

(continued on next page)

Table A.33 (continued)

"columnformat"	—	Change the format of the output light curves. By default they will have three columns: time, mag and err. Here 'formatstring' is a comma-separated list of variable names to output, optionally with a ':' after each variable name to specify the "printf"-style format to use for that variable. For example, 'columnformat t:%.17g,mag:%.5f,err:%.5f,xpos:%.3f' would output the variables t, mag, err, and xpos using 17-digit-precision for t, and floating points for mag, err and xpos, with 5, 5, and 3 digits after the decimal, respectively. Here 'xpos' is a non-default variable that one might have read-in with the -inputcolumnformat option, or produced through one of the other commands. If the light curves are output in fits format, then terms after the ':' will be used to specify the units of the column to list in the light curve header.
"fits"	—	Use this keyword to output the light curves in binary FITS table format. The output light curves will have the extension ".fits" appended to their filenames, if not already present.
"noclobber"	—	Use this keyword to prevent over-writing any existing files. VARTOOLS will terminate if it encounters an existing file when this keyword is used.

```

1  -Phase <"aov" | "ls" | "bls" | "fixcolumn" <colname | colnum>
      | "list" ["column" col] | "fix" period>
3    ["T0" <"bls" phaseTc | "fixcolumn" <colname | colnum>
      | "list" ["column" col] | "fix" T0>]
5    ["phasevar" var] ["startphase" startphase]

```

Table A.34

Input parameters for "-Phase" command (Section 2.2.6.8).

"aov"	—	Use this keyword to take the period at which to phase-fold the light curve from the most recently issued -aov or -aov_harm command.
"ls"	—	Use this keyword to take the period at which to phase-fold the light curve from the most recently issued -LS command.
"bls"	—	Use this keyword to take the period at which to phase-fold the light curve from the most recently issued -BLS command.
"T0"	—	By default the light curves are folded such that phase zero corresponds to time $t = 0$. Use this keyword to specify a different reference time for phase zero.
"bls"	—	Take the reference time to be the time of central transit found in the most recent -BLS command. Here 'phaseTc' is the phase to adopt for the central transit time.
"phasevar"	—	Use this keyword to write the phases to the variable <i>var</i> rather than overwriting the time variable. If the variable <i>var</i> does not exist it will be created. If it does exist, it must be a double-precision light curve vector (e.g., it cannot be the name of an output column, or a parameter used in a fit, etc.).
"startphase"	—	Use this keyword to change the range over which the phases are expressed. By default they run from 0 to 1. With this keyword they will run from <i>startphase</i> to <i>startphase</i> + 1.

```

1  -resample
      <"nearest" |
3    "linear" |
      "spline" ["left" yp1] ["right" ypn] |
5    "splinemonotonic" |
      "bspline" ["nbreaks" nbreaks] ["order" order] >
7  ["file" <"fix" times_file | "column" time_column] |
      "list" ["listcolumn" col] ["tcolumn" time_column] > |
9  ["tstart" <"fix" tstart | "fixcolumn" <colname | colnum> |
      "list" ["column" col] | "expr" expression > ]
11 ["tstop" <"fix" tstop | "fixcolumn" <colname | colnum> |
      "list" ["column" col] | "expr" expression > ]
13 ["delt" <"fix" delt | "fixcolumn" <colname | colnum> |
      "list" ["column" col] | "expr" expression > ]
15 | ["Npoints" <"fix" Np | "fixcolumn" <colname | colnum> |
      "list" ["column" col] | "expr" expression > ]]
17 ["gaps"
      <"fix" time_sep | "fixcolumn" <colname | colnum> |
19    "list" ["column" col] | "expr" expression |
      "frac_min_sep" val | "frac_med_sep" val | "percentile_sep" val>
21 <"nearest" |
      "linear" |
23    "spline" ["left" yp1] ["right" ypn] |
      "splinemonotonic" |
25    "bspline" ["nbreaks" nbreaks] ["order" order] >]
["extrap"
27 <"nearest" |
      "linear" |
29    "spline" ["left" yp1] ["right" ypn] |
      "splinemonotonic" |
31    "bspline" ["nbreaks" nbreaks] ["order" order] >]

```

Table A.35

Input parameters for “-resample” command (Section 2.2.6.9).

“nearest”	—	Set resampled values to the value of the observation that is closed in time to the resampled point.
“linear”	—	Perform linear interpolation between points.
“spline”	—	Perform cubic spline interpolation.
“left”	—	Specify the left boundary condition for the spline.
“right”	—	Specify the right boundary condition for the spline.
“splinemonotonic”	—	Perform cubic spline interpolation with the interpolating function constrained to be monotonic between input observations.
“bspline”	—	Interpolate with a Basis-spline function.
“nbreaks”	—	The number of breaks to use in the B-spline (the default is 15). If the number given is < 2 , then the routine will increase the number of breaks until χ^2 per degree of freedom is less than one. Caution, this can be quite slow.
“order”	—	The order of the spline function to use (the default is 3).
“file”	—	Optional keyword to read the time-base on which to resample the light curve from a file. By default VARTOOLS will resample the light curve onto a uniform time-base with the starting and stopping times equal to the minimum and maximum observed times in the light curve, respectively, and with a time step equal to the minimum time separation between consecutive points in the input light curve.
“fix”	—	Use the same time-base file for all light curves. This file is given by ‘times_file’. Use the optional “column” keyword to specify the column in the file which stores the times to use.
“list”	—	Allow a different time-base file for each light curve. The filenames in this case are taken from the input light curve list file. Use the optional “listcolumn” keyword to change the column in the input light curve list file to use, use the “tcolumn” keyword to change the columns in the files to use for the new times.
“tstart”	—	Optional keyword to change the starting time for a regularly spaced grid of times to resample the light curve at.
“expr”	—	In addition to the standard “fix”, “fixcolumn”, and “list” options for determining the starting time, one may also use this keyword to set it equal to the result from evaluating an analytic expression given by ‘expression’.
“tstop”	—	Optional keyword to change the starting time for a regularly spaced grid of times to resample the light curve at. Options for determining this parameter are the same as for “tstart”.
“delt”	—	Optional keyword to change the time step for a regularly spaced grid of times to resample the light curve at. Options for determining this parameter are the same as for “tstart”.
“Npoints”	—	Optional keyword to change the number of points to resample the light curve onto. Either “delt” or “Npoints” may be specified. Options for determining this parameter are the same as for “tstart”.
“gaps”	—	By default the same resampling method will be used for all points in the light curve. By giving the “gaps” keyword you can make the method depend on how far away a resampled time is from the closest observed time. You first need to indicate how the time separation used to distinguish between the near and far points will be determined, you then indicate the method for resampling the “far” points (the near points will be resampled using the method already indicated before providing the “gaps” keyword). Below are the options for determining the time separation. The options for determining the resampling method are the same as discussed above.
“fix”	—	Fix the time separation to value specified by ‘time_sep’.
“fixcolumn”	—	Set the time separation equal to a value calculated by a previously executed command.
“list”	—	Take the value from the input light curve list file.
“expr”	—	Set the value equal to the result from evaluating an analytic expression.
“frac_min_sep”	—	Set the time separation to a fixed factor times the minimum separation between subsequent points in the input light curve (e.g., if you give “frac_min_sep 5.0” and the minimum separation between points in the light curve is 1 day, then the separation times scale will be set to 5 days).
“frac_med_sep”	—	Same as “frac_min_sep”, except using the median separation between subsequent points rather than the minimum separation.
“percentile_sep”	—	Set the time separation to a specified percentile of the separations in the input light curve (e.g., if you give “percentile_sep 70”, then the N separations in the input light curve will be ordered, and the separation to be used for distinguishing between near and far points will be set equal to the $0.7 \times N$ longest separation in the input light curve).
“extrap”	—	Change the method for evaluating resampled points which are extrapolations rather than interpolations. Options are as already discussed. By default extrapolations and interpolations are performed using the same method.

1 **-rescalesig**

No parameters or options are available.

1 **-restorelc** savenumber**Table A.36**

Input parameters for “-restorelc” command (Section 2.2.7.4).

savenumber	—	Restores the light curve to its state at the ‘savenumber’-th call to -savelc on the command line.
------------	---	--

```

1 -restricttimes ["exclude"]
  < "JDrange" minJD maxJD |
3   "JDrangebylc"
    <"fix" minJD | "list" ["column" col] | "fixcolumn" <colname | colnum> |
5     "expr" expression>
    <"fix" maxJD | "list" ["column" col] | "fixcolumn" <colname | colnum> |
7     "expr" expression> |
  "JDlist" JDfilename |
9  "imagelist" imagefilename >

```

Table A.37

Input parameters for “-restricttimes” command (Section 2.2.4.3).

“exclude”	—	If this optional keyword is given, then the times specified below will be removed from the light curve. By default times specified are included, and other times are excluded.
“JDrange”	—	Provide a fixed range of times to keep (or exclude) with ‘minJD’ and ‘maxJD’.
“JDrangebylc”	—	Provide a range of times to keep (or exclude), allowing the limits on the range to change between light curves. If this keyword is given, then the method for determining the ‘minJD’ should be provided (either fixed to a constant value, read from the input light curve list, set to the result of a previously executed command, or set equal to the result from evaluating an analytic expression. The method for determining ‘maxJD’ should then be provided, with the same set of options.
“JDlist”	—	Provide a list of individual JD values to keep (or exclude) in the file ‘JDfilename’. The JD values should be given in the first column of this file.
“imagelist”	—	Provide a list of image IDs to keep (or exclude) in the file ‘imagefilename’. The IDs should be given in the first column of this file.

1 **—rms**

No parameters or options are available.

1 **—rmsbin** Nbin bintime1...bintimeN**Table A.38**

Input parameters for “-rmsbin” command (Section 2.2.2.7).

Nbin	—	The number of moving mean filters to use.
bintime1...bintimeN	—	A space-delimited list of filter widths, one for each of the Nbin filters used. The width of each filter is given by $2.0 * \text{bintime}$, where bintime is in minutes, assuming the times in the light curve are in days.

1 **—save1c**

No parameters or options are available.

```
1 —SoftenedTransit <"bls" | "blsfixper" | P0 T00 eta0 delta0 mconst0 cval0>
  fitephem fiteta fitcval fitdelta fitmconst correctlc
3 omodel [model_outdir] fit_harm [<"aov" | "ls" | "bls"
  | "list" ["column" col] | "fix" Pharm> nharm nsubharm]
```

Table A.39

Input parameters for “-SoftenedTransit” command (Section 2.2.3.7).

“bls”	—	Use this keyword to initialize the parameters for this model based on the highest peak found in the most recently executed -BLS command.
“blsfixper”	—	Use this keyword to initialize the parameters for this model based on the most recently executed -BLSFixPer command.
P0	—	The initial period to use for the model (P in Eq. (74)).
T00	—	The initial transit epoch to use for the model (T_0 in Eq. (74)).
eta0	—	The initial transit duration to use for the model (η in Eq. (74)).
delta0	—	The initial transit depth to use for the model (δ in Eq. (73)).
mconst0	—	The initial out-of-transit magnitude to use for the model (M_0 in Eq. (73)).
cval0	—	The initial value for the transit sharpness parameter (c in Eq. (73)).
fitephem	—	Flag indicating whether or not P and T_0 are to be varied in the fit.
fiteta	—	Flag indicating whether or not η is to be varied in the fit.
fitcval	—	Flag indicating whether or not c is to be varied in the fit.
fitdelta	—	Flag indicating whether or not δ is to be varied in the fit.
fitmconst	—	Flag indicating whether or not M_0 is to be varied in the fit.
correctlc	—	Flag indicating whether or not to subtract the best-fit model from the light curve before passing it on to the next command.
omodel	—	Flag indicating whether or not to output the best-fit model light curve, evaluated at the input times of observation, to a file. The model will be written to a file named \$model_outdir/\$basename.softenedtransit.model where \$basename is the base filename of the input light curve file, stripped of directories.
fit_harm	—	Flag indicating whether or not to simultaneously fit a harmonic series to the light curve together with the transit model.
“aov”	—	Use this keyword to take the period for the harmonic series from the most recent -aov or -aov_harm command.
“ls”	—	Use this keyword to take the period for the harmonic series from the most recent -LS command.
“bls”	—	Use this keyword to take the period for the harmonic series from the most recent -BLS command.

(continued on next page)

Table A.39 (continued)

"list"	—	Use this keyword to take the period for the harmonic series from the input light curve list file (optionally specify the column to use).
"fix"	—	Use this keyword to fix the period for the harmonic series to the value specified by 'Pharm'.
nharm	—	The number of harmonics to use.
nsubharm	—	The number of sub-harmonics to use.

—Starspot

```

2  <"aov" | "ls" | "list" ["column" col] | "fix" period |
   "fixcolumn" <colname | colnum>>
4  a0 b0 alpha0 i0 chi0 psi00 mconst0 fitP fita fitb
   fitalpha fiti fitchi fitpsi fitmconst correctlc omodel [model_outdir]

```

Table A.40

Input parameters for "—Starspot" command (Section 2.2.3.8).

"aov"	—	Take the rotation period for the model from the most recent -aov or -aov_harm command.
"ls"	—	Take the rotation period for the model from the most recent -LS command.
"list"	—	Take the rotation period for the model from the input light curve list (optionally specifying the column to use).
"fix"	—	Fix the rotation period for the model to the value specified by 'period'.
"fixcolumn"	—	Set the rotation period for the model to the value from a previously executed command.
a0	—	The initial value to use for a in Eq. (75).
b0	—	The initial value to use for b in Eq. (75).
alpha0	—	The initial value to use for the spot angular radius in degrees.
i0	—	The initial value to use for the inclination of the stellar rotation axis in degrees (90° corresponds to the rotation axis being perpendicular to the line-of-sight).
chi0	—	The initial value to use for the spot latitude in degrees (0° for a spot at the equator).
psi00	—	The initial value to use for the longitude of the spot center (at the first time instance in the light curve) in degrees.
mconst0	—	The initial value to use for the constant magnitude term (M_0 in Eq. (75)). Set this to a negative value to have it determined automatically.
fitP	—	Flag indicating whether or not the rotation period should be varied in the fit.
fita	—	Flag indicating whether or not a should be varied in the fit.
fitb	—	Flag indicating whether or not b should be varied in the fit.
fitalpha	—	Flag indicating whether or not the spot angular radius should be varied in the fit.
fiti	—	Flag indicating whether or not the inclination of the stellar rotation axis should be varied in the fit.
fitchi	—	Flag indicating whether or not the spot latitude should be varied in the fit.
fitpsi	—	Flag indicating whether or not the spot longitude should be varied in the fit.
fitmconst	—	Flag indicating whether or not the constant magnitude should be varied in the fit.
correctlc	—	Flag indicating whether or not the best-fit model should be subtracted from the light curve before passing it on to the next command.
omodel	—	Flag indicating whether or not to write the best-fit model light curve, evaluated at the times in the input light curve, to a file.
model_outdir	—	This parameter should be given if and only if omodel is set to 1. The filename for the output model light curve will be \$model_outdir/\$basename.starspot.model where \$basename is the base filename of the input light curve file, stripped of directories.

```

1  —stats var1,var2,... stats1,stats2,...

```

Table A.41

Input parameters for "—stats" command (Section 2.2.2.8).

var1,var2, ...	—	A comma-separated list of variable names to compute the statistics on.
stats1,stats2, ...	—	A comma-separated list of one or more statistics to compute for each variable (every statistic is computed for every variable). Available statistics include:
mean	—	
weightedmean	—	Mean of the vector weighted using the light curve uncertainties.
median	—	
wmedian	—	Median of the vector weighted using the light curve uncertainties.
stddev	—	Standard deviation calculated with respect to the mean.
meddev	—	Standard deviation calculated with respect to the median.
medmeddev	—	Median of the absolute deviations from the median.

(continued on next page)

Table A.41 (continued)

MAD	—	$1.483 \times \text{medmeddev}$. For a Gaussian distribution this equals the standard deviation in the limit of large N .
kurtosis	—	
skewness	—	
pct%f	—	%f percentile, where %f is a floating point number between 0 and 100. Here 0 corresponds to the minimum value and 100 to the maximum value in the vector.
wpct%f	—	Percentile using the light curve uncertainties as weights.
max	—	Maximum value, equivalent to pct100.
min	—	Minimum value, equivalent to pct0.
sum	—	Sum of all elements in the vector.

```

1 -SYSREM Ninput_color ["column" col1] Ninput_airmass initial_airmass_file
   sigma_clip1 sigma_clip2 saturation correctlc omodel [model_outdir]
3   otrends [trend_outfile] useweights

```

Table A.42

Input parameters for “-SYSREM” command (Section 2.2.4.4).

Ninput_color	—	The number of trend vectors to remove for which the color terms are to be specified initially. These will be read-in from the input light curve list file, by default starting from the next unused column. The terms must be in Ninput_color consecutive columns. You can change the column to use for the first term with the “column” keyword.
Ninput_airmass	—	The number of trend vectors to remove for which the airmass terms are to be specified initially.
initial_airmass_file	—	A file with the initial airmass trends to use. The first column in this file should be the JDs (or image IDs if the -matchstringid option was given), and the subsequent Ninput_airmass columns are the initial airmass trends.
sigma_clip1	—	σ -clipping factor used in calculating the mean magnitudes of the light curves.
sigma_clip2	—	σ -clipping factor used in determining whether or not points contribute to the airmass or color terms when performing the fit.
saturation	—	Any points with magnitude less than ‘saturation’ will not contribute to the fit.
correctlc	—	Flag indicating whether or not the best-fit trend model should be subtracted from the light curve before passing it on to the next command.
omodel	—	Flag indicating whether or not the model light curve files, evaluated at the observed times in the input light curve, should be output.
model_outdir	—	This parameter should be given if and only if omodel is set to 1. The output model light curves will be written to files named \$model_outdir/\$basename.sysrem.model where \$basename is the base filename of the input light curve, stripped of leading directories.
otrends	—	Flag indicating whether or not the final trends should be output to a file.
trend_outfile	—	The name of the file to write the trends to. This parameter should be given if and only if otrends is set to 1. The output file will have JD (or image ID) in the first column, and the subsequent columns are for each trend signal.
useweights	—	Flag indicating whether or not the light curve uncertainties are used in performing the various fits.

```

1 -TFA trendlist ["readformat" Nskip jdcol magcol]
   dates_file pixelsep ["xycol" xcol ycol]
3   correctlc ocoeff [coeff_outdir] omodel [model_outdir]

```

Table A.43

Input parameters for “-TFA” command (Section 2.2.4.5).

trendlist	—	The name of the file containing the list of light curves to be used as trends. The first column in the file should contain the names of the light curve files, the second and third columns should be the X and Y coordinates of the stars.
“readformat”	—	An optional keyword used to change the format assumed when reading in the trend light curves.
Nskip	—	The number of lines to skip from the top of the trend light curve files. The default is 0. Note that any line beginning with a ‘#’ is automatically skipped, and not counted in this number.
jdcol	—	The column storing the times in the trend light curve files. Note that if the -matchstringid option is used with VARTOOLS, then this should be the column with the image IDs. The default value is 1.
magcol	—	The column storing the magnitudes in the trend light curve files. The default is 2.
dates_file	—	The name of a file providing a full list of times in the trend light curves or a full list of image IDs (if the -matchstringid option is used). Times should be in the second column of this file, image IDs in the first column.
pixelsep	—	Trend stars within pixelsep of the light curve in question will not be used in detrending the light curve.
“xycol”	—	Optional keyword to change the columns in the input light curve list file used to read-in the X and Y coordinates of the light curves. By default these are taken from the next unused columns.
correctlc	—	Flag indicating whether or not the best-fit trend model should be subtracted from the light curve before passing it on to the next command.

(continued on next page)

Table A.43 (continued)

ocoeff	—	Flag indicating whether or not the trend coefficients should be output for each light curve.
coeff_outdir	—	The trend coefficients will be written to files named \$coeff_outdir/\$basename.tfa.coeff where \$basename is the base filename of the input light curve, stripped of directories. This parameter should be given if and only if ocoeff is set to 1.
omodel	—	Flag indicating whether or not to output the best-fit model trends, evaluated at the times in the input light curves.
model_outdir	—	The trend models will be written to files named \$model_outdir/\$basename.tfa.model where \$basename is the base filename of the input light curve, stripped of directories. This parameter should be given if and only if omodel is set to 1.

```

1 -TFA_SR trendlist ["readformat" Nskip jdcol magcol] dates_file
  ["decorr" iterativeflag Nlterms lccolumn1 lccorder1 ...] pixelsep
3 ["xycol" colx coly]
  correctlc ocoeff [coeff_outdir] omodel [model_outdir] dotfafirst
5 tfathresh maxiter <"bin" nbins ["period" <"aov" | "ls"
  | "bls" | "list" ["column" col] | "fix" period>]
7 | "signal" filename
  | "harm" Nharm Nsubharm ["period" <"aov" | "ls"
9 | "bls" | "list" ["column" col] | "fix" period>]>

```

Table A.44

Input parameters for “-TFA_SR” command (Section 2.2.4.6).

trendlist	—	The name of the file containing the list of light curves to be used as trends. The first column in the file should contain the names of the light curve files, the second and third columns should be the X and Y coordinates of the stars.
“readformat”	—	An optional keyword used to change the format assumed when reading in the trend light curves.
Nskip	—	The number of lines to skip from the top of the trend light curve files. The default is 0. Note that any line beginning with a ‘#’ is automatically skipped, and not counted in this number.
jdcol	—	The column storing the times in the trend light curve files. Note that if the -matchstringid option is used with VARTOOLS, then this should be the column with the image IDs. The default value is 1.
magcol	—	The column storing the magnitudes in the trend light curve files. The default is 2.
dates_file	—	The name of a file providing a full list of times in the trend light curves or a full list of image IDs (if the -matchstringid option is used). Times should be in the second column of this file, image IDs in the first column.
“decorr”	—	If this optional keyword is used, the light curve will be simultaneously decorrelated against additional light-curve-specific signals.
iterativeflag	—	Flag indicating whether the decorrelation and TFA will be done iteratively (flag set to 1; this is faster) or if they will be done simultaneously (flag set to 0; slower but more correct).
Nlterms	—	The number of decorrelation trends to use.
lccolumn1	—	The column in the input light curve to use for the first light-curve-specific trend to decorrelate against.
lccorder1	—	The polynomial order to use for the first trend.
...	—	The columns and orders should then be provided for the rest of the Nlterms trends.
pixelsep	—	Trend stars within pixelsep of the light curve in question will not be used in detrending the light curve.
“xycol”	—	Optional keyword to change the columns in the input light curve list file used to read-in the X and Y coordinates of the light curves. By default these are taken from the next unused columns.
correctlc	—	Flag indicating whether or not the best-fit trend model should be subtracted from the light curve before passing it on to the next command.
ocoeff	—	Flag indicating whether or not the trend coefficients should be output for each light curve.
coeff_outdir	—	The trend coefficients will be written to files named \$coeff_outdir/\$basename.tfa.coeff where \$basename is the base filename of the input light curve, stripped of directories. This parameter should be given if and only if ocoeff is set to 1.
omodel	—	Flag indicating whether or not to output the best-fit model trends, evaluated at the times in the input light curves.
model_outdir	—	The trend models will be written to files named \$model_outdir/\$basename.tfa.model where \$basename is the base filename of the input light curve, stripped of directories. This parameter should be given if and only if omodel is set to 1.
dotfafirst	—	Flag indicating whether TFA should be applied to the input light curve first, with the signal to preserve determined on the residual in each iteration (set the flag to 1), or if the signal is determined and subtracted from the light curve first and then TFA is applied to the residual in each iteration (set the flag to 0).
tfathresh	—	The iterations will stop if the fractional change in the r.m.s. is less than ‘tfathresh’.
maxiter	—	The iterations will stop once ‘maxiter’ iterations have completed.
“bin”	—	Use the binned light curve for the model signal to preserve.
nbins	—	The number of bins to use.
“period”	—	Optional keyword to do phase-binning, rather than binning in time.
“aov”	—	Take the period for phase-binning from the most recently executed -aov or -aov_harm command.
“ls”	—	Take the period for phase-binning from the most recently executed -LS command.
“bls”	—	Take the period for phase-binning from the most recently executed -BLS command.
“list”	—	Take the period for phase-binning from the input light curve list file (optionally specifying the column).
“fix”	—	Fix the period for phase-binning to the value specified by ‘period’.

(continued on next page)

Table A.44 (continued)

"signal"	–	Use a fixed signal form read-in from a file. Use 'filename' to specify a file listing the signal files, one for each light curve to process. Each signal file should contain the signal in the second column. The quantity $a \times S + b$ is fit to the light curve simultaneously with TFA, here a and b are free parameters and S is the signal.
"harm"	–	Use a harmonic series to model the signal. This will be fit simultaneously to the light curve with TFA.
Nharm	–	The number of harmonics to use.
Nsubharm	–	The number of sub-harmonics to use.
"period"	–	Use this keyword to change the period for the harmonic series (the default is the time spanned by the light curve).
"aov"	–	Use this keyword to take the period from the most recent -aov or -aov_harm command.
"ls"	–	Use this keyword to take the period from the most recent -LS command.
"bls"	–	Use this keyword to take the period from the most recent -BLS command.
"list"	–	Read the period from the light curve list file (optionally specify the column).
"fix"	–	Fix the period to a specified value.

```

1  -wwz <"maxfreq" <"auto" | maxfreq>> <"freqsamp" freqsamp>
    <"tau0" <"auto" | tau0>> <"tau1" <"auto" | tau1>>
3  <"dtau" <"auto" | dtau>> ["c" cval]
    ["outfulltransform" outdir ["fits" | "pm3d"]] ["format" format]]
5  ["outmaxtransform" outdir ["format" format]]

```

Table A.45

Input parameters for "-wwz" command (Section 2.2.1.9).

"maxfreq"	–	Indicate the maximum frequency for which the wavelet transform will be calculated.
"auto"	–	If this keyword is given then the maximum frequency will be $1/(2\Delta t_{\min})$ where $\Delta t_{\min}$ is the minimum time separation between consecutive points in the light curve.
maxfreq	–	The maximum frequency to use.
"freqsamp"	–	Specify the frequency sampling as $\text{freqsamp}/T$ where T is the time baseline of the light curve.
"tau0"	–	Indicate the minimum time-shift to test (the time-shift is τ in Eq. (17)). Use the "auto" keyword to set this to the minimum time in the light curve, otherwise provide a value.
"tau1"	–	Indicate the maximum time-shift to test. Use the "auto" keyword to set this to the maximum time in the light curve, otherwise provide a value.
"dtau"	–	Indicate the time-shift step to use. Use the "auto" keyword to set this to $\Delta t_{\min}$, otherwise provide a value.
"c"	–	Optionally provide a value for the c parameter in Eq. (18). The default value is $c = (8\pi^2)^{-1}$.
"outfulltransform"	–	Use this keyword to output the full wavelet transform for each light curve (i.e., calculated at every trial time-shift and frequency).
outdir	–	The directory to output the transforms to. The default naming convention is \$outdir/\$basename.wwz where \$basename is the base filename of the input light curve, stripped of directories.
"fits"	–	Optionally output the transforms as multi-extension FITS image files.
"pm3d"	–	Optionally output the transforms as ascii tables in a format suitable for plotting with the gnuplot pm3d plotting style.
"format"	–	Modify the naming convention for the files. In this case the file will be named \$outdir/\$format where instances of %s in \$format are replaced by \$basename, instances of %d are replaced by the light curve number (starting with 1), instances of %0nd, where n is an integer, are replaced with the formatted light curve number, and instances of %% are replaced with %.
"outmaxtransform"	–	Use this keyword to output the transform that maximizes Z (Eq. (28)) over frequencies as a function of time-shift.
outdir	–	The directory to output these files to. The default naming convention is \$outdir/\$basename.mwwz where \$basename is the base filename of the input light curve, stripped of directories.
"format"	–	Modify the naming convention for the files. In this case the file will be named \$outdir/\$format where instances of %s in \$format are replaced by \$basename, instances of %d are replaced by the light curve number (starting with 1), instances of %0nd, where n is an integer, are replaced with the formatted light curve number, and instances of %% are replaced with %.

Appendix B. Columns output by each command

The tables below list the columns added by each command to the output table. The columns will be named \$column_"\$basecolumnname_"\$commandnum where \$column_"_" is prepended only if the **-numbercolumns** option is given to VARTOOLS, \$column is the column number (starting at 1), \$basecolumnname is as listed below, and \$commandnum is the number of the command that this column was produced by (i.e., if one runs "vartools -i input.txt -rms -chi2 -rms" the columns produced by the first "-rms" call would have \$commandnum = 1, those produced by the "-chi2" command would have \$commandnum = 2, and those produced by the second "-rms" call would have \$commandnum = 3). When using a "fixcolumn" option to one of the VARTOOLS commands to set a parameter equal to the value of the command, one can either provide the column number (\$column), or the column name without the prepended column number (\$basecolumnname_"_"\$commandnum).

Table B.46

Output columns for “-addnoise” command (Section 2.2.5.1).

Addnoise_Gamma_1	—	The value used for γ , as discussed in Section 2.2.5. This is only output if the “wavelet” noise model is used, and if “list” is used to initialize γ .
Addnoise_Sig_Red_1	—	The value used for the red-noise component standard deviation. This is only output if “list” is used to initialize red-noise standard deviation.
Addnoise_Sig_White_1	—	The value used for the white-noise component standard deviation. This is only output if “list” is used to initialize the white noise standard deviation.
Addnoise_Rho_1	—	The value used for ρ , as discussed in Section 2.2.5. This is only output if the “squareexp”, “exp” or “matern” noise model is used, and if “list” is used to initialize ρ .
Addnoise_Nu_1	—	The value used for ν , as discussed in Section 2.2.5. This is only output if the “matern” noise model is used, and if “list” is used to initialize ν .

Table B.47

Output columns for “-alarm” command (Section 2.2.2.1).

Alarm	—	The computed alarm statistic.
-------	---	-------------------------------

Table B.48

Output columns for “-aov” command (Section 2.2.1.4).

Period_\$peaknum	—	The period for peak number \$peaknum. \$peaknum runs from 1 for the highest peak in the periodogram to ‘Npeak’, the number of peaks to identify as given on the command line. Note that all columns for a given peak are output together, and then the same quantities are repeated for the next peak, until all ‘Npeak’ peaks have been reported.
AOV_LOGSNR_\$peaknum	—	The spectroscopic S/N measured on the logarithmic AoV periodogram for peak \$peaknum. This is output only if the “uselog” keyword was given to the -aov command.
AOV_\$peaknum	—	The value of the AoV periodogram for peak \$peaknum. This is output only if the “uselog” keyword was not given to the -aov command.
AOV_SNR_\$peaknum	—	The spectroscopic S/N measured on the AoV periodogram for peak \$peaknum. This is output only if the “uselog” keyword was not given to the -aov command.
AOV_NEG_LN_FAP_\$peaknum	—	The negative natural logarithm of the formal false alarm probability for peak \$peaknum. This is output only if the “uselog” keyword was not given to the -aov command.
Mean_InAOV_\$peaknum	—	The mean value of the logarithmic AoV periodogram at the whitening cycle used to find peak \$peaknum. This is output only if the “uselog” and the “whiten” keywords were both given to the -aov command.
RMS_InAOV_\$peaknum	—	The standard deviation of the logarithmic AoV periodogram at the whitening cycle used to find peak \$peaknum. This is output only if the “uselog” and the “whiten” keywords were both given to the -aov command.
PeriodFix	—	The fixed period at which to report the periodogram properties. This is output only if the “fixperiodSNR” keyword was given to the -aov command.
AOV_LOGSNR_PeriodFix	—	The spectroscopic S/N measured on the logarithmic AoV periodogram for the fixed period. This is output only if the “fixperiodSNR” and the “uselog” keywords were both given to the -aov command.
AOV_PeriodFix	—	The value of the AoV periodogram at the fixed period. This is output only if the “fixperiodSNR” keyword is given and the “uselog” keyword is not given.
AOV_SNR_PeriodFix	—	The spectroscopic S/N of the AoV periodogram at the fixed period. This is output only if the “fixperiodSNR” keyword is given and the “uselog” keyword is not given.
AOV_NEG_LN_FAP_PeriodFix	—	The negative natural logarithm of the formal false alarm probability at the fixed period. This is output only if the “fixperiodSNR” keyword is given and the “uselog” keyword is not given.
Mean_InAOV	—	The mean value of the logarithmic AoV periodogram. This is output only if the “uselog” keyword is given and the “whiten” keyword is not given.
RMS_InAOV	—	The standard deviation of the logarithmic AoV periodogram. This is output only if the “uselog” keyword is given and the “whiten” keyword is not given.

Table B.49

Output columns for “-aov_harm” command (Section 2.2.1.5).

Period_\$peaknum	– The period for peak number \$peaknum. \$peaknum runs from 1 for the highest peak in the periodogram to ‘Npeak’, the number of peaks to identify as given on the command line. Note that all columns for a given peak are output together, and then the same quantities are repeated for the next peak, until all ‘Npeak’ peaks have been reported.
AOV_HARM_\$peaknum	– The value of the AoV periodogram for peak \$peaknum. This is output only if a positive number was specified for the ‘Nharm’ parameter input to the -aov_harm command.
AOV_HARM_NEG_LOG_FAP_\$peaknum	– The negative logarithm of the formal false alarm probability for peak \$peaknum. This is output here only if a value less than or equal to zero was specified for the ‘Nharm’ parameter input to the -aov_harm command.
AOV_HARM_NHARM_\$peaknum	– The optimum number of harmonics used for peak \$peaknum. This is output here only if a value less than or equal to zero was specified for the ‘Nharm’ parameter input to the -aov_harm command.
AOV_HARM_SNR_\$peaknum	– The spectroscopic S/N measured on the AoV periodogram for peak number \$peaknum.
AOV_HARM_NEG_LOG_FAP_\$peaknum	– The negative logarithm of the formal false alarm probability for peak \$peaknum. This is output here only if a positive number was specified for the ‘Nharm’ parameter input to the -aov_harm command.
Mean_AOV_HARM_\$peaknum	– The mean value of the AoV periodogram for the whitening cycle in which peak \$peaknum was found. This is only output if the “whiten” keyword was given.
RMS_AOV_HARM_\$peaknum	– The standard deviation of the AoV periodogram for the whitening cycle in which peak \$peaknum was found. This is only output if the “whiten” keyword was given.
PeriodFix	– The fixed period at which to report the periodogram properties. This is output only if the “fixperiodSNR” keyword was given to the -aov_harm command.
AOV_HARM_PeriodFix	– The value of the AoV periodogram at the fixed period. This is output only if the “fixperiodSNR” keyword is given.
AOV_HARM_SNR_PeriodFix	– The spectroscopic S/N of the AoV periodogram at the fixed period. This is output only if the “fixperiodSNR” keyword is given.
AOV_HARM_NEG_LN_FAP_PeriodFix	– The negative logarithm of the formal false alarm probability for the fixed period. This is output here only if the “fixperiodSNR” keyword is given, and if a positive value was specified for the ‘Nharm’ parameter input to the -aov_harm command.
Mean_AOV_HARM	– The mean of the AoV periodogram. This is output only if the “whiten” keyword was not provided.
RMS_AOV_HARM	– The standard deviation of the AoV periodogram. This is output only if the “whiten” keyword was not provided.

Table B.50

Output columns for “-autocorrelation” command (Section 2.2.2.2).

No columns added to the output table.

Table B.51

Output columns for “-binlc” command (Section 2.2.6.1).

No columns added to the output table.

Table B.52

Output columns for “-BLS” command (Section 2.2.1.6).

BLS_Period_\$peaknum	– The period for peak number \$peaknum. \$peaknum runs from 1 for the highest peak in the periodogram to ‘Npeak’, the number of peaks to identify as given on the command line. Note that all columns for a given peak are output together, and then the same quantities are repeated for the next peak, until all ‘Npeak’ peaks have been reported.
BLS_Tc_\$peaknum	– The time of central transit for peak number \$peaknum.
BLS_SN_\$peaknum	– The spectroscopic S/N estimate for peak number \$peaknum calculated following Eq. (14). If the “nobinnedrms” keyword is given, then this is instead equal to the SDE (Eq. (12)), except that a 3σ clipping is first applied to the BLS spectrum before computing the average value and standard deviation to use in that equation. The BLS peaks are sorted by this quantity.
BLS_SR_\$peaknum	– The signal residue for peak number \$peaknum (Eq. (10)).
BLS_SDE_\$peaknum	– The signal detection efficiency as defined by Kovács et al. (2002) for peak number \$peaknum.
BLS_Depth_\$peaknum	– The transit depth in magnitudes (or whatever units are used for the input light curve) for peak number \$peaknum.
BLS_Qtran_\$peaknum	– The fractional transit duration (duration divided by orbital period) for peak number \$peaknum.
BLS_Qingress_\$peaknum	– The fractional ingress duration (duration of ingress divided by total transit duration) for peak number \$peaknum. This is output only if the “fittrap” keyword was used.
BLS_OOTmag_\$peaknum	– The out-of-transit magnitude level for peak number \$peaknum. This is output here only if the “fittrap” keyword was used.
BLS_i1_\$peaknum	– The phase of transit ingress for peak number \$peaknum when time $t = 0$ is used as a reference time for zero phase.
BLS_i2_\$peaknum	– The phase of transit egress for peak number \$peaknum when time $t = 0$ is used as a reference time for zero phase.
BLS_deltaChi2_\$peaknum	– The value of $\Delta\chi^2 = \chi_0^2 - \chi_{\text{tr}}^2$ where χ_0^2 is for a model with no variation, and χ_{tr}^2 is for the transit model found for peak number \$peaknum.
BLS_fraconenight_\$peaknum	– The fraction of $\Delta\chi^2$ that comes from a single night for peak number \$peaknum.
BLS_Npointsintransit_\$peaknum	– The number of points in transit for peak number \$peaknum.
BLS_Ntransits_\$peaknum	– The number of distinct transits with at least one point in transit for peak number \$peaknum.
BLS_Npointsbeforetransit_\$peaknum	– The number of points in a phase bin of equal size to the transit and immediately preceding the transit, for peak number \$peaknum.
BLS_Npointsaftertransit_\$peaknum	– The number of points in a phase bin of equal size to the transit and immediately following the transit, for peak number \$peaknum.
BLS_Rednoise_\$peaknum	– An estimate of the red-noise of the light curve at the time-scale of a transit for peak number \$peaknum. This is calculated following Eq. (16).
BLS_Whitenoise_\$peaknum	– An estimate of the white-noise of the light curve for peak number \$peaknum, calculated as the standard deviation of the light curve after subtracting the transit.
BLS_SignaltoPinknoise_\$peaknum	– An estimate of the S/N of the transit accounting for both red and white noise for peak number \$peaknum. This is calculated following Eq. (15).
BLS_Period_invtransit	– Period for the highest S/N inverse transit (periodic, box-shaped brightening rather than dimming) found in the light curve.
BLS_deltaChi2_invtransit	– The value of $\Delta\chi^2 = \chi_0^2 - \chi_{\text{invtr}}^2$ where χ_0^2 is for a model with no variation, and χ_{invtr}^2 is the best-fit inverse transit model found.
BLS_MeanMag	– The mean magnitude of the light curve which is subtracted before applying the BLS algorithm.

Table B.53

Output columns for “-BLSFixPer” command (Section 2.2.1.8).

BLSFixPer_Period	–	The period at which the BLS quantities are calculated.
BLSFixPer_Tc	–	The time of transit center.
BLSFixPer_SR	–	The signal residue at this period (Eq. (10)).
BLSFixPer_Depth	–	The transit depth in magnitudes (or whatever units are used for the input light curve).
BLSFixPer_Qtran	–	The fractional transit duration (duration divided by orbital period).
BLSFixPer_Qingress	–	The fractional ingress duration (duration of ingress divided by total transit duration). This is output only if the “fittrap” keyword was used.
BLSFixPer_OOTmag	–	The out-of-transit magnitude level. This is output here only if the “fittrap” keyword was used.
BLSFixPer_i1	–	The phase of transit ingress when time $t = 0$ is used as a reference time for zero phase.
BLSFixPer_i2	–	The phase of transit egress when time $t = 0$ is used as a reference time for zero phase.
BLSFixPer_deltaChi2	–	The value of $\Delta\chi^2 = \chi_0^2 - \chi_{tr}^2$ where χ_0^2 is for a model with no variation, and χ_{tr}^2 is for the transit model.
BLSFixPer_fraconenight	–	The fraction of $\Delta\chi^2$ that comes from a single night.
BLSFixPer_Npointsintransit	–	The number of points in transit.
BLSFixPer_Ntransits	–	The number of distinct transits with at least one point in transit.
BLSFixPer_Npointsbeforetransit	–	The number of points in a phase bin of equal size to the transit and immediately preceding the transit.
BLSFixPer_Npointsaftertransit	–	The number of points in a phase bin of equal size to the transit and immediately following the transit.
BLSFixPer_Rednoise	–	An estimate of the red-noise of the light curve at the time-scale of a transit. This is calculated following Eq. (16).
BLSFixPer_Whitenoise	–	An estimate of the white-noise of the light curve, calculated as the standard deviation of the light curve after subtracting the transit.
BLSFixPer_SignaltoPinknoise	–	An estimate of the S/N of the transit accounting for both red and white noise. This is calculated following Eq. (15).
BLSFixPer_deltaChi2_invtransit	–	The value of $\Delta\chi^2 = \chi_0^2 - \chi_{invtr}^2$ where χ_0^2 is for a model with no variation, and χ_{invtr}^2 is the best-fit inverse transit model found at the fixed period.
BLSFixPer_MeanMag	–	The mean magnitude of the light curve which is subtracted before calculating the BLS statistics.

Table B.54

Output columns for “-BLSFixDurTc” command (Section 2.2.1.7).

BLSFixDurTc_Duration	—	The fixed transit duration adopted for this command.
BLSFixDurTc_Tc	—	The fixed time of transit center adopted for this command.
BLSFixDurTc_Depth	—	The fixed transit depth adopted for this command. This is output here only if the “fixdepth” keyword was used.
BLSFixDurTc_Qingress	—	The fixed fractional ingress duration (duration of ingress divided by total transit duration) adopted for this command. This is output only if the “fixdepth” keyword was used. If the “qgress” keyword was not given, this will have a value of 0.
BLSFixDurTc_Period_\$peaknum	—	The period for peak number \$peaknum. \$peaknum runs from 1 for the highest S/N peak in the periodogram to ‘Npeak’, the number of peaks to identify as given on the command line. Note that all columns for a given peak are output together, and then the same quantities are repeated for the next peak, until all ‘Npeak’ peaks have been reported.
BLSFixDurTc_SN_\$peaknum	—	The spectroscopic S/N estimate for peak number \$peaknum equal to the SDE (Eq. (12)), except that a 3σ clipping is first applied to the BLS spectrum before computing the average value and standard deviation to use in that equation. The BLS peaks are sorted by this quantity.
BLSFixDurTc_SR_\$peaknum	—	The Signal Residue for peak \$peaknum (Eq. (10)).
BLSFixDurTc_SDE_\$peaknum	—	The Signal Detection Efficiency for peak \$peaknum (Eq. (12)).
BLSFixDurTc_Depth_\$peaknum	—	The transit depth in magnitudes (or whatever units are used for the input light curve) for peak number \$peaknum.
BLSFixDurTc_Qtran_\$peaknum	—	The fractional transit duration (duration divided by orbital period) for peak number \$peaknum.
BLSFixDurTc_Qingress_\$peaknum	—	The fractional ingress duration (duration of ingress divided by total transit duration) for peak number \$peaknum. This is output only if the “fittrap” keyword was used and the “fixdepth” keyword was not used.
BLSFixDurTc_OOTmag_\$peaknum	—	The out-of-transit magnitude level for peak number \$peaknum. This is output here only if the “fittrap” keyword was used and the “fixdepth” keyword was not used.
BLSFixDurTc_deltaChi2	—	The value of $\Delta\chi^2 = \chi_0^2 - \chi_{tr}^2$ for peak number \$peaknum where χ_0^2 is for a model with no variation, and χ_{tr}^2 is for the transit model.
BLSFixDurTc_fraconenight_\$peaknum	—	The fraction of $\Delta\chi^2$ that comes from a single night for peak number \$peaknum.
BLSFixDurTc_Npointsintransit_\$peaknum	—	The number of points in transit for peak number \$peaknum.
BLSFixDurTc_Ntransits_\$peaknum	—	The number of distinct transits with at least one point in transit for peak number \$peaknum.
BLSFixDurTc_Npointsbeforetransit_\$peaknum	—	The number of points in a phase bin of equal size to the transit and immediately preceding the transit for peak number \$peaknum.
BLSFixDurTc_Npointsaftertransit_\$peaknum	—	The number of points in a phase bin of equal size to the transit and immediately following the transit for peak number \$peaknum.
BLSFixDurTc_Rednoise_\$peaknum	—	An estimate of the red-noise of the light curve at the time-scale of a transit for peak number \$peaknum. This is calculated following Eq. (16).
BLSFixDurTc_Whitenoise_\$peaknum	—	An estimate of the white-noise of the light curve for peak number \$peaknum, calculated as the standard deviation of the light curve after subtracting the transit.
BLSFixDurTc_SignaltoPinknoise_\$peaknum	—	An estimate of the S/N of the transit for peak number \$peaknum accounting for both red and white noise. This is calculated following Eq. (15).
BLSFixDurTc_Period_invtransit	—	Period for the highest S/N inverse transit (periodic, box-shaped brightening rather than dimming) found with the same duration, T_c and other constraints.
BLSFixDurTc_deltaChi2_invtransit	—	The value of $\Delta\chi^2 = \chi_0^2 - \chi_{invtr}^2$ where χ_0^2 is for a model with no variation, and χ_{invtr}^2 is for the inverse transit model.
BLSFixDurTc_MeanMag	—	The mean magnitude of the light curve which is subtracted before calculating the BLS statistics.

Table B.55

Output columns for “-changeerror” command (Section 2.2.6.2).

Mean_Mag	—	The mean of the light curve magnitudes.
RMS	—	The r.m.s. scatter of the light curve magnitudes. All light curve uncertainties (including those with input uncertainties $\sigma \leq 0$) are set equal to this quantity by this command.
Npoints	—	The number of points in the light curve which contributed to the r.m.s. scatter estimate. Points with input uncertainty $\sigma \leq 0$ and points with NaN magnitude values do not contribute to the scatter or mean estimates.

Table B.56

Output columns for “-changevariable” command (Section 2.2.7.1).

No columns added to the output table.

Table B.57

Output columns for “-chi2” command (Section 2.2.2.3).

Chi2	—	The value of $\chi_0^2/\text{d.o.f.}$, the reduced χ^2 of the light curve magnitudes.
Weighted_Mean_Mag	—	The mean magnitude of the light curve, using the input uncertainties to determine the weights. This is the value used in calculating the magnitude residuals for computing χ^2 .

Table B.58

Output columns for “-chi2bin” command (Section 2.2.2.4).

Chi2Bin_\$bintime	—	The reduced χ^2 after binning the light curve with bintime \$bintime. The value of \$bintime is displayed to two decimal places. Note that both columns for a given \$bintime (Chi2Bin and Weighted_Mean_Mag) are output together, and then the same quantities are repeated for the next \$bintime, until the results for all 'Nbin' values of \$bintime given on the command line have been reported.
Weighted_Mean_Mag_Bin_\$bintime	—	The mean magnitude of the light curve, using the input uncertainties to determine the weights, after binning with by \$bintime.

Table B.59

Output columns for “-clip” command (Section 2.2.4.1).

Nclip	—	The number of points clipped from the light curve.
-------	---	--

Table B.60

Output columns for “-converttime” command (Section 2.2.6.3).

No columns added to the output table.

Table B.61

Output columns for “-copylc” command (Section 2.2.5.2).

No columns added to the output table.

Appendix C. Analytic functions, constants, and operators

Below is the list of analytic functions, constants, and operators recognized by the VARTOOLS analytic expression interpreter. These strings are reserved and cannot be used for naming variables.

Operators		
$a + b$	—	Addition
$a - b$	—	Subtraction
$a * b$	—	Multiplication
a / b	—	Division
$a \% b$	—	Floating point remainder (fmod function in c)
$a \wedge b$	—	Exponentiation
$a > b$	—	Greater than comparison
$a >= b$	—	Greater than or equal to comparison
$a < b$	—	Less than comparison
$a <= b$	—	Less than or equal to comparison
$a == b$	—	Logical equals
$a != b$	—	Logical not equal
$a \&\& b$	—	Logical “and” comparison
$a b$	—	Logical “or” comparison
$!a$	—	Logical “not”
Functions		
$\exp(x)$	—	Exponential of x .
$\log(x)$	—	Natural logarithm of x .
$\log 10(x)$	—	Base 10 logarithm of x .
$\sqrt{x}$	—	Square root of x .
$\text{abs}(x)$	—	Absolute value of x .
$\max(x, y)$	—	The larger of x or y .
$\min(x, y)$	—	The smaller of x or y .
$\text{hypot}(x, y)$	—	$\sqrt{x^2 + y^2}$.
$\sin(x)$	—	Trigonometric sine of x . Input in radians.
$\cos(x)$	—	Trigonometric cosine of x . Input in radians.
$\tan(x)$	—	Trigonometric tangent of x . Input in radians.
$\text{sindegr}(x)$	—	Trigonometric sine of x . Input in degrees.
$\text{cosdegr}(x)$	—	Trigonometric cosine of x . Input in degrees.
$\text{tandegr}(x)$	—	Trigonometric tangent of x . Input in degrees.
$\text{asin}(x)$	—	Inverse sine of x . Output in radians.
$\text{acos}(x)$	—	Inverse cosine of x . Output in radians.
$\text{atan2}(y, x)$	—	4 quadrant inverse tangent of y/x . Output in radians.
$\text{asindegr}(x)$	—	Inverse sine of x . Output in degrees.
$\text{acosdegr}(x)$	—	Inverse cosine of x . Output in degrees.
$\text{atan2degr}(y, x)$	—	4 quadrant inverse tangent of y/x . Output in degrees.
$\sinh(x)$	—	Hyperbolic sine of x .
$\cosh(x)$	—	Hyperbolic cosine of x .
$\tanh(x)$	—	Hyperbolic tangent of x .
$\text{asinh}(x)$	—	Inverse hyperbolic sine of x .
$\text{acosh}(x)$	—	Inverse hyperbolic cosine of x .
$\text{atanh}(x)$	—	Inverse hyperbolic tangent of x .
$\text{erf}(x)$	—	Error function of x ****TBD denote convention***.
$\text{erfc}(x)$	—	Complimentary error function of x .
$\lgamma(x)$	—	Natural logarithm of $\Gamma(x)$.
$\gamma(x)$	—	$\Gamma(x)$.
$\theta(x)$	—	1 for $x \geq 0$, 0 for $x < 0$.
$\text{round}(x)$	—	Round x to the nearest integer.
$\text{ceil}(x)$	—	Smallest integer that is greater than or equal to x .
$\text{floor}(x)$	—	Largest integer that is less than or equal to x .
$\text{rand}()$	—	Random number drawn from a uniform distribution between 0 and 1.
$\text{gauss}()$	—	Random number drawn from a normal distribution with 0 mean and unit variance.
Constants		
π	—	Evaluates to the M_PI constant defined in the “math.h” c header file.
e	—	Evaluates to the M_E constant defined in the “math.h” c header file.
Special Variables:		
NR	—	Image index in the light curve starting from 0.
NF	—	Light curve index starting from 0.

Appendix D. Justification for the -ensemblerescalesig procedure

Here we justify the procedure used by the **-ensemblerescalesig** command to find parameters a and b for the transformation in Eq. (89) such that $\chi^2/\text{dof} = 1$ for the typical light curve after the transformation. The procedure is to solve the linear least squares problem in Eqs. (90)–(92) for a and b .

Table B.62

Output columns for “-decorr” command (Section 2.2.3.1).

Decorr_constant_term	–	The best-fit value for the zero-point term in the decorrelation model. This is output only if the ‘zeropointterm’ flag was set to 1.
Decorr_constant_term_err	–	The uncertainty on the zero-point term in the decorrelation model. This is output only if the ‘zeropointterm’ flag was set to 1. The uncertainties output from this command are correct only if the input magnitude uncertainties are accurate, and if the noise in the light curve is uncorrelated and follows a Gaussian distribution.
Global_\$i_coeff_\$j	–	The best-fit value of the coefficient for global parameter sequence number \$i (running from 1 to the value of ‘Nglobalterms’ given on the command line) raised to the power \$j (running from 1 to the polynomial ‘order’ specified for this sequence on the command line). A separate column is output for each combination of \$i and \$j, with the loop over \$j nested within the loop over \$i. The best-fit value, and uncertainty for a given \$i and \$j combination are output together.
Global_\$i_coeff_err_\$j	–	The uncertainty on Global_\$i_coeff_\$j.
LCColumn_\$lccolumn[\$i]_coeff_\$j	–	The best-fit value of the coefficient for the parameter sequence read from column \$lccolumn[\$i] in the input light curve (\$i runs from 1 to the value of ‘Nlcterm’ given on the command line, and \$lccolumn[\$i] is the value of ‘lccolumn’ given on the command line for term \$i) raised to the power \$j (running from 1 to the polynomial ‘order’ specified for this light curve column). A separate column is output for each combination of \$i and \$j, with the loop over \$j nested within the loop over \$i. The best-fit value, and uncertainty for a given \$i and \$j combination are output together.
LCColumn_\$lccolumn[\$i]_coeff_err_\$j	–	The uncertainty on LCColumn_\$lccolumn[\$i]_coeff_\$j.
Decorr_chi2	–	The χ^2 of the fit (not reduced).

Let

$$\chi_i^2/\text{dof} = \sum_{j=1}^{N_{\text{JD},i}} (m_{i,j} - \bar{m}_i)^2 / (N_{\text{JD},i} - 1) / \sigma_{i,j}^2 \quad (\text{D.1})$$

be the reduced χ^2 for light curve i after applying the transformation in Eq. (89). We want a and b such that

$$\chi_i^2/\text{dof} = 1 \quad (\text{D.2})$$

for the “typical” light curve. One way to do this is to perform a least-squares fit for a and b . In other words, to find the values of a and b which minimizes

$$X^2 = \sum_{i=1}^{N_{\text{LC}}} \frac{(\chi_i^2/\text{dof} - 1)^2}{2/\text{dof}} \quad (\text{D.3})$$

where the expected value of χ_i^2/dof is 1 and its variance is $2/\text{dof}$. However, substituting Eq. (89) directly into Eqs. (D.1) and (D.3) results in a non-linear least squares problem for a and b whose solution using standard non-linear least squares algorithms would be very time consuming due to the need to perform a sum over every observation in every light curve for each evaluation of X^2 . An alternative approach is to find a simple approximation which reduces this to a linear least-squares problem. The **-ensamplerescalesig** command makes the following approximation for $\sigma_{i,j}^2$ in Eq. (D.1):

$$\sigma_{i,j}^2 \approx \sigma_{i,j}^2 \frac{a\bar{\sigma}_i^2 + b}{\bar{\sigma}_i^2}. \quad (\text{D.4})$$

In other words, rather than using the transformed value of $\sigma_{i,j}^2$ directly, the individual uncertainty is scaled by the ratio of the expected variance after the transformation, to the expected variance before the transformation. Substituting this expression into Eq. (D.2), we then have

$$\sum_{j=1}^{N_{\text{JD},i}} \frac{(m_{i,j} - \bar{m}_i)^2 \bar{\sigma}_i^2}{(N_{\text{JD},i} - 1) \sigma_{i,j}^2 (a\bar{\sigma}_i^2 + b)} = 1 \quad (\text{D.5})$$

or

$$\sum_{j=1}^{N_{\text{JD},i}} \frac{(m_{i,j} - \bar{m}_i)^2 \bar{\sigma}_i^2}{(N_{\text{JD},i} - 1) \sigma_{i,j}^2} = a\bar{\sigma}_i^2 + b \quad (\text{D.6})$$

or

$$R\bar{M}S_i^2 \chi_i^2/\text{dof} = aR\bar{M}S_i^2 + b \quad (\text{D.7})$$

which may then be treated as a linear least squares problem for a and b (Eq. (90)).

Table B.63

Output columns for “-dftclean” command (Section 2.2.1.3).

DFTCLEAN_DSPEC_PEAK_FREQ_\$peaknum	— The frequency in cycles per day (assuming input times are in days) for peak number \$peaknum identified in the “dirty” DFT spectrum (i.e., the spectrum before application of the “CLEAN” deconvolution algorithm). \$peaknum runs from 1 for the highest peak in the spectrum to ‘Npeaks’, the number of peaks to identify in the “dirty” spectrum as given on the command line. Note that all columns for a given peak are output together, and then the same quantities are repeated for the next peak, until all ‘Npeaks’ peaks have been reported. These columns are only reported if the “finddirtypeaks” keyword is given.
DFTCLEAN_DSPEC_PEAK_POW_\$peaknum	— The value of the power spectrum at peak number \$peaknum. This is equal to $P(f) = F_r^2(f) + F_i^2(f)$ where F_r and F_i are the real and imaginary components, respectively, of $F(f)$ given in Eq. (7). If the “useampspec” keyword is given on the command line, then it is the square root of this quantity that is reported.
DFTCLEAN_DSPEC_PEAK_SNR_\$peaknum	— The spectroscopic S/N ratio for peak number \$peaknum. This equal to $(P(f) - \bar{P})/\sigma_P$, where $P(f)$ is the value of the power spectrum at peak number \$peaknum (or amplitude spectrum if the “useampspec” keyword is given), and $\bar{P}$ and σ_P are the average and standard deviation of the spectrum, respectively, after σ -clipping.
DFTCLEAN_DSPEC_AVESPEC	— The value of $\bar{P}$ used in computing the S/N ratio(s) for the dirty power spectrum. This is output only if the “verboseout” keyword is given, and if the “finddirtypeaks” keyword is given with ‘Npeaks’ greater than 0.
DFTCLEAN_DSPEC_STDSPEC	— The value of σ_P used in computing the S/N ratio(s) for the dirty power spectrum. This is output only if the “verboseout” keyword is given, and if the “finddirtypeaks” keyword is given with ‘Npeaks’ greater than 0.
DFTCLEAN_DSPEC_AVESPEC_NOCLIP	— The value of $\bar{P}$ for the dirty spectrum when no clipping is applied. This is output only if the “verboseout” keyword is given, and if the “finddirtypeaks” keyword is given with ‘Npeaks’ greater than 0.
DFTCLEAN_DSPEC_STDSPEC_NOCLIP	— The value of σ_P for the dirty spectrum when no clipping is applied. This is output only if the “verboseout” keyword is given, and if the “finddirtypeaks” keyword is given with ‘Npeaks’ greater than 0.
DFTCLEAN_CSPEC_PEAK_FREQ_\$peaknum	— The frequency in cycles per day (assuming input times are in days) for peak number \$peaknum identified in the “CLEAN” DFT spectrum. \$peaknum runs from 1 for the highest peak in the spectrum to ‘Npeaks’, the number of peaks to identify in the “CLEAN” spectrum as given on the command line. Note that all columns for a given peak are output together, and then the same quantities are repeated for the next peak, until all ‘Npeaks’ peaks have been reported. These columns are only reported if the CLEAN algorithm is performed, and the “findcleanpeaks” keyword is given.
DFTCLEAN_CSPEC_PEAK_POW_\$peaknum	— The value of the “CLEAN” power spectrum at peak number \$peaknum.
DFTCLEAN_CSPEC_PEAK_SNR_\$peaknum	— The spectroscopic S/N ratio for peak number \$peaknum in the “CLEAN” spectrum.
DFTCLEAN_CSPEC_AVESPEC	— The value of $\bar{P}$ used in computing the S/N ratio(s) for the clean power spectrum. This is output only if the “verboseout” keyword is given, and if the “findcleanpeaks” keyword is given with ‘Npeaks’ greater than 0.
DFTCLEAN_CSPEC_STDSPEC	— The value of σ_P used in computing the S/N ratio(s) for the clean power spectrum. This is output only if the “verboseout” keyword is given, and if the “findcleanpeaks” keyword is given with ‘Npeaks’ greater than 0.
DFTCLEAN_CSPEC_AVESPEC_NOCLIP	— The value of $\bar{P}$ for the clean spectrum when no clipping is applied. This is output only if the “verboseout” keyword is given, and if the “findcleanpeaks” keyword is given with ‘Npeaks’ greater than 0.
DFTCLEAN_CSPEC_STDSPEC_NOCLIP	— The value of σ_P for the clean spectrum when no clipping is applied. This is output only if the “verboseout” keyword is given, and if the “findcleanpeaks” keyword is given with ‘Npeaks’ greater than 0.

Table B.64

Output columns for “-difffluxtomag” command (Section 2.2.6.4).

No columns added to the output table.

Table B.65

Output columns for “-ensamplerescalesig” command (Section 2.2.6.5).

SigmaRescaleFactor	— The value of $\sqrt{\chi_{\text{new}}^2/\chi_{\text{old}}^2}$ where χ_{old}^2 and χ_{new}^2 are the reduced χ^2 values of the light curve before and after transforming the magnitude uncertainties, respectively.
--------------------	---

Table B.66

Output columns for “-expr” command (Section 2.2.6.6).

No columns added to the output table.

Table B.67

Output columns for “-findblends” command (Section 2.2.8.1).

FindBlends_Period	–	The period of the harmonic series fitted to the light curves.
FindBlends_LCname	–	The name of the light curve with the largest amplitude (in flux) harmonic series at the indicated period. This light curve is the most likely source for the measured variability.
FindBlends_FluxAmp	–	The amplitude (in flux) of the highest-amplitude light curve.

Table B.68

Output columns for “-fluxtomag” command (Section 2.2.6.7).

No columns added to the output table.

Table B.69

Output columns for “-GetLSAmpThresh” command (Section 2.2.1.2).

LS_AmplitudeScaleFactor	–	The scaling factor α for which the detected FAP would be equal to the threshold.
LS_MinimumAmplitude	–	The minimum peak-to-peak amplitude that the light curve could have had, and still have been detected by L–S with a FAP below the threshold.

Table B.70

Output columns for “-if” command (Section 2.2.7.2).

No columns added to the output table.

Table B.71

Output columns for “-Injectharm” command (Section 2.2.5.3).

Injectharm_Period	–	The period of the harmonic series added to the light curve.
Injectharm_Subharm_\$k_Amp	–	The value of B_k in Eq. (83) for $k = \$k$. Here $\$k$ varies from 2 to ‘Nsubharm’+1, where ‘Nsubharm’ is the number of sub-harmonics to use, as indicated on the command line. The amplitude and phase for sub-harmonic $\$k$ are output together, then the amplitude and phase of the next sub-harmonic are output, up to ‘Nsubharm’+1.
Injectharm_Subharm_\$k_Phase	–	The value of ψ_k in Eq. (83) for $k = \$k$.
Injectharm_Fundamental_Amp	–	The value of A_1 in Eq. (83).
Injectharm_Fundamental_Phase	–	The value of ϕ_1 in Eq. (83).
Injectharm_Harm_\$k_Amp	–	The value of A_k in Eq. (83) for $k = \$k$. Here $\$k$ varies from 2 to ‘Nharm’+1, where ‘Nharm’ is the number of harmonics to use, as indicated on the command line. The amplitude and phase for harmonic $\$k$ are output together, then the amplitude and phase of the next harmonic are output, up to ‘Nharm’+1.
Injectharm_Harm_\$k_Phase	–	The value of ϕ_k in Eq. (83) for $k = \$k$.

Table B.72

Output columns for “-Injecttransit” command (Section 2.2.5.4).

Injecttransit_Period	–	The period of the transit signal added to the light curve.
Injecttransit_Rp	–	The radius of the planet in Jupiter radii for the transit signal added to the light curve.
Injecttransit_Mp	–	The mass of the planet in Jupiter masses for the transit signal added to the light curve.
Injecttransit_phase	–	The orbital phase at time $t = 0$ (phase = 0 corresponds to transit center) for the transit signal added to the light curve.
Injecttransit_sin_i	–	The sine of the orbital inclination angle for the transit signal added to the light curve.
Injecttransit_e	–	The orbital eccentricity e for the transit signal added to the light curve. This is output only if the “eomega” keyword is used.
Injecttransit_omega	–	The argument of periastron ω (in degrees) for the transit signal added to the light curve. This is output only if the “eomega” keyword is used.
Injecttransit_h	–	The value of $e \sin \omega$ for the transit signal added to the light curve. This is output only if the “hk” keyword is used.
Injecttransit_k	–	The value of $e \cos \omega$ for the transit signal added to the light curve. This is output only if the “hk” keyword is used.
Injecttransit_Mstar	–	The mass of the star in Solar masses for the transit signal added to the light curve.
Injecttransit_Rstar	–	The radius of the star in Solar radii for the transit signal added to the light curve.
Injecttransit_ld_\$i	–	The value of limb darkening coefficient $\$i$ ($\$i$ takes values from 1 to 2 for the quadratic limb-darkening law, or from 1 to 4 for the non-linear limb-darkening law) for the transit signal added to the light curve.

Table B.73

Output columns for “-Jstet” command (Section 2.2.2.5).

Jstet	–	The computed value of Stetson’s J statistic (Eq. (42)).
Kurtosis	–	The estimated kurtosis of the light curve (Eq. (44)).
Lstet	–	The computed value of Stetson’s L statistic (Eq. (45)).

Table B.74

Output columns for “-Killharm” command (Section 2.2.3.2).

Killharm_Mean_Mag	–	The fitted value of m_0 from Eq. (48).
Killharm_Period_\$i	–	The value of the period for harmonic series $i = \$i + 1$ ($1/f_i$ in Eq. (48)). Here $\$i$ takes values from 1 to ‘Nper’, where ‘Nper’ is the number of periods indicated on the command-line (‘Nper’ is 1 for the “aov”, “ls” and “injectharm” options, it is 2 for the “both” option). All columns for a given value of $\$i$ are output before the columns for the next value of $\$i$ are output.
Killharm_Per\$Subharm_\$k_Sincoeff	–	The fitted value of $c_{i,k}$ from Eq. (48) where $k = \$k$. $\$k$ takes values from 2 to ‘Nsubharm’+1, where ‘Nsubharm’ is the number of sub-harmonics to use as specified on the command line. This is output only if none of the keywords “outampphase”, “outampradphase”, “outRphi” or “outRadphi” are given.
Killharm_Per\$Subharm_\$k_Coscoeff	–	The fitted value of $d_{i,k}$ from Eq. (48) where $k = \$k$. $\$k$ takes values from 2 to ‘Nsubharm’+1, where ‘Nsubharm’ is the number of sub-harmonics to use as specified on the command line. This is output only if none of the keywords “outampphase”, “outampradphase”, “outRphi” or “outRadphi” are given.
Killharm_Per\$Subharm_Amp_\$k	–	The fitted value of $B_{i,k}$ from Eq. (50). This is output only if one of the keywords “outampphase” or “outampradphase” is given.
Killharm_Per\$Subharm_Phi_\$k	–	The fitted value of $\psi_{i,k}$ from Eq. (54). This is output only if one of the keywords “outampphase” or “outampradphase” is given. If “outampphase” is given, then $\psi_{i,k}/(2\pi)$ is reported.
Killharm_Per\$Subharm_R_\$k_1	–	The fitted value of the relative amplitude $Q_{i,k,1}$ from Eq. (52). This is output only if the “outRphi” or “outRadphi” keyword is given.
Killharm_Per\$Subharm_Phi_\$k_1	–	The fitted value of the relative phase $\psi_{i,k,1}$ from Eq. (56). This is output only if the “outRphi” or “outRadphi” keyword is given. If “outRphi” is given, then $\psi_{i,k,1}/(2\pi)$ is reported.
Killharm_Per\$Fundamental_Sincoeff	–	The fitted value of $a_{i,1}$ from Eq. (48). This is output only if none of the keywords “outampphase”, “outampradphase”, “outRphi” or “outRadphi” are given.
Killharm_Per\$Fundamental_Coscoeff	–	The fitted value of $b_{i,1}$ from Eq. (48). This is output only if none of the keywords “outampphase”, “outampradphase”, “outRphi” or “outRadphi” are given.
Killharm_Per\$Fundamental_Amp	–	The fitted value of $A_{i,1}$ from Eq. (49). This is output only if one of the keywords “outampphase”, “outampradphase”, “outRphi” or “outRadphi” is given.
Killharm_Per\$Fundamental_Phi	–	The fitted value of $\phi_{i,1}$ from Eq. (53). This is output only if one of the keywords “outampphase”, “outampradphase”, “outRphi” or “outRadphi” is given. If “outampphase” or “outRphi” is given, then $\phi_{i,1}/(2\pi)$ is reported.
Killharm_Per\$Harm_\$k_Sincoeff	–	The fitted value of $a_{i,k}$ from Eq. (48) where $k = \$k$. $\$k$ takes values from 2 to ‘Nharm’+1, where ‘Nharm’ is the number of harmonics to use as specified on the command line. This is output only if none of the keywords “outampphase”, “outampradphase”, “outRphi” or “outRadphi” are given.
Killharm_Per\$Harm_\$k_Coscoeff	–	The fitted value of $b_{i,k}$ from Eq. (48) where $k = \$k$. $\$k$ takes values from 2 to ‘Nharm’+1, where ‘Nharm’ is the number of harmonics to use as specified on the command line. This is output only if none of the keywords “outampphase”, “outampradphase”, “outRphi” or “outRadphi” are given.
Killharm_Per\$Harm_Amp_\$k	–	The fitted value of $A_{i,k}$ from Eq. (49). This is output only if one of the keywords “outampphase” or “outampradphase” is given.
Killharm_Per\$Harm_Phi_\$k	–	The fitted value of $\phi_{i,k}$ from Eq. (53). This is output only if one of the keywords “outampphase” or “outampradphase” is given. If “outampphase” is given, then $\phi_{i,k}/(2\pi)$ is reported.
Killharm_Per\$Harm_R_\$k_1	–	The fitted value of the relative amplitude $R_{i,k,1}$ from Eq. (51). This is output only if the “outRphi” or “outRadphi” keyword is given.
Killharm_Per\$Harm_Phi_\$k_1	–	The fitted value of the relative phase $\phi_{i,k,1}$ from Eq. (55). This is output only if the “outRphi” or “outRadphi” keyword is given. If “outRphi” is given, then $\phi_{i,k,1}/(2\pi)$ is reported.
Killharm_Per\$Amplitude	–	The peak-to-peak amplitude of the full Fourier series (fundamental, sub-harmonics and harmonics) for period $\$i$.

Table B.75

Output columns for “-linfit” command (Section 2.2.3.3).

Linfit_\$paramname	–	The fitted value for parameter \$paramname, where \$paramname is the name of a variable in ‘paramlist’ supplied on the command line. The Linfit_\$paramname and Linfit_err\$paramname columns for each parameter \$paramname are output together, followed by the respective columns for the next value of \$paramname, until the results have been shown for all varied parameters.
Linfit_err\$paramname	–	The formal uncertainty for parameter \$paramname, this is accurate only if the noise in the light curve is Gaussian and uncorrelated in time, and if the measurement uncertainties supplied in the light curve are correct.

Table B.76

Output columns for “-LS” command (Section 2.2.1.1).

LS_Period_\$peaknum	–	The period for peak number \$peaknum. \$peaknum runs from 1 for the highest peak in the periodogram to ‘Npeaks’, the number of peaks to identify as given on the command line. Note that all columns for a given peak are output together, and then the same quantities are repeated for the next peak, until all ‘Npeaks’ peaks have been reported.
Log10_LS_Prob_\$peaknum	–	The base-10 logarithm of the false alarm probability for peak number \$peaknum, calculated using Eq. (3), or following Press et al. (1992) if the “noGLS” keyword is given.
LS_Periodogram_Value_\$peaknum	–	The value of the Generalized L–S periodogram (Eq. (1)) at peak number \$peaknum. If the “noGLS” keyword is given, this is instead the traditional normalized L–S periodogram value.
LS_SNR_\$peaknum	–	The spectroscopic S/N ratio for peak number \$peaknum (Eq. (6)).
LS_PeriodFix	–	The fixed period at which the Generalized L–S periodogram statistics are to be evaluated. This is output only if the “fixperiodSNR” keyword is given.
Log10_LS_Prob_PeriodFix	–	The base-10 logarithm of the false alarm probability at the fixed period. This is output only if the “fixperiodSNR” keyword is given.
LS_Periodogram_Value_PeriodFix	–	The value of the Generalized L–S periodogram at the fixed period. This is output only if the “fixperiodSNR” keyword is given.
LS_SNR_PeriodFix	–	The spectroscopic S/N at the fixed period. This is output only if the “fixperiodSNR” keyword is given.

Table B.77

Output columns for “-MandelAgolTransit” command (Section 2.2.3.4).

MandelAgolTransit_Period	–	The period of the transit signal. This is either the input value if the parameter is not varied, or the fitted parameter if it is. This also holds for the other parameters output by this command.
MandelAgolTransit_T0	–	The time of transit center.
MandelAgolTransit_r	–	The ratio of the planetary and stellar radii (R_p/R_*).
MandelAgolTransit_a	–	The ratio of the semimajor axis to the stellar radius (a/R_*).
MandelAgolTransit_bimpact	–	The normalized impact parameter of the transit (in units of $R_* + R_p$).
MandelAgolTransit_inc	–	The orbital inclination angle of the transiting system (90° is an edge on system giving central transits).
MandelAgolTransit_e	–	The orbital eccentricity.
MandelAgolTransit_omega	–	The argument of periastron in degrees.
MandelAgolTransit_mconst	–	The out-of-transit magnitude.
MandelAgolTransit_Idcoeff1	–	The first limb darkening coefficient.
MandelAgolTransit_Idcoeff2	–	The second limb darkening coefficient.
MandelAgolTransit_Idcoeff3	–	The third limb darkening coefficient. This is output only if the nonlinear limb darkening law is used.
MandelAgolTransit_Idcoeff4	–	The fourth limb darkening coefficient. This is output only if the nonlinear limb darkening law is used.
MandelAgolTransit_K	–	The semi-amplitude of the RV orbit. This is output only if the fitRV flag is set to 1.
MandelAgolTransit_gamma	–	The zero-point velocity of the RV orbit. This is output only if the fitRV flag is set to 1.
MandelAgolTransit_chi2	–	χ^2 per degree of freedom about the best-fit model. This includes the RVs if they are fitted.

Table B.78

Output columns for “-medianfilter” command (Section 2.2.4.2).

No columns added to the output table.

Table B.79

Output columns for “-microlens” command (Section 2.2.3.5).

Microlens_f0	–	The parameter f_0 in Eq. (63). This is either the input value if the parameter is not varied (the “novary” keyword was used for this parameter), or the fitted parameter if it is. This also holds for the other parameters output by this command.
Microlens_f1	–	The parameter f_1 in Eq. (63)
Microlens_u0	–	The parameter u_0 in Eq. (65).
Microlens_t0	–	The parameter t_0 in Eq. (65).
Microlens_tmax	–	The parameter $t_{\max}$ in Eq. (65).
Microlens_chi2perdof	–	χ^2 per degree of freedom about the best-fit microlens model.

Table B.80

Output columns for “-nonlinfit” command (Section 2.2.3.6).

Nonlinfit_HasConverged	–	This is “1” if the DHSX fitting procedure converged to a solution, or “0” if the routine terminated before convergence had been achieved. The parameter is output only if the “amoeba” fitting mode is used.
Nonlinfit_\$paramname[\$i]_BestFit	–	The best-fit (minimum X^2 ; Eq. (66)) value for \$paramname[\$i], the name of the \$ith parameter listed in ‘paramlist’ (together with ‘linfitparams’ if the “linfit” keyword is used) given on the command line. Here \$i runs from 1 to the total number of parameters listed in ‘paramlist’ and ‘linfitparams’, and all columns for a given parameter are output before continuing to the next parameter in the sequence.
Nonlinfit_\$paramname[\$i]_Err	–	The formal uncertainty on \$paramname[\$i] determined by finding the size of the perturbation to the parameter which leads to $\Delta X^2 = 1$ from the best-fit solution, when all other parameters are held constant. This is output only if the “amoeba” fitting mode is used.
Nonlinfit_BestFit_Chi2	–	The value of X^2 for the best-fit model.
Nonlinfit_\$expression[\$i]_Stats[\$j]	–	The result of calculating the statistic \$stats[\$j] on the expression \$expression[\$i] evaluated on the DEMCMC chains. This is output only if the “mcmc” fitting mode is used. Here \$expression[\$i] is the \$ith expression listed in ‘exprlist’ on the command-line if the “chainstats” keyword is used, or the \$ith parameter name in ‘paramlist’ if the “chainstats” keyword is not used. The term \$stats[\$j] is a string based on the \$jth statistic in ‘statslist’ (if the “chainstats” keyword is not given then ‘median, stddev’ is assumed for ‘statslist’). A separate column is output for each combination of \$i and \$j, with the loop over \$j nested within the loop over \$i. The possible values for \$stats[\$j] are: “MEAN”, “WEIGHTEDMEAN”, “MEDIAN”, “WEIGHTEDMEDIAN”, “STDDEV”, “MEDDEV”, “MEDMEDDEV”, “MAD”, “KURTOSIS”, “SKEWNESS”, “PCTSpctval” (where \$pctval is the percentile value to two decimal places), “WPCTSpctval”, “MAX”, “MIN” and “SUM”. These terms are output only if the “mcmc” fitting mode is used.

Table B.81

Output columns for “-o” command (Section 2.2.7.3).

No columns added to the output table.

Table B.82

Output columns for “-Phase” command (Section 2.2.6.8).

No columns added to the output table.

Table B.83

Output columns for “-resample” command (Section 2.2.6.9).

No columns added to the output table.

Table B.84

Output columns for “-rescalesig” command (Section 2.2.6.10).

SigmaRescaleFactor	—	The factor by which the magnitude uncertainties are scaled to achieve $\chi^2/\text{dof} = 1$ for that light curve.
--------------------	---	---

Table B.85

Output columns for “-restorelc” command (Section 2.2.7.4).

No columns added to the output table.

Table B.86

Output columns for “-restricttimes” command (Section 2.2.4.3).

RestrictTimes_MinJD	—	The minimum JD used for restricting the light curve times. This is output only if the “JDrange” or “JDrangebylc” keyword is used.
RestrictTimes_MaxJD	—	The maximum JD used for restricting the light curve times. This is output only if the “JDrange” or “JDrangebylc” keyword is used.

Table B.87

Output columns for “-rms” command (Section 2.2.2.6).

Mean_Mag	—	The mean of the light curve magnitudes.
RMS	—	The r.m.s. scatter of the light curve magnitudes.
Expected_RMS	—	The expected r.m.s. scatter of the light curve magnitudes based on the magnitude uncertainties (Eq. (47)).
Npoints	—	The number of points in the light curve which contribute to r.m.s. measurement. Only points with magnitude values that are not NaN, and which have magnitude uncertainties greater than zero contribute.

Table B.88

Output columns for “-rmsbin” command (Section 2.2.2.7).

RMSBin_\$bintime	—	The r.m.s. scatter of the light curve magnitudes after binning the light curve with bintime \$bintime. The value of \$bintime is displayed to two decimal places. Note that both columns for a given \$bintime (RMSBin and Expected_RMS_Bin) are output together, and then the same quantities are repeated for the next \$bintime, until the results for all ‘Nbin’ values of \$bintime given on the command line have been reported.
Expected_RMS_Bin_\$bintime	—	The expected r.m.s. scatter of the light curve magnitudes, based on the magnitude uncertainties, after binning with by \$bintime.

Table B.89

Output columns for “-savelc” command (Section 2.2.7.4).

No columns added to the output table.

Table B.90

Output columns for “-SoftenedTransit” command (Section 2.2.3.7).

SoftenedTransit_Period	–	The period for the transit model (P in Eq. (74)). This is either the input value if the parameter is not varied, or the fitted parameter if it is. This also holds for the other parameters output by this command.
SoftenedTransit_T0	–	The transit epoch (T_0 in Eq. (74)).
SoftenedTransit_eta	–	The transit duration (η in Eq. (74)).
SoftenedTransit_cval	–	The transit sharpness (c in Eq. (73)).
SoftenedTransit_delta	–	The transit depth (δ in Eq. (73)).
SoftenedTransit_mconst	–	The out-of-transit magnitude (M_0 in Eq. (73)).
SoftenedTransit_perharm	–	The period for the harmonic series fit simultaneously to the light curve with the transit model (this is included only if the ‘fit_harm’ flag is set to ‘1’).
SoftenedTransit_Subharm_\${k}_Sincoeff	–	Coefficient of the sine function for sub-harmonic k where k varies from 2 to ‘nsubharm’+1 (i.e., this is $c_{1,k}$ in Eq. (48) with $N_p = 1$). This is output only if the ‘fit_harm’ flag is set to ‘1’.
SoftenedTransit_Subharm_\${k}_Coscoeff	–	Coefficient of the cosine function for sub-harmonic k where k varies from 2 to ‘nsubharm’+1 (i.e., this is $d_{1,k}$ in Eq. (48) with $N_p = 1$). This is output only if the ‘fit_harm’ flag is set to ‘1’.
SoftenedTransit_Fundamental_Sincoeff	–	Coefficient of the sine function for the fundamental mode in the harmonic series fit simultaneously to the transit model (i.e., this is $a_{1,1}$ in Eq. (48) with $N_p = 1$). This is output only if the ‘fit_harm’ flag is set to ‘1’.
SoftenedTransit_Fundamental_Coscoeff	–	Coefficient of the cosine function for the fundamental mode in the harmonic series fit simultaneously to the transit model (i.e., this is $b_{1,1}$ in Eq. (48) with $N_p = 1$). This is output only if the ‘fit_harm’ flag is set to ‘1’.
SoftenedTransit_Harm_\${k}_Sincoeff	–	Coefficient of the sine function for harmonic k where k varies from 2 to ‘nharm’+1 (i.e., this is $a_{1,k}$ in Eq. (48) with $N_p = 1$). This is output only if the ‘fit_harm’ flag is set to ‘1’.
SoftenedTransit_Harm_\${k}_Coscoeff	–	Coefficient of the cosine function for harmonic k where k varies from 2 to ‘nharm’+1 (i.e., this is $b_{1,k}$ in Eq. (48) with $N_p = 1$). This is output only if the ‘fit_harm’ flag is set to ‘1’.
SoftenedTransit_chi2perdof	–	χ^2 per degree of freedom for the best-fit model.

Table B.91

Output columns for “-Starspot” command (Section 2.2.3.8).

Starspot_Period	–	The rotation period of the star. This is either the input value if the parameter is not varied, or the fitted parameter if it is. This also holds for the other parameters output by this command.
Starspot_a	–	The parameter a in Eq. (75).
Starspot_b	–	The parameter b in Eq. (75).
Starspot_alpha	–	The angular radius of the starspot in degrees.
Starspot_inclination	–	The inclination angle of the stellar rotation axis in degrees (90° corresponds to the rotation axis being perpendicular to the line-of-sight).
Starspot_chi	–	The spot latitude in degrees (0° for a spot at the equator).
Starspot_psi0	–	The longitude of the spot center at the first time instance in the light curve, in degrees.
Starspot_mconst	–	The constant magnitude term (M_0 in Eq. (75)).
Starspot_chi2perdof	–	χ^2 per degree of freedom for the best-fit model.

Table B.92

Output columns for “-stats” command (Section 2.2.2.8).

STATS_\${varname}[\${i}]_\${stats}[\${j}]	–	The statistic $\$stats[{\$j}]$ calculated for variable $\$varname[{\$i}]$. Here $\$varname[{\$i}]$ is the i th variable listed in ‘var1, var2, ...’ on the command-line. The term $\$stats[{\$j}]$ is a string based on the j th statistic in ‘stats1, stats2, ...’. A separate column is output for each combination of i and j , with the loop over j nested within the loop over i . The possible values for $\$stats[{\$j}]$ are: “MEAN”, “WEIGHTEDMEAN”, “MEDIAN”, “WEIGHTEDMEDIAN”, “STDDEV”, “MEDDEV”, “MEDMEDDEV”, “MAD”, “KURTOSIS”, “SKEWNESS”, “PCT\${pctval}” (where $\$pctval$ is the percentile value to two decimal places), “WPCT\${pctval}”, “MAX”, “MIN” and “SUM”.
---	---	--

Table B.93

Output columns for “-SYSREM” command (Section 2.2.4.4).

SYSREM_MeanMag	—	The mean magnitude of the light curve.
SYSREM_Trend_\$i_Coeff	—	The output value for the color term for the \$i\$th corrected trend. Here \$i\$ ranges from 0 to 'Ninput_color'+'Ninput_airmass'–1, with the coefficients from the trends with airmass terms specified initially listed before those from the trends with color terms specified initially.
SYSREM_RMS	—	The r.m.s. scatter of the light curve after removing the trends.

Table B.94

Output columns for “-TFA” command (Section 2.2.4.5).

TFA_MeanMag	—	The mean magnitude of the light curve.
TFA_RMS	—	The r.m.s. scatter of the light curve after removing the trends.

Table B.95

Output columns for “-TFA_SR” command (Section 2.2.4.6).

TFA_SR_MeanMag	—	The mean magnitude of the light curve.
TFA_SR_RMS	—	The r.m.s. scatter of the light curve after removing the trends.

Table B.96

Output columns for “-wwz” command (Section 2.2.1.9).

MaxWWZ	—	The maximum value of Z (Eq. (28)) over all values of time-shifts τ and frequencies f scanned.
MaxWWZ_Freq	—	The value of f at which Z is maximized.
MaxWWZ_TShift	—	The value of τ at which Z is maximized.
MaxWWZ_Power	—	The value of the WWT statistic (Eq. (27)) at the maximum Z peak.
MaxWWZ_Amplitude	—	The value of the Weighted-Wavelet-Amplitude (WWA; Eq. (32)) at the maximum Z peak.
MaxWWZ_Neffective	—	The effective number of data points contributing to the signal at the maximum Z peak (Eq. (29)).
MaxWWZ_AverageMag	—	The mean magnitude of the signal at the maximum Z peak.
Med_WWZ	—	The median value of $Z_{\max}(\tau)$, where $Z_{\max}(\tau)$ is the maximum value of Z over all frequencies, at a fixed value of τ , and the median is then taken over all τ values.
Med_Freq	—	The median value of $f_{Z\max}(\tau)$, where $f_{Z\max}(\tau)$ is the frequency at which Z is maximum at a fixed value of τ , and the median is then taken over all τ values.
Med_Power	—	The median value of $WWT_{Z\max}(\tau)$, where $WWT_{Z\max}(\tau)$ is the value of WWT (Eq. (27)) calculated at the Z maximum for fixed τ , and the median is then taken over all τ values.
Med_Amplitude	—	The median value of $WWA_{Z\max}(\tau)$, where $WWA_{Z\max}(\tau)$ is the value of WWA (Eq. (32)) calculated at the Z maximum for fixed τ , and the median is then taken over all τ values.
Med_Neffective	—	The median value of $N_{\text{eff},Z\max}(\tau)$, where $N_{\text{eff},Z\max}(\tau)$ is the value of N_{eff} (Eq. (29)) calculated at the Z maximum for fixed τ , and the median is then taken over all τ values.
Med_AverageMag	—	The median value of $M_{\text{ave},Z\max}(\tau)$, where $M_{\text{ave},Z\max}(\tau)$ is the average magnitude of the signal calculated at the Z maximum for fixed τ , and the median is then taken over all τ values.

References

- Acton, C.H., 1996. Ancillary data services of NASA's navigation and ancillary information facility. *Planet. Space Sci.* 44, 65–70. doi:10.1016/0032-0633(95)00107-7.
- Alard, C., Lupton, R.H., 1998. A method for optimal image subtraction. *Astrophys. J.* 503, 325. doi:10.1086/305984, arXiv:astro-ph/9712287.
- Bakos, G.Á., Csabry, Z., Penev, K., Bayliss, D., Jordán, A., Afonso, C., Hartman, J.D., Henning, T., Kovács, G., Noyes, R.W., Béky, B., Suc, V., Csák, B., Rabus, M., Lázár, J., Papp, I., Sári, P., Conroy, P., Zhou, G., Sackett, P.D., Schmidt, B., Mancini, L., Sasselov, D.D., Ueltzhoeffer, K., 2013. HATSouth: a global network of fully automated identical wide-field telescopes. *Publ. Astron. Soc. Pac.* 125, 154. doi:10.1086/669529.
- Bond, I.A., Udalski, A., Jaroszyński, M., Rattenbury, N.J., Paczyński, B., Soszyński, I., Wyrzykowski, L., Szymański, M.K., Kubiak, M., Szewczyk, O., Żebruń, K., Pietrzyński, G., Abe, F., Bennett, D.P., Eguchi, S., Furuta, Y., Hearnshaw, J.B., Kamiya, K., Kilmartin, P.M., Kurata, Y., Masuda, K., Matsubara, Y., Muraki, Y., Noda, S., Okajima, K., Sako, T., Sekiguchi, T., Sullivan, D.J., Sumi, T., Tristram, P.J., Yanagisawa, T., Yock, P.C.M., OGLE Collaboration, 2004. OGLE 2003-BLG-235/MOA 2003-BLG-53: A Planetary Microlensing Event. *Astrophys. J.* 606, L155–L158. doi:10.1086/420928, arXiv:astro-ph/0404309.
- Borucki, W.J., Koch, D., Basri, G., Batalha, N., Brown, T., Caldwell, D., Caldwell, J., Christensen-Dalsgaard, J., Cochran, W.D., DeVore, E., Dunham, E.W., Dupree, A.K., Gautier, T.N., Geary, J.C., Gilliland, R., Gould, A., Howell, S.B., Jenkins, J.M., Kondo, Y., Latham, D.W., Marcy, G.W., Meibom, S., Kjeldsen, H., Lissauer, J.J., Monet, D.G., Morrison, D., Sasselov, D., Tarter, J., Boss, A., Brownlee, D., Owen, T., Buzasi, D., Charbonneau, D., Doyle, L., Fortney, J., Ford, E.B., Holman, M.J., Seager, S., Steffen, J.H., Welsh, W.F., Rowe, J., Anderson, H., Buchhave, L., Ciardi, D., Walkowicz, L., Sherry, W., Horch, E., Isaacson, H., Everett, M.E., Fischer, D., Torres, G., Johnson, J.A., Endl, M., MacQueen, P., Bryson, S.T., Dotson, J., Haas, M., Kolodziejczak, J., Van Cleve, J., Chandrasekaran, H., Twicken, J.D., Quintana, E.V., Clarke, B.D., Allen, C., Li, J., Wu, H., Tenenbaum, P., Verner, E., Bruhweiler, F., Barnes, J., Prsa, A., 2010. Kepler planet-detection mission: Introduction and first results. *Science* 327, 977–980. doi:10.1126/science.1185402.
- Charbonneau, D., Brown, T.M., Latham, D.W., Mayor, M., 2000. Detection of planetary transits across a sun-like star. *Astrophys. J.* 529, L45–L48. doi:10.1086/312457, arXiv:astro-ph/9911436.
- Cumming, A., Marcy, G.W., Butler, R.P., 1999. The lick planet search: Detectability and mass thresholds. *Astrophys. J.* 526, 890–915. doi:10.1086/308020, arXiv:astro-ph/9906466.
- Devor, J., 2005. Solutions for 10,000 eclipsing binaries in the bulge fields of OGLE II using DEBI. *Astrophys. J.* 628, 411–425. doi:10.1086/431170, arXiv:astro-ph/0504399.
- Dorren, J.D., 1987. A new formulation of the starspot model, and the consequences of starspot structure. *Astrophys. J.* 320, 756–767. doi:10.1086/165593.
- Eastman, J., Siverd, R., Gaudi, B.S., 2010. Achieving better than 1 minute accuracy in the heliocentric and barycentric julian dates. *PASP* 122, 935–946. doi:10.1086/655938, arXiv:1005.4415.
- Edelson, R.A., Krolik, J.H., 1988. The discrete correlation function - a new method for analyzing unevenly sampled variability data. *Astrophys. J.* 333, 646–659. doi:10.1086/166773.
- Etzel, P.B., 1981. A simple synthesis method for solving the elements of well-detached eclipsing systems. In: Carling, E.B., Kopal, Z. (Eds.), *Photometric and Spectroscopic Binary Systems*, p. 111.
- Foster, G., 1996. Wavelets for period analysis of unevenly sampled time series. *Astron. J.* 112, 1709. doi:10.1086/118137.
- Galassi, M., Davies, J., Theiler, J., Gough, B., Jungman, G., Alken, P., Booth, M., Rossi, F., 2009. *GNU Scientific Library Reference Manual*, third ed. Network Theory Ltd.
- Goupillaud, P., Grossmann, A., Morlet, J., 1984. Cycle-octave and related transforms in seismic signal analysis. *Geophysical J.* 85, 85–102. doi:10.1016/0016-7142(84)90025-5.
- Handcock, M.S., Stein, M.L., 1993. A bayesian analysis of kriging. *Technometrics* 35, 403–410. doi:10.1080/00401706.1993.10485354, URL: <http://amstat.tandfonline.com/doi/abs/10.1080/00401706.1993.10485354>, arXiv:10.1080/00401706.1993.10485354.
- Hartman, J.D., Bakos, G.Á., Noyes, R.W., Sipőcz, B., Kovács, G., Mazeh, T., Shporer, A., Pál, A., 2011. A photometric variability survey of field K and M dwarf stars with HATNet. *Astron. J.* 141, 166. doi:10.1088/0004-6256/141/5/166.
- Hartman, J.D., Gaudi, B.S., Holman, M.J., McLeod, B.A., Stanek, K.Z., Barranco, J.A., Pinsonneault, M.H., Kalirai, J.S., 2008. Deep MMT transit survey of the open cluster M37. II. variable stars. *Astrophys. J.* 675, 1254–1277. doi:10.1086/527460, arXiv:0709.3484.
- Hartman, J.D., Stanek, K.Z., Gaudi, B.S., Holman, M.J., McLeod, B.A., 2005. Pushing the limits of ground-based photometric precision: Submillimagnitude time-series photometry of the open cluster NGC 6791. *Astron. J.* 130, 2241–2251. doi:10.1086/462405, arXiv:astro-ph/0504487.
- Henry, G.W., Marcy, G., Butler, R.P., Vogt, S.S., 1999. HD 209458. *IAU Circ.* 7307, 1.
- Horne, J.H., Baliunas, S.L., 1986. A prescription for period analysis of unevenly sampled time series. *Astrophys. J.* 302, 757–763. doi:10.1086/164037.
- Hubble, E.P., 1929. A spiral nebula as a stellar system, Messier 31. *Astrophys. J.* 69, 103–158. doi:10.1086/143167.
- Kipping, D.M., 2012. An analytic model for rotational modulations in the photometry of spotted stars. *Mon. Not. R. Astron. Soc.* 427, 2487–2511. doi:10.1111/j.1365-2966.2012.1224.x, arXiv:1209.2985.
- Konacki, M., Torres, G., Jha, S., Sasselov, D.D., 2003. An extrasolar planet that transits the disk of its parent star. *Nature* 421, 507–509. doi:10.1038/nature01379.
- Kopal, Z., 1959. *Close Binary Systems*. The International Astrophysics Series, London: Chapman and Hall, 1959.
- Kovacs, G., 1980. Period analysis at high noise level. *Astrophys. Space Sci.* 69, 485–493. doi:10.1007/BF00661932.
- Kovács, G., Bakos, G., Noyes, R.W., 2005. A trend filtering algorithm for wide-field variability surveys. *Mon. Not. R. Astron. Soc.* 356, 557–567. doi:10.1111/j.1365-2966.2004.08479.x, arXiv:astro-ph/0411724.
- Kovács, G., Zucker, S., Mazeh, T., 2002. A box-fitting algorithm in the search for periodic transits. *Astron. Astrophys.* 391, 369–377. doi:10.1051/0004-6361:20020802, arXiv:astro-ph/0206099.
- Kurtz, D.W., 1985. An algorithm for significantly reducing the time necessary to compute a discrete fourier transform periodogram of unequally spaced data. *Mon. Not. R. Astron. Soc.* 213, 773–776.
- Lenz, P., Breger, M., 2005. Period04 user guide. *Communications in Asteroseismology* 146, 53–136. doi:10.1553/cia146s53.
- Lomb, N.R., 1976. Least-squares frequency analysis of unequally spaced data. *Astrophys. Space Sci.* 39, 447–462. doi:10.1007/BF00648343.
- Mandel, K., Agol, E., 2002. Analytic light curves for planetary transit searches. *Astrophys. J.* 580, L171–L175. doi:10.1086/345520, arXiv:astro-ph/0210099.
- McCoy, E.J., Walden, A.T., 1996. Wavelet analysis and synthesis of stationary long-memory processes. *J. Comput. Graph. Statist.* 5, 26–56. URL: <http://www.jstor.org/stable/1390751>.
- Nelder, J.A., Mead, R., 1965. A simplex method for function minimization. *Comput. J.* 7, 308–313. doi:10.1093/comjnl/7.4.308.
- Nelson, B., Davis, W.D., 1972. Eclipsing-binary solutions by sequential optimization of the parameters. *Astrophys. J.* 174, 617. doi:10.1086/151524.
- Paczynski, B., 1986. Gravitational microlensing by the galactic halo. *Astrophys. J.* 304, 1–5. doi:10.1086/164140.
- Palmer, D.M., 2009. A fast chi-squared technique for period search of irregularly sampled data. *Astrophys. J.* 695, 496–502. doi:10.1088/0004-637X/695/1/496, arXiv:0901.1913.
- Paltani, S., 2004. Searching for periods in X-ray observations using Kuiper's test. Application to the ROSAT PSPC archive. *Astron. Astrophys.* 420, 789–797. doi:10.1051/0004-6361:20034220, arXiv:astro-ph/0403186.
- Paunzen, E., Vanmunster, T., 2016. Peranso - light curve and period analysis software. *Astron. Nachr.* 337, 239. doi:10.1002/asna.201512254.
- Pence, W., 1999. CFITSIO, v2.0: A new full-featured data interface. In: Mehringer, D.M., Plante, R.L., Roberts, D.A. (Eds.), *Astronomical Data Analysis Software and Systems VIII*, p. 487.
- Perlmutter, S., Aldering, G., Goldhaber, G., Knop, R.A., Nugent, P., Castro, P.G., Deustua, S., Fabbro, S., Goobar, A., Groom, D.E., Hook, I.M., Kim, A.G., Kim, M.Y., Lee, J.C., Nunes, N.J., Pain, R., Pennypacker, C.R., Quimby, R., Lidman, C., Ellis, R.S., Irwin, M., McMahon, R.G., Ruiz-Lapuente, P., Walton, N., Schaefer, B., Boyle, B.J., Filippenko, A.V., Matheson, T., Fruchter, A.S., Panagia, N., Newberg, H.J.M., Couch, W.J., Supernova Cosmology Project, 1999. Measurements of omega and lambda from 42 high-redshift supernovae. *Astrophys. J.* 517, 565–586. doi:10.1086/307221, arXiv:astro-ph/9812133.
- Pont, F., Zucker, S., Queloz, D., 2006. The effect of red noise on planetary transit detection. *Mon. Not. R. Astron. Soc.* 373, 231–242. doi:10.1111/j.1365-2966.2006.11012.x, arXiv:astro-ph/0608597.
- Popper, D.M., Etzel, P.B., 1981. Photometric orbits of seven detached eclipsing binaries. *Astron. J.* 86, 102–120. doi:10.1086/112862.
- Press, W.H., Rybicki, G.B., 1989. Fast algorithm for spectral analysis of unevenly sampled data. *Astrophys. J.* 338, 277–280. doi:10.1086/167197.
- Press, W.H., Teukolsky, S.A., Vetterling, W.T., Flannery, B.P., 1992. *Numerical Recipes in C: The Art of Scientific Computing*, second ed. University Press, Cambridge, C1992.
- Protopapas, P., Jimenez, R., Alcock, C., 2005. Fast identification of transits from light-curves. *Mon. Not. R. Astron. Soc.* 362, 460–468. doi:10.1111/j.1365-2966.2005.09305.x, arXiv:astro-ph/0502301.
- Richards, M., Whitby-Stevens, C., 1979. *BCPL, the Language and Its Compiler*. Cambridge: Cambridge University Press, 1979.
- Riess, A.G., Filippenko, A.V., Challis, P., Clocchiatti, A., Diercks, A., Garnavich, P.M., Gilliland, R.L., Hogan, C.J., Jha, S., Kirshner, R.P., Leibundgut, B., Phillips, M.M., Reiss, D., Schmidt, B.P., Schommer, R.A., Smith, R.C., Spyromilio, J., Stubbs, C., Suntzeff, N.B., Tonry, J., 1998. Observational evidence from supernovae for an accelerating universe and a cosmological constant. *Astron. J.* 116, 1009–1038. doi:10.1086/300499, arXiv:astro-ph/9805201.
- Roberts, D.H., Lehar, J., Dreher, J.W., 1987. Time series analysis with clean - part one - derivation of a spectrum. *Astron. J.* 93, 968. doi:10.1086/114383.
- Scargle, J.D., 1982. Studies in astronomical time series analysis. II - Statistical aspects of spectral analysis of unevenly spaced data. *Astrophys. J.* 263, 835–853. doi:10.1086/160554.
- Schwarzenberg-Czerny, A., 1989. On the advantage of using analysis of variance for period search. *Mon. Not. R. Astron. Soc.* 241, 153–165.
- Schwarzenberg-Czerny, A., 1996. Fast and statistically optimal period search in uneven sampled observations. *Astrophys. J.* 460, L107. doi:10.1086/309985.
- Schwarzenberg-Czerny, A., 2012. On the sensitivity of period searches. In: Griffin, E., Hanisch, R., Seaman, R. (Eds.), *IAU Symposium*, pp. 81–86. doi:10.1017/S1743921312000294.
- Southworth, J., Maxted, P.F.L., Smalley, B., 2004. Eclipsing binaries in open clusters - II. V453 Cyg in NGC 6871. *Mon. Not. R. Astron. Soc.* 351, 1277–1289. doi:10.1111/j.1365-2966.2004.07871.x, arXiv:astro-ph/0403572.
- Steffen, M., 1990. A simple method for monotonic interpolation in one dimension. *Astron. Astrophys.* 239, 443.
- Stellingwerf, R.F., 1978. Period determination using phase dispersion minimization. *Astrophys. J.* 224, 953–960. doi:10.1086/156444.
- Stetson, P.B., 1996. On the automatic determination of light-curve parameters for cepheid variables. *PASP* 108, 851. doi:10.1086/133808.

- Still, M., Barclay, T., 2012. PyKE: Reduction and analysis of Kepler Simple Aperture Photometry data. [arXiv:1208.004](#). Astrophysics Source Code Library.
- Tamuz, O., Mazeh, T., North, P., 2006. Automated analysis of eclipsing binary light curves - I. EBAS - a new Eclipsing Binary Automated Solver with EBOP. *Mon. Not. R. Astron. Soc.* 367, 1521–1530. [doi:10.1111/j.1365-2966.2006.10049.x](#), [arXiv:astro-ph/0601199](#).
- Tamuz, O., Mazeh, T., Zucker, S., 2005. Correcting systematic effects in a large set of photometric light curves. *Mon. Not. R. Astron. Soc.* 356, 1466–1470. [doi:10.1111/j.1365-2966.2004.08585.x](#), [arXiv:astro-ph/0502056](#).
- ter Braak, C.J.F., 2006. A Markov Chain Monte Carlo version of the genetic algorithm differential evolution: easy Bayesian computing for real parameter spaces. *Stat. Comput.* 16, 239–249.
- Thompson, S.E., Mullally, F., 2009. Wqed: A lightcurve analysis suite. *J. Phys. Conf. Ser.* 172, 012081. [doi:10.1088/1742-6596/172/1/012081](#).
- Torres, G., Andersen, J., Giménez, A., 2010. Accurate masses and radii of normal stars: modern results and applications. *Astron. Astrophys. Rev.* 18, 67–126. [doi:10.1007/s00159-009-0025-1](#), [arXiv:0908.2624](#).
- Wozniak, P.R., Udalski, A., Szymanski, M., Kubiak, M., Pietrzynski, G., Soszynski, I., Zebrun, K., 2001. Difference image analysis of the OGLE-II bulge data. II. Microlensing events. *Acta Astronomica* 51, 175–219. [arXiv:astro-ph/0106474](#).
- Zechmeister, M., Kürster, M., 2009. The generalised Lomb-Scargle periodogram. A new formalism for the floating-mean and Keplerian periodograms. *Astron. Astrophys.* 496, 577–584. [doi:10.1051/0004-6361:200811296](#), [arXiv:0901.2573](#).